Scientific Computing

Aditya Dendukuri

UC Santa Barbara

Contents

Preface

These notes started as a study guide for CS 111 at UC Santa Barbara and grew through my PhD as teaching and research kept exposing things I thought I understood until I tried to explain them: why LU factorization is numerically stable, why the QR algorithm converges, what the Koopman operator actually is. Writing forces the gaps to surface. The result is a reference written by a graduate student for himself. Treat it accordingly.

For a rigorous treatment of this material, see Principles of Scientific Computing by Bindel and Goodman, and G. W. Stewart's Afternotes on Numerical Analysis. For further reading, the CS 6241 reading list at Cornell covers the literature well.

Computers were built to do numerical computation. The word "computer" originally meant a person whose job was to evaluate functions by hand. The U.S. Army employed hundreds of them during the Second World War to compute artillery firing tables, a task so large it directly motivated the construction of ENIAC (1945). The machine's first classified application was thermonuclear weapon simulation, directed by John von Neumann, whose 1947 paper with Goldstine laid the foundations of numerical stability analysis. In 1950, Jule Charney used ENIAC to run the first machine weather forecast: Lewis Fry Richardson had estimated the same computation would require 64,000 human computers working in parallel. The Apollo guidance computer navigated to the Moon in real time with 4 kilobytes of memory. The Cooley-Tukey FFT (1965) reduced a core signal processing computation from $O(n^2)$ to $O(n \log n)$. The demand for numerical computation at scale was the original motivation for programmable machines.

Almost every numerical algorithm reduces a nonlinear problem to a sequence of linear ones, because linear systems are the only class of problems we understand completely: their solution structure, their sensitivity to perturbations, their reliable algorithms. Newton's method, finite element discretization, PCA, gradient-based optimization, each is, at its core, a repeated linearization. Linear algebra is the substrate of scientific computing.

Much of the perspective here came from **CS 210A: Matrix Analysis**, taught by Professor Shivkumar Chandrasekaran in the ECE department at UC Santa Barbara. If you have the chance to take that course, take it.

Aditya Dendukuri
UC Santa Barbara, 2025

1 Fundamentals

1.1 Mathematical Notation and Proof Techniques

Standard vocabulary for proofs and algorithms.

1.1.1 Logic and Set Notation

Def 1.1 **(Logical Quantifiers)**

- **Universal ($\forall$):** "For all."
- **Existential ($\exists$):** "There exists."
- **Example:** "$\forall \varepsilon > 0,\ \exists \delta > 0$" reads as "for every positive ε there is a positive δ."

Def 1.2 **(Implication and Equivalence)**

- **Implication ($P \Rightarrow Q$):** P implies Q.
- **Equivalence ($P \Leftrightarrow Q$):** P if and only if Q (iff).
- **Contrapositive:** $\neg Q \Rightarrow \neg P$. Logically equivalent to $P \Rightarrow Q$.

Def 1.3 **(Set Notation)**

- $x \in S$: x is an element of S.
- $S \subseteq T$: S is a subset of T.
- $S \cup T$: Union; $S \cap T$: Intersection; $S \setminus T$: Set difference.
- $\{x \in S : P(x)\}$: Set builder notation.
- **Number Sets:** $\mathbb{Z}$ (integers), $\mathbb{R}$ (reals), $\mathbb{C}$ (complex), $\mathbb{R}^n$ (vectors), $\mathbb{R}^{m \times n}$ (matrices).

Ex 1.1

1. Write "every invertible matrix has a nonzero determinant" using quantifiers and implication.
2. State the contrapositive of: "if $\mathbf{A}$ is invertible, then $\mathbf{A}\boldsymbol{x} = \mathbf{0} \Rightarrow \boldsymbol{x} = \mathbf{0}$."
3. Let $S = \{1, 2, 3, 4\}$ and $T = \{3, 4, 5, 6\}$. Compute $S \cup T, S \cap T, S \setminus T$.

1.1.2 Proof Techniques

Def 1.4 **(Direct Proof)** Assume P is true and derive Q through a chain of valid logical steps.

Def 1.5 **(Proof by Contradiction)** Assume $\neg P$ and derive a logical contradiction $(R \wedge \neg R)$. Proves P must hold.

Def 1.6 **(Proof by Induction)** To prove $P(n)$ holds for all integers $n \geq n_0$:

1. **Base case:** Verify $P(n_0)$ directly.
2. **Inductive step:** Assume $P(k)$ and prove $P(k+1)$.

Remark 1.1 **Numerical Proofs:** Often validate that an approximation is stable or that an error term converges. Frequently involves showing $|f(x) - \hat{f}(x)| \leq \varepsilon$ for small ε.

Ex 1.2

1. Prove: if $\mathbf{A}$ and $\mathbf{B}$ are symmetric, then $\mathbf{A} + \mathbf{B}$ is symmetric.
2. Prove by contradiction: if $\mathbf{A}\boldsymbol{x} = \mathbf{0} \Rightarrow \boldsymbol{x} = \mathbf{0}$, then $\mathbf{A}$ has no zero eigenvalue.
3. By induction, prove $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

1.1.3 Notation Conventions

Remark 1.2 **(Scalars, Vectors, and Matrices)**

- **Scalars:** Lowercase italic (a, b, x).
- **Vectors:** Lowercase bold $(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{a})$. Default is column vector.
- **Matrices:** Uppercase bold $(\mathbf{A}, \mathbf{B}, \mathbf{Q})$.
- **Indexing:** 1-based (unlike Python). x_i is i-th entry; a_{ij} is (i, j) entry.

Remark 1.3 **(Transpose and Inverse)**

- $\mathbf{A}^T$: Transpose.
- $\mathbf{A}^{-1}$: Matrix inverse (square, nonsingular matrices).
- $\mathbf{A}^{-T}$: Transpose of the inverse $(\mathbf{A}^{-1})^T$.
- $\mathbf{A}^k$: k-th power. $\mathbf{A}^0 = \mathbf{I}$.

Remark 1.4 **(Sums and Products)**

$$\sum_{i=1}^{n} a_i = a_1 + \cdots + a_n, \qquad \prod_{i=1}^{n} a_i = a_1 \dots a_n.$$

Remark 1.5 **(Functions)**

- $f : \mathbb{R}^n \to \mathbb{R}^m$: Maps n-vectors to m-vectors.
- $f \circ g$: Function composition $(f \circ g)(\boldsymbol{x}) = f(g(\boldsymbol{x}))$.

Remark 1.6 **(Asymptotic Notation)** Scaling of algorithms is described using Big-O notation ($O(n^2), O(n^3)$). Details in the Computational Complexity section.

1.2 Vectors and Matrices

Vectors and matrices serve as the primary data structures for representing numerical data and linear operators. Before introducing these formally, we fix notation for the standard number systems that provide the raw material for all numerical computation.

(Standard Number Systems) Throughout these notes, $\mathbb{R}$ denotes the set of all real numbers, $\mathbb{Z}$ the set of all integers, and $\mathbb{C}$ the set of all complex numbers. Our primary focus is real-valued computation; complex numbers arise naturally in eigenvalue analysis and signal processing.

Def 1.7 **(Scalar)** An element of a number system ($\mathbb{R}$, $\mathbb{Z}$, or $\mathbb{C}$) is called a **scalar**. Scalars represent individual numerical quantities and serve as the atomic building blocks from which vectors and matrices are constructed.

Ex 1.3 Refer to Def 1.7.

1. Classify each of the following as a real scalar, an integer, a complex (non-real) scalar, or *not* a scalar: 3.14; -7; $2 + 3i$; $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$.
2. In Python, evaluate `type(3.14)`, `type(3)`, and `type(3+2j)`. Which NumPy `dtype` would you use to store each?

Def 1.8 **(Vectors)** A **vector** is a one-dimensional ordered array of scalars. Vectors can represent points, directions, or more abstract quantities within a vector space, and are fundamental in encoding data and linear relationships.

Remark 1.7 Vectors may be written as row vectors or column vectors. Although both forms carry the same information, the distinction matters when applying linear transformations and

computing inner products. **In these notes, we assume vectors are column vectors by default.**

Example

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 1+2i \\ 7i \\ 3 \end{pmatrix}, \ldots$$

Ex 1.4 Refer to Def 1.8.

1. Write a column vector $\boldsymbol{x} \in \mathbb{R}^4$ whose i-th entry equals 2^i for $i = 1, 2, 3, 4$.
2. In NumPy, construct this vector with `np.array([...])` and confirm `x.shape == (4,)`. **Why does NumPy report shape `(4,)` rather than `(4,1)`?** This is known as a *rank-1 array*; it is neither a row nor a column vector, and its broadcasting behavior differs from 2D arrays.
3. Can a vector in $\mathbb{R}^3$ be added to a vector in $\mathbb{R}^4$? Justify in one sentence.

Just as scalars belong to number systems like $\mathbb{R}$ or $\mathbb{C}$, vectors inhabit **vector spaces**. While these notes treat vector spaces informally, subsequent chapters provide more rigorous definitions and properties.

Def 1.9 ($\mathbb{R}^n$) The set of all real vectors of size n. When we write $\boldsymbol{x} \in \mathbb{R}^n$, it means $\boldsymbol{x}$ has n real entries. Vectors in $\mathbb{R}^n$ take one of the following forms:

$$\boldsymbol{x} \in \mathbb{R}^{n \times 1} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad \text{(column vector, default convention)}$$

$$\boldsymbol{x} \in \mathbb{R}^{1 \times n} = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix} \quad \text{(row vector)}$$

Remark 1.8 These notes focus on finite-dimensional spaces. Infinite-dimensional vector spaces, central to functional analysis, arise in differential equations and quantum mechanics but are beyond our scope here.

Ex 1.5 Refer to Def 1.9.

1. Let $\boldsymbol{x} = (1, -2, 3)^T \in \mathbb{R}^3$. Identify the value of entry x_2. What is the dimension of $\mathbb{R}^3$?
2. In NumPy, create a column vector (shape `(3,1)`) from the list `[1, -2, 3]` in two ways: using `np.reshape` and using `[:, np.newaxis]`.

3. What does `x.T` return when `x.shape == (3,)`? How does the result differ when `x.shape == (3,1)`?

Def 1.10 **(Matrices)** A **matrix** is a two-dimensional array of scalars. A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is written as

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}.$$

Matrices are used to represent linear transformations, systems of linear equations, and more complex data structures.

Example Two-dimensional matrices include:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \in \mathbb{R}^{2\times 3}, \quad \begin{pmatrix} 1 & 2 \\ 4 & 5 \\ 6 & 7 \end{pmatrix} \in \mathbb{R}^{3\times 2}.$$

Remark 1.9 **(Structural Intuition: Matrix Columns)** Matrices can be interpreted as vectors whose entries are themselves vectors:

$$\mathbf{A} = \begin{pmatrix} \text{—} & \boldsymbol{r}_1 & \text{—} \\ & \vdots & \\ \text{—} & \boldsymbol{r}_n & \text{—} \end{pmatrix} = \begin{pmatrix} | & & | \\ \boldsymbol{c}_1 & \cdots & \boldsymbol{c}_n \\ | & & | \end{pmatrix},$$

where $r_i = \mathbf{A}[i,:]$ is the i-th row and $c_i = \mathbf{A}[:,i]$ is the i-th column. This viewpoint is vital: the product $\mathbf{A}\boldsymbol{x}$ is simply a **linear combination of the columns of A** weighted by the entries of $\boldsymbol{x}$.

Ex 1.6 Refer to Def 1.10.

1. State the dimensions (m, n) of $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$. What is the shape of its second column as a vector?
2. In NumPy (0-based indexing), write the expression to access entry a_{12} (first row, second column in 1-based notation). Then extract the second column of $\mathbf{A}$ and verify its shape is `(2,)`.
3. True or False: a vector in $\mathbb{R}^n$ is a special case of a matrix in $\mathbb{R}^{n\times 1}$. How does NumPy handle this distinction in practice?

Def 1.11 **(Matrix Transpose)** The **transpose** of $\mathbf{A} \in \mathbb{R}^{m\times n}$, denoted $\mathbf{A}^T$, is the $n \times m$ matrix obtained by interchanging rows and columns:

$$(\mathbf{A}^T)_{ij} = a_{ji}, \quad 1 \le i \le n,\ 1 \le j \le m.$$

For a column vector $\boldsymbol{x} \in \mathbb{R}^n$, the transpose $\boldsymbol{x}^T$ is its corresponding row vector and vice versa.

Def 1.12 **(Symmetric and Skew-Symmetric Matrices)** A square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is:

- **symmetric** if $\mathbf{A}^T = \mathbf{A}$, equivalently $a_{ij} = a_{ji}$ for all i, j.
- **skew-symmetric** if $\mathbf{A}^T = -\mathbf{A}$, equivalently $a_{ij} = -a_{ji}$ for all i, j.

Remark 1.10 Every square matrix $\mathbf{A}$ decomposes uniquely into a symmetric part and a skew-symmetric part:

$$\mathbf{A} = \underbrace{\tfrac{1}{2}(\mathbf{A} + \mathbf{A}^T)}_{\text{symmetric}} + \underbrace{\tfrac{1}{2}(\mathbf{A} - \mathbf{A}^T)}_{\text{skew-symmetric}}.$$

Symmetric matrices arise constantly in scientific computing: covariance matrices in statistics, graph Laplacians in network problems, and Hessians of quadratic objectives in optimization are all symmetric.

Thm 1.1 **(Properties of the Transpose)** Let $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times p}$, and $\alpha \in \mathbb{R}$. Then:

1. $(\mathbf{A}^T)^T = \mathbf{A}$
2. $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$ (for conformable $\mathbf{A}, \mathbf{B}$)
3. $(\alpha \mathbf{A})^T = \alpha \mathbf{A}^T$
4. $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$ **(Reversal Law)**

Proof *Proof.* Properties (1)–(3) follow directly from the entry-wise definition $(\mathbf{A}^T)_{ij} = a_{ji}$. For (2):

$$\begin{aligned}(\mathbf{A} + \mathbf{B})^T_{ij} &= (\mathbf{A} + \mathbf{B})_{ji} \\ &= a_{ji} + b_{ji} \\ &= (\mathbf{A}^T)_{ij} + (\mathbf{B}^T)_{ij} \\ &= (\mathbf{A}^T + \mathbf{B}^T)_{ij}.\end{aligned}$$

For (4), let $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}$. By the definition of matrix product:

$$(\mathbf{C}^T)_{ij} = c_{ji} = \sum_{k=1}^{n} a_{jk}\, b_{ki}.$$

Expanding the (i, j) entry of $\mathbf{B}^T \mathbf{A}^T$ using $(\mathbf{B}^T)_{ik} = b_{ki}$ and $(\mathbf{A}^T)_{kj} = a_{jk}$:

$$\begin{aligned}(\mathbf{B}^T \mathbf{A}^T)_{ij} &= \sum_{k=1}^{n} (\mathbf{B}^T)_{ik}\, (\mathbf{A}^T)_{kj} \\ &= \sum_{k=1}^{n} b_{ki}\, a_{jk}.\end{aligned}$$

Both summations are identical, so $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$. □

Cor 1.2 **(Reversal Law for Multiple Factors)** By induction on Thm 1.1(4):

$$(\mathbf{A}_1\mathbf{A}_2\cdots\mathbf{A}_k)^T = \mathbf{A}_k^T\cdots\mathbf{A}_2^T\mathbf{A}_1^T.$$

Ex 1.7 Refer to Def 1.11 and Thm 1.1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$.

1. State the dimensions of $\mathbf{A}$, $\mathbf{A}^T$, $\mathbf{A}\mathbf{A}^T$, and $\mathbf{A}^T\mathbf{A}$.
2. Compute $\mathbf{A}^T$ explicitly. Then compute $\mathbf{A}\mathbf{A}^T$ and verify it is symmetric using Def 1.12.
3. In NumPy (0-based indexing), write the expression to retrieve element a_{12} (first row, second column in 1-based notation). Then write a one-liner for $\mathbf{A}\mathbf{A}^T$.
4. Is $\mathbf{A}\mathbf{A}^T = \mathbf{A}^T\mathbf{A}$ in general? What can you say about each product individually?

Cor 1.3 **(Outer Product is Symmetric of Rank 1)** For any nonzero $\boldsymbol{x} \in \mathbb{R}^n$, the outer product $\boldsymbol{x}\boldsymbol{x}^T \in \mathbb{R}^{n\times n}$ is symmetric and has rank 1. The inner product $\boldsymbol{x}^T\boldsymbol{x} \in \mathbb{R}$ satisfies $\boldsymbol{x}^T\boldsymbol{x} \geq 0$ with equality iff $\boldsymbol{x} = \mathbf{0}$.

Ex 1.8 In this exercise, we will prove Cor 1.3. Refer to Def 1.11 and Thm 1.1.
Let $\boldsymbol{x} = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$ be a nonzero column vector.

1. The **outer product** is $\boldsymbol{x}\boldsymbol{x}^T \in \mathbb{R}^{n\times n}$. Write out entry (i, j) explicitly.
2. Use the Reversal Law (Thm 1.1(4)) to show $\boldsymbol{x}\boldsymbol{x}^T$ is symmetric.
3. Show that $\boldsymbol{x}\boldsymbol{x}^T$ has rank 1 by arguing about the column space. (*Hint: every column of $\boldsymbol{x}\boldsymbol{x}^T$ is a scalar multiple of $\boldsymbol{x}$.*)
4. Express $\boldsymbol{x}^T\boldsymbol{x}$ as a sum of squares. Conclude $\boldsymbol{x}^T\boldsymbol{x} \geq 0$ with equality iff $\boldsymbol{x} = \mathbf{0}$, and identify the geometric quantity $\sqrt{\boldsymbol{x}^T\boldsymbol{x}}$.

1.3 Matrix Arithmetic

We are all familiar with scalar arithmetic: addition, subtraction, multiplication, and properties such as associativity and distributivity. In the following sections we extend these operations to vectors and matrices, building the algebraic machinery used throughout scientific computing.

1.3.1 Addition and Subtraction

Matrix addition generalizes scalar addition by operating entry-wise. The key constraint is dimensional compatibility: two matrices can be added only when they share the same shape.

Def 1.13 **(Matrix Sum)** Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m\times n}$. Their sum is defined as

$$\mathbf{A}+\mathbf{B} = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} + \begin{pmatrix} b_{11} & \cdots & b_{1n} \\ \vdots & \ddots & \vdots \\ b_{n1} & \cdots & b_{nn} \end{pmatrix} = \begin{pmatrix} (a+b)_{11} & \cdots & (a+b)_{1n} \\ \vdots & \ddots & \vdots \\ (a+b)_{n1} & \cdots & (a+b)_{nn} \end{pmatrix},$$

where each entry satisfies $(a+b)_{ij} = a_{ij} + b_{ij}$ for $1 \le i \le m$ and $1 \le j \le n$.

Remark 1.11 For $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{p\times q}$, the sum $\mathbf{A}+\mathbf{B}$ is defined if and only if $m = p$ and $n = q$.

Ex 1.9 Refer to Def 1.13.

1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. Compute $\mathbf{A}+\mathbf{B}$ and $2\mathbf{A}-\mathbf{B}$ by hand.
2. In NumPy, verify your answers using `np.array`. Then check whether `np.array([1,2,3]) + np.array([1,2])` raises an error and explain why.

Commutativity and Associativity of Matrix Addition

Let

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Then,

$$\mathbf{A}+\mathbf{B} = \begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix} \quad \text{and} \quad \mathbf{B}+\mathbf{A} = \begin{pmatrix} e+a & f+b \\ g+c & h+d \end{pmatrix}.$$

Since addition of real numbers is commutative, $\mathbf{A}+\mathbf{B} = \mathbf{B}+\mathbf{A}$. Furthermore, for any third matrix $\mathbf{C} = \begin{pmatrix} i & j \\ k & l \end{pmatrix}$,

$$(\mathbf{A}+\mathbf{B})+\mathbf{C} = \begin{pmatrix} (a+e)+i & (b+f)+j \\ (c+g)+k & (d+h)+l \end{pmatrix},$$

and

$$\mathbf{A}+(\mathbf{B}+\mathbf{C}) = \begin{pmatrix} a+(e+i) & b+(f+j) \\ c+(g+k) & d+(h+l) \end{pmatrix}.$$

By associativity of real addition, these are identical. Hence matrix addition is both commutative and associative.

Thm 1.4 **($\mathbb{R}^{m\times n}$ as a Vector Space)** The set $\mathbb{R}^{m\times n}$ equipped with matrix addition and scalar multiplication forms a real vector space of dimension mn.

Proof *Proof.* Each axiom follows from the corresponding property of $\mathbb{R}$ applied entry-wise. For **commutativity**:

$$\begin{aligned}(\mathbf{A}+\mathbf{B})_{ij} &= a_{ij}+b_{ij}\\ &= b_{ij}+a_{ij}\\ &= (\mathbf{B}+\mathbf{A})_{ij}.\end{aligned}$$

The **additive identity** is $\mathbf{0}$ since $(\mathbf{A}+\mathbf{0})_{ij} = a_{ij}+0 = a_{ij}$. The **additive inverse** of $\mathbf{A}$ is $-\mathbf{A}$, satisfying $(\mathbf{A}+(-\mathbf{A}))_{ij} = a_{ij}-a_{ij} = 0$.
For the **dimension**: let $\mathbf{E}^{(ij)}$ be matrices with a 1 in position (i,j) and zeros elsewhere. Every $\mathbf{A}\in\mathbb{R}^{m\times n}$ decomposes uniquely as:

$$\mathbf{A} = \sum_{i,j} a_{ij}\,\mathbf{E}^{(ij)}.$$

Since these mn matrices are linearly independent and span $\mathbb{R}^{m\times n}$, they form a basis of size mn. □

Def 1.14 **(Scalar Multiplication)** Let $\mathbf{A}\in\mathbb{R}^{m\times n}$ and $c\in\mathbb{R}$. Then scalar multiplication is defined as

$$c\mathbf{A} = c\begin{pmatrix} a_{11} & \cdots & a_{1n}\\ \vdots & \ddots & \vdots\\ a_{n1} & \cdots & a_{nn}\end{pmatrix} = \begin{pmatrix} ca_{11} & \cdots & ca_{1n}\\ \vdots & \ddots & \vdots\\ ca_{n1} & \cdots & ca_{nn}\end{pmatrix},$$

where each entry satisfies $(ca)_{ij} = c\cdot a_{ij}$.

Distributivity of Scalar Multiplication over Matrix Addition

Let $\alpha\in\mathbb{R}$ and

$$\mathbf{A} = \begin{pmatrix} a & b\\ c & d\end{pmatrix},\quad \mathbf{B} = \begin{pmatrix} e & f\\ g & h\end{pmatrix}.$$

Then,

$$\alpha(\mathbf{A}+\mathbf{B}) = \alpha\begin{pmatrix} a+e & b+f\\ c+g & d+h\end{pmatrix} = \begin{pmatrix} \alpha(a+e) & \alpha(b+f)\\ \alpha(c+g) & \alpha(d+h)\end{pmatrix}.$$

Since scalar multiplication distributes over addition for real numbers, this equals

$$\begin{pmatrix} \alpha a+\alpha e & \alpha b+\alpha f\\ \alpha c+\alpha g & \alpha d+\alpha h\end{pmatrix} = \alpha\mathbf{A}+\alpha\mathbf{B}.$$

Ex 1.10 Refer to Def 1.14 and Thm 1.4.

1. Compute $3\mathbf{A}-2\mathbf{B}$ for $\mathbf{A} = \begin{pmatrix} 1 & 0\\ -1 & 2\end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 2 & 1\\ 0 & -1\end{pmatrix}$.
2. The standard basis for $\mathbb{R}^{2\times 2}$ consists of four matrices $\mathbf{E}^{(ij)}$. Write them out explicitly and verify that $\mathbf{A} = \sum_{i,j} a_{ij}\mathbf{E}^{(ij)}$ holds for your $\mathbf{A}$ above.

1.3.2 Dot Product

Before defining the general matrix product, we introduce the dot product of two vectors. The matrix product will then be understood entry-wise as a collection of dot products.

Def 1.15 **(Dot Product)** For vectors $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^n$, the **dot product** (or inner product) is defined as

$$\boldsymbol{a} \cdot \boldsymbol{b} = \boldsymbol{a}^T \boldsymbol{b} = \begin{pmatrix} a_1 & \cdots & a_n \end{pmatrix} \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix} = \sum_{i=1}^{n} a_i b_i. \tag{1}$$

Ex 1.11 Refer to Def 1.15.

1. Compute $\boldsymbol{a} \cdot \boldsymbol{b}$ for $\boldsymbol{a} = (1, 2, 3)^T$ and $\boldsymbol{b} = (-1, 0, 2)^T$ by hand.
2. In NumPy, evaluate this using `np.dot(a, b)` and also with `a @ b`. Confirm both give the same result.
3. Show that $\boldsymbol{a} \cdot \boldsymbol{a} \geq 0$ with equality iff $\boldsymbol{a} = \mathbf{0}$. What geometric quantity does $\sqrt{\boldsymbol{a} \cdot \boldsymbol{a}}$ represent?

Def 1.16 **(Matrix Product)** Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$. Their product $\mathbf{AB}$ is an $m \times p$ matrix defined by

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj},$$

or equivalently, if $\boldsymbol{a}_i$ denotes the i-th row of $\mathbf{A}$ and $\boldsymbol{b}_j$ the j-th column of $\mathbf{B}$:

$$\mathbf{AB} = \begin{pmatrix} \boldsymbol{a}_1 \cdot \boldsymbol{b}_1 & \cdots & \boldsymbol{a}_1 \cdot \boldsymbol{b}_p \\ \vdots & \ddots & \vdots \\ \boldsymbol{a}_m \cdot \boldsymbol{b}_1 & \cdots & \boldsymbol{a}_m \cdot \boldsymbol{b}_p \end{pmatrix}.$$

Remark 1.12 Matrix multiplication is defined only when the number of columns of $\mathbf{A}$ equals the number of rows of $\mathbf{B}$. It is associative and distributive over addition, but in general not commutative: $\mathbf{AB} \neq \mathbf{BA}$ for most matrices.

Lem 1.5 **(Computational Cost of Matrix Multiplication)** Computing $\mathbf{AB}$ where $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$ requires exactly mnp multiplications and $m(n-1)p$ additions using the standard algorithm, giving $O(mnp)$ floating-point operations (flops). For two square $n \times n$ matrices this reduces to $O(n^3)$ flops.

Remark 1.13 The $O(n^3)$ cost of naive matrix multiplication is a central bottleneck in scientific computing. Strassen's algorithm reduces this to $O(n^{\log_2 7}) \approx O(n^{2.807})$, but is rarely used in

practice due to numerical stability concerns and cache-unfriendly memory access. Modern BLAS/LAPACK implementations approach peak hardware throughput via blocking strategies, but the asymptotic cost remains $O(n^3)$. For many applications, avoiding explicit matrix-matrix products entirely (e.g., exploiting sparsity or factored structure) is more impactful than algorithmic improvements.

Compatibility for Matrix Multiplication

Let $\mathbf{A}$ be a 3×4 matrix and $\mathbf{B}$ a 4×2 matrix. Since the number of columns of $\mathbf{A}$ (4) equals the number of rows of $\mathbf{B}$ (4), the product $\mathbf{AB}$ is defined and results in a 3×2 matrix. Conversely, if $\mathbf{B}$ were a 5×2 matrix, the product would be undefined.

Distributive Property of Matrix Multiplication

Let

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}.$$

Then,

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{A} \begin{pmatrix} b_{11} + c_{11} & b_{12} + c_{12} \\ b_{21} + c_{21} & b_{22} + c_{22} \end{pmatrix},$$

and by computing the entries, one can verify that

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}.$$

Non-Commutativity of Matrix Multiplication

Consider

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Then,

$$\mathbf{AB} = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} \quad \text{and} \quad \mathbf{BA} = \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}.$$

Since $\mathbf{AB} \neq \mathbf{BA}$, matrix multiplication is not commutative.

Ex 1.12 Refer to Def 1.16 and Lem 1.5. Let $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 1 & -1 \\ 2 & 4 \end{pmatrix}$.

1. Compute $\mathbf{AB}$ and $\mathbf{BA}$ by hand. Confirm $\mathbf{AB} \neq \mathbf{BA}$.
2. Find a nonzero matrix $\mathbf{C} \neq \mathbf{I}$ that commutes with $\mathbf{A}$ (i.e., $\mathbf{AC} = \mathbf{CA}$). *Hint: try* $\mathbf{C} = \alpha\mathbf{A} + \beta\mathbf{I}$.

3. Verify associativity: compute $(\mathbf{AB})\boldsymbol{c}$ and $\mathbf{A}(\mathbf{B}\boldsymbol{c})$ for $\boldsymbol{c} = (1, 0)^T$ and confirm they are equal.

4. Compare the flop count of $(\mathbf{AB})\boldsymbol{c}$ versus $\mathbf{A}(\mathbf{B}\boldsymbol{c})$ for general $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{c} \in \mathbb{R}^n$. Which order is cheaper and by what factor?

Many properties of scalar arithmetic extend naturally to vectors and matrices. In fact, matrices can be viewed as elements of a higher-dimensional vector space, and matrix multiplication can be interpreted as the composition of linear transformations. While the notations for vectors and matrices are similar, their roles differ:

- Vectors typically represent points or directions in space.
- Matrices often represent linear maps or transformations between vector spaces.

This conceptual distinction becomes crucial when discussing eigenvalues and linear transformations.

Thm 1.6 **(Associativity as an Optimization)** For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{x} \in \mathbb{R}^n$, computing $\mathbf{A}(\mathbf{B}\boldsymbol{x})$ via two matrix-vector products costs $O(n^2)$ flops, whereas forming $(\mathbf{AB})\boldsymbol{x}$ via a matrix-matrix product costs $O(n^3)$ flops.

Ex 1.13 Suppose $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{x} \in \mathbb{R}^n$.

1. *Approach 1:* Compute $\mathbf{M} = \mathbf{AB}$ first (cost?), then $\mathbf{M}\boldsymbol{x}$ (cost?). Write the total flop count.

2. *Approach 2:* Compute $\boldsymbol{y} = \mathbf{B}\boldsymbol{x}$ first (cost?), then $\mathbf{A}\boldsymbol{y}$ (cost?). Write the total flop count.

3. Conclude the asymptotic improvement factor as $n \to \infty$.

4. Write NumPy code for both approaches using `@`. Does NumPy automatically choose the optimal evaluation order for a chain like `A @ B @ x`?

1.3.3 Matrix Inverse

We now turn to a fundamental question: under what conditions can a matrix operation be defined that is analogous to scalar division? The matrix inverse plays the role of the reciprocal in scalar arithmetic, but it exists only under specific conditions.

Def 1.17 **(Matrix Inverse)** For a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the **inverse** of $\mathbf{A}$, denoted $\mathbf{A}^{-1}$, is the unique matrix satisfying

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I},$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. A matrix that has an inverse is called *invertible* or *nonsingular*.

Remark 1.14 A square matrix $\mathbf{A}$ is invertible if and only if $\det(\mathbf{A}) \neq 0$, which also implies that $\mathbf{A}$ has full rank.

Uniqueness of the Inverse

Suppose $\mathbf{A}$ is invertible and both $\mathbf{B}$ and $\mathbf{C}$ satisfy the conditions of Def 1.17. Then,

$$\mathbf{B} = \mathbf{BI} = \mathbf{B}(\mathbf{AC}) = (\mathbf{BA})\mathbf{C} = \mathbf{IC} = \mathbf{C}.$$

Thus the inverse of $\mathbf{A}$ is unique.

Thm 1.7 **(Properties of the Matrix Inverse)** Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ be invertible and $\alpha \in \mathbb{R} \setminus \{0\}$. Then:

1. $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$
2. $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ **(Reversal Law)**
3. $(\alpha\mathbf{A})^{-1} = \frac{1}{\alpha}\mathbf{A}^{-1}$
4. $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$

Proof *Proof.* **(1):** The conditions $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$ and $\mathbf{AA}^{-1} = \mathbf{I}$ show that $\mathbf{A}$ is both a left and right inverse of $\mathbf{A}^{-1}$. By the uniqueness of the inverse:

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A}.$$

(2): Check the right-inverse condition for $\mathbf{B}^{-1}\mathbf{A}^{-1}$:

$$\begin{aligned}(\mathbf{AB})(\mathbf{B}^{-1}\mathbf{A}^{-1}) &= \mathbf{A}(\mathbf{BB}^{-1})\mathbf{A}^{-1} \\ &= \mathbf{AIA}^{-1} \\ &= \mathbf{AA}^{-1} = \mathbf{I}.\end{aligned}$$

The left-inverse condition $(\mathbf{B}^{-1}\mathbf{A}^{-1})(\mathbf{AB}) = \mathbf{I}$ follows similarly. Thus $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$.
(3): For any $\alpha \neq 0$:

$$(\alpha\mathbf{A})\left(\frac{1}{\alpha}\mathbf{A}^{-1}\right) = \frac{\alpha}{\alpha}(\mathbf{AA}^{-1}) = \mathbf{I}.$$

(4): Transpose both sides of $\mathbf{AA}^{-1} = \mathbf{I}$ using the Reversal Law for transposes:

$$\begin{aligned}(\mathbf{AA}^{-1})^T &= \mathbf{I}^T \\ (\mathbf{A}^{-1})^T\mathbf{A}^T &= \mathbf{I}.\end{aligned}$$

This shows $(\mathbf{A}^{-1})^T$ is the inverse of $\mathbf{A}^T$, so $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$. □

Remark 1.15 **(The Decomposition Principle)** In scientific computing, we almost never compute $\mathbf{A}^{-1}$ explicitly. To solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, we decompose $\mathbf{A}$ into simpler factors (like LU or QR) and

solve a sequence of simpler systems. This is more numerically stable and computationally efficient. The Reversal Law $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ reflects this: to solve $(\mathbf{AB})\boldsymbol{x} = \boldsymbol{b}$, we first solve $\mathbf{A}\boldsymbol{y} = \boldsymbol{b}$ for $\boldsymbol{y}$, then $\mathbf{B}\boldsymbol{x} = \boldsymbol{y}$.

Ex 1.14 Refer to Def 1.17 and Thm 1.7.

1. For $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}$, compute $\mathbf{A}^{-1}$ by hand using the 2×2 formula $\mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. Verify $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$.
2. Given $\mathbf{B} = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}$, use the Reversal Law to compute $(\mathbf{AB})^{-1}$ without inverting a new 2×2 system.
3. Explain why `np.linalg.solve(A, b)` is preferred over `np.linalg.inv(A) @ b` for solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. Address both numerical accuracy and computational cost.

Def 1.18 **(Involutory Matrix)** A square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is called **involutory** if $\mathbf{A}^2 = \mathbf{I}$.

Ex 1.15 In this exercise, we will prove that a matrix is involutory (Def 1.18) if and only if it is its own inverse. Refer to Def 1.17 and Thm 1.7.

1. Suppose $\mathbf{A}^2 = \mathbf{I}$. Multiply both sides of $\mathbf{AA} = \mathbf{I}$ on the right by $\mathbf{A}^{-1}$ to deduce $\mathbf{A}^{-1} = \mathbf{A}$.
2. Conversely, suppose $\mathbf{A}^{-1} = \mathbf{A}$. Show $\mathbf{A}^2 = \mathbf{I}$.
3. Give a concrete 2×2 example of an involutory matrix other than $\pm\mathbf{I}$. Verify it satisfies $\mathbf{A}^2 = \mathbf{I}$.

Ex 1.16 **(Identifying Invertibility)** For each matrix below, determine without computing the inverse whether it is invertible. Give a one-sentence justification.

$$(a)\ \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} \qquad (b)\ \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \qquad (c)\ \begin{pmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & -1 \end{pmatrix} \qquad (d)\ \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 2 & 1 \end{pmatrix}$$

1.4 Introduction to Norms

A *norm* is a function that assigns a non-negative scalar "size" to a vector or matrix. Norms are the fundamental tool for measuring errors, bounding perturbations, and defining convergence throughout scientific computing. This chapter introduces the key vector norm definitions and the families most used in practice. We will study norms more rigorously in later chapters, where we develop matrix norms, operator inequalities, Cauchy-Schwarz, Hölder's inequality, and the condition number.

Def 1.19 **(Vector Norm)** A function $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ is a *vector norm* if for all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and $c \in \mathbb{R}$:

1. $\|\boldsymbol{x}\| \geq 0$ and $\|\boldsymbol{x}\| = 0 \Leftrightarrow \boldsymbol{x} = \mathbf{0}$ (positive definiteness)
2. $\|c\boldsymbol{x}\| = |c|\|\boldsymbol{x}\|$ (absolute homogeneity)
3. $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$ (triangle inequality)

Def 1.20 **(ℓ^p Norms)** For $\boldsymbol{x} \in \mathbb{R}^n$ and $p \geq 1$, the *ℓ^p norm* is

$$\|\boldsymbol{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}.$$

The three most common cases are:

- $\|\boldsymbol{x}\|_1 = \sum_{i=1}^{n} |x_i|$ (*Manhattan norm*, the sum of absolute values)
- $\|\boldsymbol{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{\boldsymbol{x}^T \boldsymbol{x}}$ (*Euclidean norm*)
- $\|\boldsymbol{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$ (*max norm*, the limit as $p \to \infty$)

Remark 1.16 **(Physical Intuition)** In scientific computing, the choice of norm depends on what you want to measure:

- ℓ_2 **(Energy):** The standard measure of magnitude; corresponds to physical energy or distance.
- ℓ_∞ **(Worst-case):** Measures the single largest entry. Useful for "tolerance" checks (e.g., "every entry must be below 10^{-6}").
- ℓ_1 **(Sparsity):** Encourages or measures sparsity; often used in optimization and compressed sensing.

Example For $\boldsymbol{x} = (3, -4, 0)^T$:

$$\|\boldsymbol{x}\|_1 = 3 + 4 + 0 = 7, \qquad \|\boldsymbol{x}\|_2 = \sqrt{9 + 16 + 0} = 5, \qquad \|\boldsymbol{x}\|_\infty = 4.$$

The unit balls $\{\boldsymbol{x} : \|\boldsymbol{x}\|_p \leq 1\}$ in $\mathbb{R}^2$ are: a diamond ($p = 1$), a circle ($p = 2$), and a square ($p = \infty$). As p increases, the unit ball grows toward the ℓ^∞ square.

Remark 1.17 Throughout these notes, $\|\cdot\|$ without a subscript denotes the Euclidean norm $\|\cdot\|_2$ unless stated otherwise. Errors and residuals are almost always measured in $\|\cdot\|_2$. In NumPy: `np.linalg.norm(x)` computes $\|\boldsymbol{x}\|_2$; `np.linalg.norm(x, ord=p)` computes $\|\boldsymbol{x}\|_p$.

Remark 1.18 **(Numerical Zero)** In floating-point arithmetic, the condition $\|\boldsymbol{x}\| = 0$ is rarely met exactly due to rounding. We instead use a **tolerance** ε: we say $\boldsymbol{x}$ is "numerically zero" if $\|\boldsymbol{x}\| \leq \varepsilon \cdot \|\boldsymbol{x}_0\|$, where $\boldsymbol{x}_0$ is some initial reference vector.

Thm 1.8 **(All Norms are Equivalent on $\mathbb{R}^n$)** For any two norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$ on $\mathbb{R}^n$, there exist constants $c_1, c_2 > 0$ such that

$$c_1\|\boldsymbol{x}\|_\beta \le \|\boldsymbol{x}\|_\alpha \le c_2\|\boldsymbol{x}\|_\beta \quad \text{for all } \boldsymbol{x} \in \mathbb{R}^n.$$

In particular: $\|\boldsymbol{x}\|_2 \le \|\boldsymbol{x}\|_1 \le \sqrt{n}\,\|\boldsymbol{x}\|_2$ and $\|\boldsymbol{x}\|_\infty \le \|\boldsymbol{x}\|_2 \le \sqrt{n}\,\|\boldsymbol{x}\|_\infty$.

Proof *Proof.* Since $\mathbb{R}^n$ is finite-dimensional, the unit sphere $S_\beta = \{\boldsymbol{x} : \|\boldsymbol{x}\|_\beta = 1\}$ is compact. Any norm $\|\cdot\|_\alpha$ is a continuous function on $\mathbb{R}^n$, and by the Extreme Value Theorem, it attains its minimum c_1 and maximum c_2 on S_β:

$$c_1 = \min_{\boldsymbol{x}\in S_\beta} \|\boldsymbol{x}\|_\alpha, \quad c_2 = \max_{\boldsymbol{x}\in S_\beta} \|\boldsymbol{x}\|_\alpha.$$

For any nonzero $\boldsymbol{y} \in \mathbb{R}^n$, let $\boldsymbol{x} = \boldsymbol{y}/\|\boldsymbol{y}\|_\beta \in S_\beta$. Then:

$$c_1 \le \left\| \frac{\boldsymbol{y}}{\|\boldsymbol{y}\|_\beta} \right\|_\alpha \le c_2 \quad \Rightarrow \quad c_1\|\boldsymbol{y}\|_\beta \le \|\boldsymbol{y}\|_\alpha \le c_2\|\boldsymbol{y}\|_\beta.$$

The explicit inequality $\|\boldsymbol{x}\|_1 \le \sqrt{n}\|\boldsymbol{x}\|_2$ follows from Cauchy-Schwarz applied to $|\boldsymbol{x}|$ and the all-ones vector $\mathbf{1}$:

$$\begin{aligned} \|\boldsymbol{x}\|_1 &= \sum_{i=1}^{n} |x_i| \cdot 1 \\ &\le \left(\sum_{i=1}^{n} x_i^2\right)^{1/2} \left(\sum_{i=1}^{n} 1^2\right)^{1/2} \\ &= \sqrt{n}\|\boldsymbol{x}\|_2. \end{aligned}$$

□

Ex 1.17 Refer to Defs 1.19–1.20 and Thm 1.8.

1. For $\boldsymbol{x} = (-1, 2, -3, 4)^T$, compute $\|\boldsymbol{x}\|_1$, $\|\boldsymbol{x}\|_2$, $\|\boldsymbol{x}\|_\infty$ by hand. Verify all three norm-equivalence inequalities hold.
2. Verify the triangle inequality $\|\boldsymbol{x} + \boldsymbol{y}\|_2 \le \|\boldsymbol{x}\|_2 + \|\boldsymbol{y}\|_2$ for $\boldsymbol{x} = (1, 2)^T$, $\boldsymbol{y} = (-3, 1)^T$ by computing both sides.
3. Using NumPy, confirm your hand calculations with `np.linalg.norm(x, 1)`, `np.linalg.norm(x`
and `np.linalg.norm(x, np.inf)`.
4. Show that $\|\boldsymbol{x}\|_\infty = \lim_{p\to\infty} \|\boldsymbol{x}\|_p$. (*Hint: let* $M = \max_i |x_i|$*; factor* M *out of* $\|\boldsymbol{x}\|_p$ *and take the limit.*)

1.5 Computational Complexity

Algorithmic cost measured in arithmetic operations (flops). Determines scalability and feasibility.

1.5.1 Big-O Notation

Def 1.21 **(Big-O Notation)** $f(n) = O(g(n))$ if $\exists C, n_0 > 0$ s.t. $|f(n)| \leq C|g(n)|$ for $n \geq n_0$.

- $\Omega(g)$: Grows at least as fast.
- $\Theta(g)$: Grows at the same rate.
- $o(g)$: Grows strictly slower.

Remark 1.19 **(Typical Rates)** $O(1) \ll O(\log n) \ll O(n) \ll O(n \log n) \ll O(n^2) \ll O(n^3) \ll O(2^n)$. For $n = 1000$: $O(n^2)$ is $\sim 10^6$ ops (ms); $O(n^3)$ is $\sim 10^9$ ops (s).

Remark 1.20 **(Properties)** 1. **Transitivity:** $f = O(g), g = O(h) \Rightarrow f = O(h)$. 2. **Sum Rule:** $O(f) + O(g) = O(\max(f, g))$. 3. **Product Rule:** $O(f) \cdot O(g) = O(fg)$. 4. **Constant Factors:** $O(cf) = O(f)$.

Ex 1.18

1. Show $3n^2 + 100n + 5 = O(n^2)$.
2. Compare growth: $n \log n$ vs n^2.
3. Rank by cost: $5n^3 + n, 100n^2, n^2 \log n, 2^{10}n^2$.
4. Use Master Theorem: $T(n) = 2T(n/2) + n$.

1.5.2 Flop Counting

Hardware-independent measure of work.

Def 1.22 **(Flop)** A single floating-point operation $(+, -, \times, \div)$.

Thm 1.9 **(Basic Costs)** For $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}$:

- **Dot product:** $2n$ flops.
- **Matrix-vector product:** $2mn$ flops.
- **Matrix-matrix product:** $2mnp$ flops ($2n^3$ for square).
- **Triangular solve:** n^2 flops.

Remark 1.21 **Data Locality and Performance:** On modern architectures, computational throughput often significantly exceeds memory bandwidth. Consequently, algorithms with high cache locality, such as blocked matrix multiplication (GEMM), may achieve superior performance compared to algorithms with lower theoretical operation counts but poorer data reuse.

Remark 1.22 **(Benchmarks)**

- **LU Factorization:** $\frac{2}{3}n^3$.
- **Cholesky:** $\frac{1}{3}n^3$.
- **QR (Householder):** $\frac{4}{3}n^3$.

Ex 1.19

1. Verify $2mn$ flops for $\mathbf{A}\boldsymbol{x}$ by counting multiply-adds.
2. Estimate wall-clock time for 10^9 flops on a 10^9 flop/s CPU.
3. Python: Plot `np.dot(A, B)` time vs n (log-log). Verify exponent ≈ 3.
4. Why might the empirical exponent be less than 3? (Cache hierarchy, vectorization).

1.6 Floating Point Arithmetic

Digital computers represent real numbers using a finite number of bits. The result is not the real line, but a finite, non-uniform grid of representable numbers. Numerical analysis begins with this mismatch: the mathematical algorithm acts on real numbers, while the computer executes a nearby algorithm on rounded numbers.

Floating-point arithmetic is usually accurate locally: one elementary operation is rounded to a nearby representable number. Instability appears when an algorithm repeatedly amplifies these tiny local errors, or when it subtracts nearly equal quantities and exposes digits that were already contaminated by rounding.

1.6.1 Floating Point Representation

Most modern systems adhere to the IEEE 754 standard, which represents numbers in a scientific-like notation. Normalized numbers take the form $x = (-1)^s \times (1.f) \times 2^e$.

- **s:** Sign bit, determining whether the number is positive or negative.
- **f:** Fractional significand (mantissa); the leading 1 is implicit to maximize precision.
- **e:** Biased exponent, allowing both very large and very small numbers to be represented.

The exponent controls scale and the significand controls precision. This means floating-point numbers have roughly fixed **relative** spacing, not fixed absolute spacing. There are many representable numbers near zero and far fewer, in absolute terms, near 10^{100}. The gap between adjacent numbers near x is proportional to $|x|$.

Remark 1.23 **(IEEE 754 Standards)**

- **Single (32-bit):** 1 sign, 8 exponent, 23 fraction bits.
- **Double (64-bit):** 1 sign, 11 exponent, 52 fraction bits.

Example **(16-bit float)** 1 sign, 5 exp (bias 15), 10 frac. Represent -3.25:

1. $s = 1$.
2. $3.25_{10} = 11.01_2 = 1.101_2 \times 2^1$.
3. $e = 1 + 15 = 16 = 10000_2$.
4. $f = 1010000000_2$.

Result: `1 10000 1010000000`.

Ex 1.20

1. Encode $+5.5$ in 16-bit format.
2. How many distinct normalized numbers for 5 exp bits, 10 frac bits?

Def 1.23 **(Machine Epsilon ($\varepsilon_{\text{mach}}$))** Machine epsilon is the distance from 1 to the next larger floating-point number. Equivalently, it is the smallest positive floating-point number ε such that $\text{fl}(1 + \varepsilon) > 1$. It characterizes the spacing of representable numbers near 1 and is given by $\varepsilon_{\text{mach}} = 2^{1-p}$, where p is the number of bits in the significand.

- For double precision (64-bit), $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.

Remark 1.24 **(Spacing rule)** Near a normalized number x, the distance to the next representable number is approximately $\varepsilon_{\text{mach}}|x|$. Near 10^{16}, the gap is on the order of 1; near 1, the gap is on the order of 10^{-16}. This is why adding a small number to a huge number may do nothing.

Thm 1.10 **(Absorption Criterion)** In round-to-nearest arithmetic, if $|y|$ is less than half the spacing between adjacent floating-point numbers near x, then

$$\text{fl}(x + y) = x.$$

Equivalently, small increments disappear when they are below the local resolution of the floating-point grid.

Ex 1.21

1. Use Def 1.23 to predict whether `1.0 + 1e-16` and `1.0 + 2e-16` differ from `1.0`; then check in Python.
2. Find $\varepsilon_{\text{mach}}$ empirically via loop.

3. Use Thm 1.10 to predict the smallest power of 10 for which `1e16 + 10**k` changes `1e16`.

4. Explain why the answer changes if `1e16` is replaced by `1.0`.

Def 1.24 **(Unit Roundoff (u))** Unit roundoff is the maximum relative error caused by rounding one real number to the nearest floating-point number:

$$\frac{|\mathrm{fl}(x) - x|}{|x|} \leq u.$$

For round-to-nearest arithmetic, $u = \varepsilon_{\text{mach}}/2 = 2^{-p}$.

Remark 1.25 **(Machine epsilon vs. unit roundoff)** $\varepsilon_{\text{mach}}$ is a spacing near 1. The unit roundoff u is an error bound for rounding to nearest. Many texts use "machine epsilon" for both quantities, so always check the convention. In these notes, elementary rounding errors are bounded by u.

Thm 1.11 **(Fundamental Axiom)** IEEE 754 guarantees $\mathrm{fl}(a \circ b) = (a \circ b)(1+\delta)$ for $\circ \in \{+, -, \times, \div\}$ and $|\delta| \leq u$. Individual operations are as accurate as the representation allows.

Proof *Proof.* The real line near x is partitioned into intervals, each containing exactly one floating point number. In round-to-nearest mode, $\mathrm{fl}(x)$ is the closer endpoint, so:

$$|\mathrm{fl}(x) - x| \leq \frac{1}{2}\text{gap} = \frac{1}{2}(\varepsilon_{\text{mach}}|x|) = u|x|.$$

Dividing by $|x|$ gives the relative error bound $|\delta| \leq u$. For any operation $\circ$, IEEE 754 mandates computing the exact result first, then rounding, ensuring:

$$\mathrm{fl}(a \circ b) = \mathrm{round}(a \circ b) = (a \circ b)(1 + \delta).$$

□

Remark 1.26 **Warning:** After n operations, error often grows like $O(nu)$ when errors do not systematically amplify. This is a rule of thumb, not a guarantee. Operation order matters because each intermediate result is rounded.

Ex 1.22

1. Let $a = 1, b = -0.999\ldots$. Is error in $\mathrm{fl}(a + b)$ bounded by u?

2. Estimate worst-case relative error for 10^6 sequential additions.

1.6.2 IEEE 754 Special Values

- **Signed Zero:** $+0, -0$. $1/(+0) = \infty$, $1/(-0) = -\infty$.

- **Subnormals:** Fill the gap between 0 and the smallest normalized number. Reduced precision, allows gradual underflow.
- **Infinities:** $\pm\infty$. Result of overflow or $x/0$.
- **NaN:** Not a Number ($0/0$, $\infty - \infty$). Unequal to everything, including itself.

Remark 1.27 **(NaN propagation)** A NaN usually contaminates all later arithmetic. This is useful for debugging: it marks an invalid operation that happened earlier, even if the failure is observed much later.

Ex 1.23

1. Python: `1.0/0.0` vs `1.0/-0.0`.
2. Result of `float('inf') - float('inf')` and `float('nan') == float('nan')`.

1.6.3 Arithmetic Pitfalls

Because floating-point numbers are discrete, the standard laws of algebra do not always hold. In particular, operations are neither associative nor distributive in general.

- **Non-Associativity:** The order of operations can change the result, i.e., $(a+b)+c \neq a+(b+c)$.
- **Non-Distributivity:** $a(b+c) \neq ab+ac$.
- **Absorption:** If $|a|$ is much larger than $|b|$, then $a+b$ may equal a exactly because b is smaller than the gap between representable numbers near a.

Example **(Absorption)** In double precision, `1e16 + 1.0 == 1e16` is true. The number 1 is smaller than the spacing between adjacent representable numbers near 10^{16}, so it is rounded away. Consequently `(1e16 + 1.0) - 1e16` returns `0.0`, not `1.0`.

Example **(Non-associativity)** Let $a = 10^{16}$, $b = -10^{16}$, and $c = 1$. Then

$$(a+b)+c = 1, \qquad a+(b+c) = 0$$

in double precision. The second expression first rounds $b+c$ back to -10^{16}, so the 1 is lost before the final addition.

Remark 1.28 **(Summation rule)** Add numbers of similar magnitude when possible. Summing from smallest to largest, pairwise summation, or compensated summation reduces the loss of small terms. This is why library reductions may give different answers than a hand-written loop while still being more accurate.

Ex 1.24

1. Before running Python, use Thm 1.10 to predict `(1e16 + 1.0) - 1e16`.
2. `a=1e16, b=1`. Find `(a+b)-a` in Python. Explain result.
3. Find a, b, c where `a*(b+c) != a*b + a*c`.
4. Before running Python, predict the results of summing $[10^{16}, 1, -10^{16}]$ in the listed order and in the order $[10^{16}, -10^{16}, 1]$.
5. Compare naive summation, pairwise summation, and Kahan summation on a list containing one large number and many small numbers. Which method best preserves the small terms?

1.7 Quantifying Approximation Error

1.7.1 Absolute and Relative Errors

- **Absolute Error:** $|\hat{x} - x|$.
- **Relative Error:** $\varepsilon = \frac{|\hat{x}-x|}{|x|}$. This is the standard measure for precision in numerical computing.

Absolute error measures the size of the mistake in the original units. Relative error measures the size of the mistake compared to the quantity being computed. In numerical computing, relative error is usually more informative: an error of 10^{-6} is excellent for a number of size 10^3 and terrible for a number of size 10^{-9}.

Remark 1.29 **(Comparing floating-point numbers)** Avoid testing computed real numbers with exact equality. Use a tolerance such as

$$|\hat{x} - x| \leq \text{atol} + \text{rtol}|x|.$$

This is the logic behind `np.isclose`. Absolute tolerance handles values near zero; relative tolerance handles scale.

Ex 1.25

1. $\sqrt{2} \approx 1.414$. Errors?
2. Relative error of product $\hat{x}\hat{y}$? (Sum of individual relative errors).

Def 1.25 **(Catastrophic Cancellation)** The most dangerous error in numerical computing is **catastrophic cancellation**. This occurs when two nearly equal numbers are subtracted, causing the most significant digits to cancel out and leaving only the rounding errors as the "leading" digits.

- Relative error bound: $\varepsilon_{\text{rel}} \leq u \cdot \frac{x+y}{x-y}$.
- As $x \to y$, the amplification factor $\frac{x+y}{x-y}$ tends to infinity.

Proof *Proof.* Let $\hat{x} = x(1+\varepsilon_x)$ and $\hat{y} = y(1+\varepsilon_y)$ with $|\varepsilon_x|, |\varepsilon_y| \leq u$. The error in the subtraction is:

$$\begin{aligned}(\hat{x} - \hat{y}) - (x - y) &= x(1+\varepsilon_x) - y(1+\varepsilon_y) - x + y \\ &= x\varepsilon_x - y\varepsilon_y.\end{aligned}$$

The relative error is bounded by:

$$\begin{aligned}\frac{|x\varepsilon_x - y\varepsilon_y|}{x-y} &\leq \frac{x|\varepsilon_x| + y|\varepsilon_y|}{x-y} \\ &\leq \frac{xu + yu}{x-y} \\ &= u \cdot \frac{x+y}{x-y}.\end{aligned}$$

As $x \to y$, the denominator $x - y \to 0$, causing the relative error to explode. □

Remark 1.30 **Numerical Stability Principle:** Rounding error is an inevitable consequence of finite precision. However, catastrophic cancellation is an algorithmic flaw that can often be avoided by mathematically rearranging the problem.

Example **(Stabilizing Cancellation)** $f(x) = \sqrt{x+1} - \sqrt{x}$ for $x = 10^8$.

- **Direct:** $10000.00000005 - 10000 \approx 0$ (Catastrophic).
- **Stable:** $\frac{1}{\sqrt{x+1}+\sqrt{x}} \approx 5 \times 10^{-5}$.

Example **(Quadratic formula)** The roots of $ax^2 + bx + c = 0$ are

$$x_{\pm} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

If $b^2 \gg 4ac$, one numerator subtracts nearly equal numbers. A stable strategy computes the root with no cancellation first, then uses $x_1 x_2 = c/a$ to recover the other root.

Remark 1.31 **(Use stable library functions)** Functions such as `log1p(x)`, `expm1(x)`, and `hypot(x, y)` implement stable rearrangements for common cancellation and overflow patterns. Prefer them over hand-expanded formulas when the inputs may be small, large, or nearly cancelling.

Ex 1.26 Stabilize:

1. Use Def 1.25 to explain why $(1-\cos x)/x^2$ is unstable for small x, then derive a stable form.
2. Use Def 1.25 to identify which root of $x^2 - 10^8 x + 1 = 0$ is unstable in the quadratic formula, then compute it using $x_1 x_2 = c/a$.
3. Use the Taylor series of e^x to explain why $e^x - 1$ loses relative accuracy for small x. Compare with `np.expm1`.

1.7.2 Error Analysis Frameworks

- **Forward Error Analysis:** Tracks the accumulation of error from inputs to outputs.
- **Backward Error Analysis:** Determines the input perturbation that would make the computed result exact.
- **Backward Stability:** An algorithm is backward stable if the backward error is $O(u)$, where u is the unit roundoff.

Forward error asks whether the computed answer is close to the exact answer. Backward error asks whether the computed answer is the exact answer to a nearby problem. Backward error is often easier to prove and more useful in practice: if the data are uncertain at the level of the backward error, the algorithm has not introduced a meaningful additional error.

Def 1.26 **(Condition Number (κ))** Inherent sensitivity of the problem. Forward Error $\leq \kappa \cdot$ Backward Error.

Remark 1.32 **(Stability vs. conditioning)** Conditioning is a property of the mathematical problem. Stability is a property of the algorithm. A stable algorithm can still return a poor answer on an ill-conditioned problem, because the problem itself amplifies small perturbations.

Ex 1.27

1. $\kappa(A) = 10^8$, Backward Error $= 10^{-15}$. Max forward error?
2. Prove BE for linear systems: $(\mathbf{A}+\mathbf{E})\hat{\boldsymbol{x}} = \boldsymbol{b}$ for $\mathbf{E} = -\boldsymbol{r}\hat{\boldsymbol{x}}^T/\|\hat{\boldsymbol{x}}\|^2$.

Proof *Proof.* **(1):** Substituting $\mathbf{E}$ into $(\mathbf{A}+\mathbf{E})\hat{\boldsymbol{x}}$:

$$\begin{aligned}(\mathbf{A}+\mathbf{E})\hat{\boldsymbol{x}} &= \mathbf{A}\hat{\boldsymbol{x}} + \mathbf{E}\hat{\boldsymbol{x}} \\ &= \mathbf{A}\hat{\boldsymbol{x}} - \frac{\boldsymbol{r}\hat{\boldsymbol{x}}^T}{\|\hat{\boldsymbol{x}}\|^2}\hat{\boldsymbol{x}} \\ &= \mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{r}\frac{\hat{\boldsymbol{x}}^T\hat{\boldsymbol{x}}}{\|\hat{\boldsymbol{x}}\|^2} \\ &= \mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{r}.\end{aligned}$$

Since $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$, we have $\mathbf{A}\hat{\boldsymbol{x}} - (\boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}) = \boldsymbol{b}$.
(2): For the relative error, using the rank-1 norm property $\|\boldsymbol{r}\boldsymbol{v}^T\|_2 = \|\boldsymbol{r}\|_2\|\boldsymbol{v}\|_2$:

$$\begin{aligned}\|\mathbf{E}\|_2 &= \left\| \frac{\boldsymbol{r}\hat{\boldsymbol{x}}^T}{\|\hat{\boldsymbol{x}}\|^2} \right\|_2 \\ &= \frac{\|\boldsymbol{r}\|_2\|\hat{\boldsymbol{x}}\|_2}{\|\hat{\boldsymbol{x}}\|^2} = \frac{\|\boldsymbol{r}\|_2}{\|\hat{\boldsymbol{x}}\|_2}.\end{aligned}$$

Dividing by $\|\mathbf{A}\|_2$ gives the result:

$$\frac{\|\mathbf{E}\|_2}{\|\mathbf{A}\|_2} = \frac{\|\boldsymbol{r}\|_2}{\|\mathbf{A}\|_2\|\hat{\boldsymbol{x}}\|_2}.$$

□

1.8 Python and NumPy Primer

Condensed reference for the toolset used throughout these notes.

1.8.1 Arrays and Basic Operations

Def 1.27 **(NumPy Arrays)** The `ndarray` is the fundamental structure.

- **Creation:** `np.array([1, 2])`, `np.zeros((m, n))`, `np.random.randn(m, n)`.
- **Indexing:** 0-indexed. `A[i, j]` for entries; `A[i, :]` for rows; `A[:, j]` for columns.

Remark 1.33 **(Shapes and Rank-1 Arrays)** `A.shape` returns `(m, n)`. Note that a 1D array has shape `(n,)`, which is **not** equivalent to a column vector `(n, 1)`. These "rank-1 arrays" are a common source of broadcasting bugs. Use `x[:, None]` or `x.reshape(-1, 1)` to convert to 2D.

Ex 1.28

1. Create 4×4 identity matrix. Verify `shape` and `dtype`.
2. Extract row 2 and column 3 from a random 3×3 $\mathbf{A}$.
3. Confirm that `np.zeros((3,))` fails where a `(3, 1)` column vector is required in a matrix-vector product.

1.8.2 Linear Algebra Operations

Core routines in `numpy.linalg`:

Function	Computes
`A @ B`	Matrix product $\mathbf{AB}$
`np.linalg.solve(A, b)`	Solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ ($O(n^3)$)
`np.linalg.inv(A)`	Matrix inverse (Avoid; use `solve`)
`np.linalg.norm(x)`	Euclidean norm $\|\boldsymbol{x}\|_2$
`np.linalg.eig(A)`	Eigenvalues and eigenvectors
`np.linalg.svd(A)`	Singular Value Decomposition
`np.linalg.qr(A)`	QR Factorization
`np.linalg.cond(A)`	Condition number $\kappa_2(\mathbf{A})$

Remark 1.34 **(Performance Law: Vectorization)** Explicit `for` loops in Python are slow. Push heavy lifting into NumPy/BLAS routines by **vectorizing** operations. For example, use `A @ B` instead of triple-nested loops.

Remark 1.35 **(Stability: solve vs inv)** `np.linalg.solve(A, b)` is faster and more stable than `np.linalg.inv(A) @ b`. Explicit inversion is almost never desirable in scientific computation.

Remark 1.36 **(Computed equality)** Use `np.allclose(x, y)` or `np.isclose(x, y)` for computed floating-point quantities. Exact equality is appropriate for integers, booleans, and deliberately exact structural checks.

Remark 1.37 **(Sparse matrices)** Large discretized PDE and graph problems should usually use `scipy.sparse`, not dense NumPy arrays. Use `scipy.sparse.linalg` for sparse solves, eigenvalue computations, CG, and GMRES.

Ex 1.29

1. Solve a 2×2 system and verify the residual $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|$.
2. Profile `solve` vs `inv @ b` for $n = 1000$.
3. Compare `np.linalg.eig` and `np.linalg.eigh` (optimized for symmetric matrices).

1.8.3 Key Idioms

- **Element-wise:** `*` and `**` are entry-wise. Matrix powers use `np.linalg.matrix_power`.
- **Broadcasting:** Automatic dimension expansion. `A * v` scales columns of $\mathbf{A}$ by entries of v.
- **Aliases, Views, Copies:** `B = A` makes a second name for the same array. Slices are usually views. Use `B = A.copy()` for independent data.
- **Sparse construction:** use `scipy.sparse.diags` for banded matrices and `scipy.sparse.csr_m` for efficient arithmetic.

Ex 1.30

1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$. Compare `A * A` and `A @ A`.
2. Demonstrate the "view pitfall" by modifying a slice of a matrix.
3. Use broadcasting to perform diagonal scaling $\mathbf{AD}$ without constructing $\mathbf{D} = \text{diag}(d)$.

2 Direct Methods for Linear Systems

2.1 Linearity

Linear systems are the basic computational problem in scientific computing. Even when the original problem is nonlinear, algorithms often reduce it to a sequence of linear systems by linearizing the problem near the current estimate.

A matrix equation $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ asks whether the data vector $\boldsymbol{b}$ can be produced by combining the columns of $\mathbf{A}$. The unknown vector $\boldsymbol{x}$ contains the weights in that combination. Existence, uniqueness, and numerical difficulty are all controlled by the geometry of those columns.

2.1.1 Linear Systems

A linear system is a collection of linear equations involving the same set of variables. In matrix-vector notation, we package these equations into a single compact form that is amenable to both theoretical analysis and high-performance computation.

Def 2.1 **(Linear System)** The matrix-vector form of a linear system is $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, where:

- $\mathbf{A} \in \mathbb{R}^{m\times n}$ is the coefficient matrix representing the system's structure.
- $\boldsymbol{x} \in \mathbb{R}^n$ is the vector of unknown variables we wish to determine.
- $\boldsymbol{b} \in \mathbb{R}^m$ is the right-hand side vector, often representing observed data or a source term.

Remark 2.1 **(Shape discipline)** If $\mathbf{A} \in \mathbb{R}^{m\times n}$, then $\boldsymbol{x}$ must have n entries and $\boldsymbol{b}$ must have m entries. The number of columns is the number of unknowns. The number of rows is the number of equations or measurements.

Example **(Two equations)** The system

$$x + 2y = 3, \qquad 3x - y = 1$$

can be written as

$$\begin{pmatrix} 1 & 2 \\ 3 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}.$$

The first row encodes the first equation; the second row encodes the second equation.

Ex 2.1

1. Convert $\{x + 2y = 3,\ 3x - y = 1\}$ to the form in Def 2.1.

2. Check the dimensions of $\mathbf{A}$, $\boldsymbol{x}$, and $\boldsymbol{b}$ before solving. Which part of Def 2.1 determines each shape?

3. Solve in NumPy via `np.linalg.solve(A, b)`. Verify with the residual from Def 2.2.

2.1.2 Rows, Columns, and Geometry

There are two complementary ways to read $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$.

- **Row view:** Each row is one linear equation. In two or three dimensions, each equation describes a line, plane, or hyperplane. Solving the system means finding their intersection.
- **Column view:** The product $\mathbf{A}\boldsymbol{x}$ is a weighted sum of the columns of $\mathbf{A}$. Solving the system means expressing $\boldsymbol{b}$ as a linear combination of those columns.

Remark 2.2 **(The Column Perspective)** If $\mathbf{A} = [\boldsymbol{a}_1, \boldsymbol{a}_2, \ldots, \boldsymbol{a}_n]$, then

$$\mathbf{A}\boldsymbol{x} = x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2 + \cdots + x_n\boldsymbol{a}_n.$$

The system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is consistent exactly when $\boldsymbol{b}$ lies in the span of the columns of $\mathbf{A}$.

Thm 2.1 **(Superposition Principle)** If $\mathbf{A}\boldsymbol{x}_1 = \boldsymbol{b}_1$ and $\mathbf{A}\boldsymbol{x}_2 = \boldsymbol{b}_2$, then for any scalars α, β:

$$\mathbf{A}(\alpha\boldsymbol{x}_1 + \beta\boldsymbol{x}_2) = \alpha\boldsymbol{b}_1 + \beta\boldsymbol{b}_2.$$

Implication: Solutions to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ can be decomposed and recombined linearly.

Ex 2.2

1. Use Thm 2.1 to solve $\mathbf{A}\boldsymbol{x} = 3\boldsymbol{b}_1 - 2\boldsymbol{b}_2$ given only the solutions $\boldsymbol{x}_1, \boldsymbol{x}_2$ for $\boldsymbol{b}_1, \boldsymbol{b}_2$.

2. How does `np.linalg.solve(A, B)` for a matrix $\mathbf{B}$ exploit this?

2.1.3 Invertibility and Existence

For square systems ($\mathbf{A} \in \mathbb{R}^{n\times n}$), the most critical question is whether a unique solution exists for any given right-hand side. This property is known as invertibility or nonsingularity.

For rectangular systems, the same questions still matter but the answer changes.

- **Square:** $m = n$. A unique solution may exist.
- **Overdetermined:** $m > n$. There are more equations than unknowns; usually no exact solution exists, so we solve a least squares problem.

- **Underdetermined:** $m < n$. There are fewer equations than unknowns; if a solution exists, it is usually not unique.

Thm 2.2 **(Equivalences for Invertibility)** The following are equivalent for $\mathbf{A} \in \mathbb{R}^{n \times n}$:

1. $\mathbf{A}$ is **invertible** ($\det(\mathbf{A}) \neq 0$).
2. $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a **unique solution** for every $\boldsymbol{b}$.
3. $\mathbf{A}\boldsymbol{x} = \mathbf{0}$ has only the **trivial solution** ($\boldsymbol{x} = \mathbf{0}$).
4. $\operatorname{rank}(\mathbf{A}) = n$ (Full Rank).
5. Columns of $\mathbf{A}$ are **linearly independent**.

Remark 2.3 **(Determinants are not algorithms)** The determinant is useful theoretically, but numerical codes do not test invertibility by computing $\det(\mathbf{A})$. Rank, singular values, and condition numbers are more informative and more stable.

Ex 2.3

1. Determine invertibility using Thm 2.2: $\mathbf{A} = \begin{pmatrix} 2 & -1 \\ 4 & -2 \end{pmatrix}$, $\mathbf{B} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}$.
2. Check consistency of $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 6 \end{pmatrix}, \boldsymbol{b} = \begin{pmatrix} 1 \\ 4 \end{pmatrix}$ using the column perspective in Remark 2.2 and `np.linalg.matrix_rank`.

2.1.4 Residuals and Approximate Solutions

Def 2.2 **(Residual)** For an approximate solution $\hat{\boldsymbol{x}}$ to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, the residual is

$$\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}.$$

It measures how well $\hat{\boldsymbol{x}}$ satisfies the equations.

Remark 2.4 **(Residual vs. error)** The error is $\hat{\boldsymbol{x}} - \boldsymbol{x}^*$, where $\boldsymbol{x}^*$ is the exact solution. The residual is computable; the error usually is not. A small residual means the equations are nearly satisfied, but it does not always mean the solution is accurate. Ill-conditioned matrices can turn small residuals into large errors.

Example **(Inconsistent system)** The equations

$$x + y = 1, \qquad x + y = 3$$

cannot both be true. In matrix form, $\boldsymbol{b}$ is not in the column space of $\mathbf{A}$. The best we can do is choose $\hat{\boldsymbol{x}}$ so that the residual $\boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$ is small in a chosen norm. This is the least squares problem.

2.1.5 Linear Combinations and Bases

Def 2.3 **(Linear Independence)** Vectors $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\}$ are independent if $\sum \alpha_i \boldsymbol{x}_i = \mathbf{0} \Rightarrow \alpha_i = 0$ for all i.

Def 2.4 **(Basis)** A linearly independent set that spans the space. The number of vectors in any basis is the **dimension**.

2.1.6 Rank

The rank of a matrix is the number of independent directions its columns span. It measures how many genuinely different constraints or directions are present after removing redundancy.

Def 2.5 **(Rank)** The rank, $\text{rank}(\mathbf{A})$, is the dimension of the column space. It is always true that $\text{rank}(\mathbf{A}) \leq \min(m, n)$.

- **Full Rank:** The matrix has the maximum possible rank, $\text{rank}(\mathbf{A}) = \min(m, n)$.
- **Rank Deficient (Singular):** A square matrix with $\text{rank}(\mathbf{A}) < n$. Such systems do not have unique solutions.

Remark 2.5 **(Rank and solution structure)** For $\mathbf{A} \in \mathbb{R}^{m \times n}$:

- Full column rank ($\text{rank}(\mathbf{A}) = n$) means the columns are independent.
- Full row rank ($\text{rank}(\mathbf{A}) = m$) means the rows are independent.
- Rank deficiency means there is redundancy: some row or column information can be produced from the rest.

Remark 2.6 **(Numerical Rank)** In code, do not use exact row-reduction to find rank; it is numerically unstable. Use `np.linalg.matrix_rank`, which estimates rank from the singular values. A matrix can be mathematically full rank but numerically rank deficient if one singular value is tiny compared to the others.

Thm 2.3 **(Rank-Nullity)** For $\mathbf{A} \in \mathbb{R}^{m \times n}$: $\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = n$.

- **Nullity:** Dimension of the null space $\{\boldsymbol{x} : \mathbf{A}\boldsymbol{x} = \mathbf{0}\}$.

Remark 2.7 **(General solution form)** If $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is consistent and $\boldsymbol{x}_p$ is one particular solution, then every solution has the form

$$\boldsymbol{x} = \boldsymbol{x}_p + \boldsymbol{z}, \qquad \boldsymbol{z} \in \mathcal{N}(\mathbf{A}).$$

The null space contains the directions you can add without changing $\mathbf{A}\boldsymbol{x}$.

Ex 2.4

1. Use Thm 2.3 to explain why a "wide" matrix ($m < n$) must have a non-trivial null space.
2. Find the general solution to a consistent 2×3 system in the form $\boldsymbol{x} = \boldsymbol{x}_p + \boldsymbol{z}$, where $\boldsymbol{z} \in \mathcal{N}(\mathbf{A})$.
3. Check directly that adding a null-space vector leaves $\mathbf{A}\boldsymbol{x}$ unchanged.

2.1.7 The Four Fundamental Subspaces

The structure of any linear operator $\mathbf{A} \in \mathbb{R}^{m \times n}$ with rank r is completely characterized by four subspaces. Understanding these spaces is essential for solving least squares problems and understanding the behavior of iterative solvers.

1. **Column Space ($\mathcal{R}(\mathbf{A})$):** $\mathbb{R}^m, \dim = r$. Span of columns.
2. **Null Space ($\mathcal{N}(\mathbf{A})$):** $\mathbb{R}^n, \dim = n - r$. Solutions to $\mathbf{A}\boldsymbol{x} = \mathbf{0}$.
3. **Row Space ($\mathcal{R}(\mathbf{A}^T)$):** $\mathbb{R}^n, \dim = r$. Span of rows.
4. **Left Null Space ($\mathcal{N}(\mathbf{A}^T)$):** $\mathbb{R}^m, \dim = m - r$. Solutions to $\mathbf{A}^T\boldsymbol{y} = \mathbf{0}$.

Thm 2.4 **(Orthogonal Complements)**

- $\mathcal{R}(\mathbf{A}^T) \perp \mathcal{N}(\mathbf{A})$ in $\mathbb{R}^n$.
- $\mathcal{R}(\mathbf{A}) \perp \mathcal{N}(\mathbf{A}^T)$ in $\mathbb{R}^m$.

Solvability: $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a solution iff $\boldsymbol{b} \in \mathcal{R}(\mathbf{A})$, i.e., $\boldsymbol{b} \perp \mathcal{N}(\mathbf{A}^T)$.

Ex 2.5

1. For $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$, use Def 2.5 and Thm 2.3 to find dimensions and bases for all four spaces.
2. Verify the two orthogonality statements in Thm 2.4.
3. Verify $\boldsymbol{b} = (1, 3)^T$ is not in the column space. What is the projection of $\boldsymbol{b}$ onto $\mathcal{R}(\mathbf{A})$?

2.2 LU Factorization

Most direct methods for solving linear systems rely on matrix factorizations that reduce a complex system into multiple simpler ones. The **LU factorization** is the most fundamental of these, decomposing a square matrix $\mathbf{A}$ into a unit lower triangular matrix $\mathbf{L}$ and an upper triangular matrix $\mathbf{U}$. This process is essentially the matrix form of Gaussian elimination.

Gaussian elimination transforms $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ into an equivalent upper triangular system. LU factorization stores the elimination steps, so the same work can be reused for many right-hand sides.

2.2.1 LU and Gaussian Elimination

Triangular systems are easy because the unknowns can be recovered one at a time. If $\mathbf{U}$ is upper triangular, the last equation contains only x_n, the next contains only x_{n-1} and x_n, and so on. This is backward substitution. Lower triangular systems are solved similarly by forward substitution.

Def 2.6 **(LU Factorization)** $\mathbf{A} = \mathbf{LU}$:

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & \dots \\ \ell_{21} & 1 & \dots \\ \vdots & \vdots & \ddots \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} u_{11} & u_{12} & \dots \\ 0 & u_{22} & \dots \\ \vdots & \vdots & \ddots \end{pmatrix}.$$

The matrix $\mathbf{U}$ contains the row-echelon form produced by elimination. The matrix $\mathbf{L}$ stores the multipliers used to eliminate entries below each pivot. If row i is updated by subtracting ℓ_{ik} times row k, then ℓ_{ik} appears in $\mathbf{L}$.

Example **(One elimination)** Let

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 6 & 5 \end{pmatrix}.$$

To eliminate the entry 6 below the pivot 2, subtract 3 times row 1 from row 2:

$$\begin{pmatrix} 2 & 1 \\ 6 & 5 \end{pmatrix} \longrightarrow \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}.$$

The multiplier is $\ell_{21} = 3$, so

$$\mathbf{L} = \begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix}, \qquad \mathbf{U} = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}, \qquad \mathbf{A} = \mathbf{LU}.$$

Ex 2.6

1. Compare free parameters: how many in $\mathbf{L}$ vs. $\mathbf{U}$? Do they sum to n^2?
2. If $\mathbf{A} = \mathbf{LU}$ is invertible, prove $\mathbf{L}$ and $\mathbf{U}$ are also invertible.

Remark 2.8 **(Existence and Uniqueness)** $\mathbf{A}$ admits a unique LU factorization iff all leading principal submatrices $\mathbf{A}_k$ are nonsingular ($k = 1, \ldots, n-1$). The algorithm fails if a zero pivot is encountered.

Remark 2.9 **(Elimination without pivoting is fragile)** A zero pivot causes the algebraic algorithm to stop. A tiny pivot may be worse numerically: dividing by a tiny number creates huge multipliers, which amplify rounding errors in later rows.

Remark 2.10 **(Flop Cost)** Computing $\mathbf{L}$ and $\mathbf{U}$ via Gaussian elimination costs $\frac{2}{3}n^3$ **flops**.

Remark 2.11 **(The Solve Pattern)** To solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ given $\mathbf{A} = \mathbf{LU}$:

1. Solve $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$ via forward substitution ($O(n^2)$).
2. Solve $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$ via backward substitution ($O(n^2)$).

Advantage: Once $\mathbf{A}$ is factored, each new $\boldsymbol{b}$ costs only $O(n^2)$.

Example **(Solving with the factors)** For the previous example, solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with $\boldsymbol{b} = (3, 11)^T$. First solve $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$:

$$y_1 = 3, \qquad 3y_1 + y_2 = 11 \Rightarrow y_2 = 2.$$

Then solve $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$:

$$2x_2 = 2 \Rightarrow x_2 = 1, \qquad 2x_1 + x_2 = 3 \Rightarrow x_1 = 1.$$

The solution is $\boldsymbol{x} = (1, 1)^T$.

Ex 2.7

1. Factor $\mathbf{A} = \begin{pmatrix} 2 & 3 & 1 \\ 4 & 7 & 3 \\ -2 & -3 & 1 \end{pmatrix}$ by hand.
2. Solve $\mathbf{A}\boldsymbol{x} = (5, 15, 4)^T$ using your factors.
3. SciPy: Use `scipy.linalg.lu` and explain why it always returns a permutation $\mathbf{P}$.

2.2.2 Pivoting and Stability

While mathematically elegant, basic LU factorization is numerically unstable if any pivot (the diagonal element used to eliminate entries below it) is small relative to the entries it is eliminating. This can lead to catastrophic rounding errors. **Partial Pivoting** mitigates

this by ensuring that at each step, we swap rows to place the largest available entry in the pivot position.

Example **(Small pivot)** Consider

$$\mathbf{A} = \begin{pmatrix} 10^{-20} & 1 \\ 1 & 1 \end{pmatrix}.$$

Without pivoting, the first multiplier is 10^{20}. The elimination step creates huge intermediate entries even though the original matrix has modest entries. Partial pivoting swaps the two rows first, using the pivot 1 instead of 10^{-20}.

Def 2.7 **(LUP Decomposition)** The result of Gaussian elimination with partial pivoting is recorded as $\mathbf{PA} = \mathbf{LU}$, where $\mathbf{P}$ is a permutation matrix that tracks the row swaps.

Remark 2.12 **(Permutation matrices)** Multiplying by $\mathbf{P}$ reorders rows. Permutation matrices are orthogonal: $\mathbf{P}^{-1} = \mathbf{P}^T$. Applying a permutation is cheap because no arithmetic is required; the code only reorders indices.

Remark 2.13 **(Stability and Growth Factor)** The numerical stability of LUP is characterized by the growth factor $\rho = \max |u_{ij}| / \max |a_{ij}|$. If ρ is large, the solution may be inaccurate. Partial pivoting keeps ρ small in practice (typically $O(n)$), ensuring the algorithm is backward stable. While the theoretical worst-case growth is 2^{n-1}, such matrices are rarely encountered in engineering applications.

Ex 2.8

1. Give a 2×2 matrix where LU fails (due to a zero or small pivot) but LUP succeeds.
2. Why is $\mathbf{P}^{-1}$ trivial to compute for any permutation matrix? (*Hint: consider* $\mathbf{P}^T$).
3. Complete pivoting (swapping both rows and columns) provides even stronger stability guarantees but is rarely used. Why? (Consider the trade-off between search cost and marginal stability gain).

2.2.3 Algorithm Cost Analysis

Understanding the computational cost is vital for choosing the right solver for large-scale engineering problems. The bulk of the work is in the factorization step.

There are two separate costs:

- **Factorization cost:** Build $\mathbf{L}$ and $\mathbf{U}$ once. This costs $O(n^3)$.
- **Solve cost:** Use the factors for a specific $\boldsymbol{b}$. This costs $O(n^2)$.

This distinction is the reason factorizations are valuable.

Remark 2.14 **(Total Cost for k Right-Hand Sides)** For a system with n unknowns and k different right-hand side vectors $\boldsymbol{b}$, the total cost is:

$$\text{Cost} = \frac{2}{3}n^3 + 2kn^2 \text{ flops.}$$

When k is large, the $O(n^3)$ cost of factorization is amortized over many $O(n^2)$ solves. For example, solving for 500 $\boldsymbol{b}$'s for $n = 1000$ using one factorization is significantly faster than re-factorizing for each new $\boldsymbol{b}$.

Proof *Proof.* Forward substitution solves $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$ where $\ell_{ii} = 1$. The i-th entry y_i is given by:

$$y_i = b_i - \sum_{j=1}^{i-1} \ell_{ij} y_j.$$

Counting the floating-point operations:

- For a fixed i, the summation requires $i-1$ multiplications and $i-1$ additions/subtractions.
- Total flops: $\sum_{i=1}^{n} 2(i-1) = 2\sum_{k=0}^{n-1} k = 2\frac{(n-1)n}{2} = n^2 - n$.

For large n, each triangular solve costs approximately n^2 flops. □

Remark 2.15 **(Implementation rule)** In NumPy/SciPy, use `solve` for one right-hand side or `lu_factor`/`lu_solve` when reusing a factorization. Do not form $\mathbf{A}^{-1}$ to solve a linear system.

Ex 2.9

1. Sum the operations for forward substitution and show the cost is exactly $n^2 - n$.
2. Use `scipy.linalg.lu_factor` and `lu_solve` for a system with 3 right-hand sides. Verify residuals.

2.3 Symmetric Positive Definite (SPD) Matrices

Symmetric Positive Definite (SPD) matrices form one of the most important classes of matrices in scientific computing. They possess several remarkable properties that allow for algorithms that are twice as fast and significantly more stable than those for general matrices.

An SPD matrix is the matrix version of positive curvature or positive energy. The quadratic form $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}$ is always positive away from the origin, so it defines a geometry, a convex energy landscape, and a stable class of linear systems.

2.3.1 Definitions and Properties

SPD matrices arise naturally in a wide variety of engineering contexts, including optimization (as Hessian matrices of convex functions), statistics (as covariance matrices), and physical simulations (as stiffness matrices in structural mechanics).

The expression $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}$ is called a **quadratic form**. It converts a vector into a scalar. If $\mathbf{A}$ is symmetric, then this scalar measures the energy of $\boldsymbol{x}$ in the geometry defined by $\mathbf{A}$.

Def 2.8 **(Positive Definite)** A symmetric matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is **positive definite** (PD) if the quadratic form $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}$ is strictly positive for every nonzero vector $\boldsymbol{x} \in \mathbb{R}^n$:

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0 \quad \text{for every nonzero } \boldsymbol{x} \in \mathbb{R}^n.$$

If $\mathbf{A}$ is both symmetric and PD, it is **SPD**.

Def 2.9 **(Positive Semidefinite)** A symmetric matrix $\mathbf{A}$ is **positive semidefinite** (PSD) if

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} \geq 0 \quad \text{for every } \boldsymbol{x} \in \mathbb{R}^n.$$

Positive definite means strictly positive for every nonzero vector. Positive semidefinite allows flat directions where $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = 0$.

Example **(Definite vs. semidefinite)**

$$\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \text{ is SPD, while } \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \text{ is PSD but not PD.}$$

The second matrix has a flat direction: $(0, 1)^T$ has zero quadratic energy.

Ex 2.10

1. Test $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ for PD property by expanding $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}$.
2. Prove a diagonal matrix $\mathbf{D}$ is PD iff all diagonal entries $d_i > 0$.

Remark 2.16 **(Spectral Characterization)** A symmetric matrix is PD iff **all its eigenvalues are strictly positive**. Every SPD matrix is invertible ($\det \mathbf{A} = \prod \lambda_i > 0$).

Remark 2.17 **(Equivalent tests for SPD)** For a symmetric matrix $\mathbf{A}$, the following are equivalent:

1. $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0$ for every nonzero $\boldsymbol{x}$.
2. Every eigenvalue of $\mathbf{A}$ is positive.
3. Cholesky factorization succeeds with positive pivots.
4. All leading principal minors are positive.

The eigenvalue and Cholesky tests are the most useful computationally.

Ex 2.11

1. If $\mathbf{A}$ is SPD, show $\mathbf{B} = \mathbf{A} + c\mathbf{I}$ for $c > 0$ is also SPD.
2. Use `np.linalg.eigvalsh` to verify SPD property for several test matrices.

2.3.2 Geometry and Metric

An SPD matrix $\mathbf{A}$ can be thought of as defining a new geometry or "metric" on $\mathbb{R}^n$, where the distance and angle between vectors are weighted by the entries of $\mathbf{A}$. This perspective is fundamental to understanding the behavior of iterative solvers like the Conjugate Gradient method.

If $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$, then

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = \sum_{i=1}^{n} \lambda_i z_i^2, \qquad \boldsymbol{z} = \mathbf{Q}^T\boldsymbol{x}.$$

The eigenvectors give the principal directions of the geometry, and the eigenvalues determine how expensive motion is in each direction.

Def 2.10 **(A-Inner Product)** For SPD $\mathbf{A}$, the mapping $\langle \boldsymbol{x}, \boldsymbol{y} \rangle_{\mathbf{A}} = \boldsymbol{x}^T\mathbf{A}\boldsymbol{y}$ satisfies the axioms of an inner product.

- **A-Norm:** The induced norm is $\|\boldsymbol{x}\|_{\mathbf{A}} = \sqrt{\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}}$.
- **Ellipsoid:** The unit ball in this norm, $\{\boldsymbol{x} : \|\boldsymbol{x}\|_{\mathbf{A}} = 1\}$, is an n-dimensional ellipsoid whose principal axes are aligned with the eigenvectors of $\mathbf{A}$.

Remark 2.18 **Physical Meaning:** Directions with large eigenvalues are "stiff" or "expensive" (small axes in the unit ellipsoid); small eigenvalues represent "sensitive" directions.

Example **(Ellipse)** For

$$\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix},$$

the unit $\mathbf{A}$-norm curve is

$$4x_1^2 + x_2^2 = 1.$$

The axis in the x_1 direction is shorter because that direction has larger eigenvalue. Moving in that direction costs more energy.

Ex 2.12

1. For $\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, compute the distance between $(1, 0)^T$ and origin in both Euclidean and $\mathbf{A}$-metrics.

2. Where does the $\mathbf{A}$-metric appear in statistics? (Mahalanobis distance).

2.3.3 Cholesky Decomposition

The symmetry and positive definiteness of $\mathbf{A}$ allow for a specialized version of the LU factorization known as the **Cholesky decomposition**. The decomposition can be interpreted as a generalized square root for SPD matrices, where $\mathbf{A} = \mathbf{L}\mathbf{L}^T$.

Cholesky is the natural direct solver for SPD systems. It stores only one triangular factor instead of two, avoids pivoting, and preserves the symmetry of the problem.

Def 2.11 **(Cholesky Decomposition)** Every SPD matrix $\mathbf{A}$ admits a unique decomposition $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is lower triangular with strictly positive diagonal entries.

Example **($\mathbf{A}$ 2×2 Cholesky factor)** Let

$$\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}.$$

Assume

$$\mathbf{L} = \begin{pmatrix} \ell_{11} & 0 \\ \ell_{21} & \ell_{22} \end{pmatrix}.$$

Matching $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ gives $\ell_{11} = 2$, $\ell_{21} = 1$, and $\ell_{22} = \sqrt{2}$, so

$$\mathbf{L} = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix}.$$

Remark 2.19 **(Flop Cost)** The computational cost of the Cholesky decomposition is $\frac{1}{3}n^3$ **flops**, approximately half that of a general LU factorization.

Remark 2.20 **Stability Advantage:** Unlike general LU, Cholesky is **guaranteed to be stable without pivoting**. In code, `np.linalg.cholesky` is the standard numerical test for positive definiteness; if it raises a `LinAlgError`, the matrix is not PD.

Remark 2.21 **(Solving with Cholesky)** If $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, then solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ requires two triangular solves:

1. Solve $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$.

2. Solve $\mathbf{L}^T\boldsymbol{x} = \boldsymbol{y}$.

The factorization costs $O(n^3)$, but each solve costs only $O(n^2)$.

Ex 2.13

1. Factor $\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$ by hand.
2. Why does Cholesky not require row swaps?
3. Solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ using Cholesky for $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$.

Ex 2.14 **(Applications)**

1. **Covariance:** Generate `X = np.random.randn(100, 4)`. Verify `X.T @ X` is SPD via Cholesky.
2. **Gram Matrix:** Prove that $\mathbf{G} = \mathbf{X}^T\mathbf{X}$ is at least positive semidefinite for any $\mathbf{X}$, and PD if $\mathbf{X}$ has full column rank.

Remark 2.22 **(Where SPD matrices appear)**

- **Optimization:** Hessians of strictly convex quadratic functions.
- **Statistics:** Covariance and Gram matrices, often PSD and SPD when data are full rank.
- **PDEs:** Discrete Laplacians and stiffness matrices for elliptic problems.
- **Least squares:** Normal equations $\mathbf{A}^T\mathbf{A}\boldsymbol{x} = \mathbf{A}^T\boldsymbol{b}$ are SPD when $\mathbf{A}$ has full column rank.

Remark 2.23 **(Connection to CG)** Conjugate Gradient is designed for large sparse SPD systems. It minimizes the quadratic energy

$$\phi(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} - \boldsymbol{b}^T\boldsymbol{x},$$

whose unique minimizer satisfies $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ when $\mathbf{A}$ is SPD.

Proof *Proof.* For any nonzero vector $\boldsymbol{v} \in \mathbb{R}^n$, consider the quadratic form:

$$\begin{aligned} \boldsymbol{v}^T\mathbf{G}\boldsymbol{v} &= \boldsymbol{v}^T(\mathbf{X}^T\mathbf{X})\boldsymbol{v} \\ &= (\mathbf{X}\boldsymbol{v})^T(\mathbf{X}\boldsymbol{v}) \\ &= \|\mathbf{X}\boldsymbol{v}\|_2^2. \end{aligned}$$

Since the squared Euclidean norm is always non-negative, $\boldsymbol{v}^T\mathbf{G}\boldsymbol{v} \geq 0$, proving $\mathbf{G}$ is positive semidefinite. If $\mathbf{X}$ has full column rank, then $\mathbf{X}\boldsymbol{v} = \mathbf{0}$ iff $\boldsymbol{v} = \mathbf{0}$. Thus $\boldsymbol{v}^T\mathbf{G}\boldsymbol{v} > 0$ for all nonzero $\boldsymbol{v}$, making $\mathbf{G}$ positive definite. □

Ex 2.15 **(SPD Condition Number)** $\kappa_2(\mathbf{A}) = \lambda_{\max}/\lambda_{\min}$.

1. If $\mathbf{A}$ is SPD with eigenvalues $\{10, 0.01\}$, what is $\kappa_2(\mathbf{A})$?
2. Verify $\kappa_2(\mathbf{L}) = \sqrt{\kappa_2(\mathbf{A})}$ where $\mathbf{L}$ is the Cholesky factor.

3 Orthogonality and Least Squares

3.1 Orthogonal Matrices

Transformations that preserve lengths and angles. Numerically ideal due to optimal conditioning ($\kappa_2(\mathbf{Q}) = 1$).

Orthogonal transformations rotate or reflect space without stretching it. Because they preserve lengths, they do not amplify errors in the Euclidean norm. This is why stable numerical linear algebra tries to use orthogonal transformations whenever possible.

3.1.1 Orthogonality and Orthonormal Bases

Def 3.1 **(Orthogonal Vectors)** $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ are orthogonal if $\boldsymbol{x}^T\boldsymbol{y} = 0$.

- **Mutually Orthogonal Set:** $\boldsymbol{v}_i^T\boldsymbol{v}_j = 0$ for all $i \neq j$.

Def 3.2 **(Orthonormal Basis)** A set $\{\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n\}$ where $\boldsymbol{q}_i^T\boldsymbol{q}_j = \delta_{ij}$ (unit length and mutually orthogonal).

- **Matrix Form:** If $\mathbf{Q} = [\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n]$, then $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$.

Remark 3.1 **(Efficiency in Orthonormal Bases)** In an orthonormal basis, the coefficients of a vector $\boldsymbol{v} = \sum c_i\boldsymbol{q}_i$ are given by standard inner products: $c_i = \boldsymbol{q}_i^T\boldsymbol{v}$. This allows the solution of a linear system, which generally requires $O(n^3)$ operations, to be computed using a sequence of inner products in $O(n^2)$ time.

Ex 3.1

1. Verify $\boldsymbol{q}_1 = \frac{1}{\sqrt{2}}(1,1)^T, \boldsymbol{q}_2 = \frac{1}{\sqrt{2}}(1,-1)^T$ is an orthonormal basis using Def 3.2.
2. Express $\boldsymbol{v} = (3,-1)^T$ in this basis using Remark 3.1.
3. Prove that any set of n mutually orthogonal nonzero vectors is linearly independent.

Def 3.3 **(Orthogonal Matrix)** Square matrix $\mathbf{Q}$ where $\mathbf{Q}^T = \mathbf{Q}^{-1}$. Columns (and rows) form an orthonormal basis.

Example **(Rotation and reflection)** In $\mathbb{R}^2$,

$$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \text{ rotates, while } \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \text{ reflects.}$$

Both are orthogonal. The determinant distinguishes orientation: rotations have determinant 1, reflections have determinant -1.

Thm 3.1 **(Invariance Properties)** For any orthogonal $\mathbf{Q}$ from Def 3.3 and vectors $\boldsymbol{x}, \boldsymbol{y}$:

1. **Inner Product Preserving:** $(\mathbf{Q}\boldsymbol{x})^T(\mathbf{Q}\boldsymbol{y}) = \boldsymbol{x}^T\boldsymbol{y}$.
2. **Isometry:** $\|\mathbf{Q}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2$.
3. **Conditioning:** $\kappa_2(\mathbf{Q}) = 1$ (Optimally conditioned).

Proof *Proof.* **(1):** Applying the Reversal Law for transposes and the fact that $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$:

$$\begin{aligned}(\mathbf{Q}\boldsymbol{x})^T(\mathbf{Q}\boldsymbol{y}) &= \boldsymbol{x}^T\mathbf{Q}^T\mathbf{Q}\boldsymbol{y} \\ &= \boldsymbol{x}^T(\mathbf{Q}^T\mathbf{Q})\boldsymbol{y} \\ &= \boldsymbol{x}^T\mathbf{I}\boldsymbol{y} = \boldsymbol{x}^T\boldsymbol{y}.\end{aligned}$$

(2): Using the result from (1) with $\boldsymbol{x} = \boldsymbol{y}$:

$$\begin{aligned}\|\mathbf{Q}\boldsymbol{x}\|_2^2 &= (\mathbf{Q}\boldsymbol{x})^T(\mathbf{Q}\boldsymbol{x}) \\ &= \boldsymbol{x}^T\boldsymbol{x} = \|\boldsymbol{x}\|_2^2.\end{aligned}$$

Taking the square root gives $\|\mathbf{Q}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2$.
(3): From (2), we see that the induced norm is $\|\mathbf{Q}\|_2 = 1$. Similarly, $\|\mathbf{Q}^T\|_2 = 1$ because $\mathbf{Q}^T$ is also orthogonal. Thus:

$$\kappa_2(\mathbf{Q}) = \|\mathbf{Q}\|_2\|\mathbf{Q}^T\|_2 = 1 \cdot 1 = 1.$$

□

Ex 3.2

1. Verify rotation matrix $\mathbf{Q} = \begin{pmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{pmatrix}$ is orthogonal using Def 3.3.
2. Use Thm 3.1 to explain why rotations preserve Euclidean distance.
3. Prove that the product of two orthogonal matrices is orthogonal.
4. If $\mathbf{Q}$ is orthogonal, show $\det(\mathbf{Q}) = \pm 1$.

3.1.2 QR Decomposition

Factors $\mathbf{A}$ into an orthogonal $\mathbf{Q}$ and upper triangular $\mathbf{R}$. Default choice for least squares and eigenvalue algorithms.

QR factorization builds an orthonormal basis for the column space of $\mathbf{A}$. Once the columns have been replaced by an orthonormal basis, projections and least squares problems become much easier because coefficients are computed by inner products.

Def 3.4 **(QR Decomposition)** For $\mathbf{A} \in \mathbb{R}^{m\times n}$ $(m \geq n)$: $\mathbf{A} = \mathbf{QR}$. The columns of $\mathbf{Q}$ form an orthonormal basis for the column space introduced in Def 2.5.

- **Full QR:** $\mathbf{Q} \in \mathbb{R}^{m\times m}, \mathbf{R} \in \mathbb{R}^{m\times n}$.
- **Thin QR:** $\mathbf{Q}_1 \in \mathbb{R}^{m\times n}, \mathbf{R}_1 \in \mathbb{R}^{n\times n}$. Columns of $\mathbf{Q}_1$ span $\mathcal{R}(\mathbf{A})$.

Remark 3.2 **(Thin and full QR)** For least squares with $m \geq n$, the thin QR is usually the useful one:

$$\mathbf{A} = \mathbf{Q}_1\mathbf{R}_1, \qquad \mathbf{Q}_1 \in \mathbb{R}^{m\times n}, \quad \mathbf{R}_1 \in \mathbb{R}^{n\times n}.$$

The remaining $m - n$ columns in full QR span the orthogonal complement of $\mathcal{R}(\mathbf{A})$, but they are often unnecessary to store.

Def 3.5 **(Householder Reflector)** A reflector $\mathbf{H} = \mathbf{I} - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{\boldsymbol{v}^T\boldsymbol{v}}$ reflects $\boldsymbol{x}$ across the hyperplane $\boldsymbol{v}^\perp$.

- **Zeroing Property:** For any $\boldsymbol{x}$, we can choose $\boldsymbol{v}$ such that $\mathbf{H}\boldsymbol{x} = \pm\|\boldsymbol{x}\|\boldsymbol{e}_1$.
- **Efficiency:** Never form $\mathbf{H}$ explicitly. Apply as $\mathbf{H}\boldsymbol{x} = \boldsymbol{x} - \boldsymbol{v}(2\frac{\boldsymbol{v}^T\boldsymbol{x}}{\boldsymbol{v}^T\boldsymbol{v}})$, a rank-1 update costing $O(m)$.

Remark 3.3 **(Householder QR cost)** For $\mathbf{A} \in \mathbb{R}^{m\times n}$ via Householder: Cost $\approx 2mn^2 - \frac{2}{3}n^3$. Square case $(m = n)$: $\frac{4}{3}n^3$ (twice the cost of LU).

Remark 3.4 **(Householder stability)** A Householder reflector is orthogonal, so applying it preserves vector norms. QR by Householder is a sequence of orthogonal transformations, which avoids the error amplification caused by subtracting nearly dependent projections in classical Gram-Schmidt.

Ex 3.3

1. Find $\boldsymbol{v}$ so that the reflector in Def 3.5 satisfies $\mathbf{H}(3, 4)^T = (-5, 0)^T$.
2. Use Householder reflectors to triangularize $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 3 \\ 2 & 4 \end{pmatrix}$ and identify the resulting $\mathbf{Q}$ and $\mathbf{R}$ from Def 3.4.
3. Use Thm 3.1 to explain Remark 3.4.
4. Compare thin vs. full QR shapes in NumPy for a 5×3 matrix.

3.1.3 Gram-Schmidt Process

Gram-Schmidt constructs an orthonormal basis by taking the columns of $\mathbf{A}$ one at a time and removing the components already explained by previous basis vectors. It is conceptually simple, but finite precision can destroy orthogonality when the original columns are nearly linearly dependent.

Remark 3.5 **(Classical and modified Gram-Schmidt)**

- **Classical (CGS):** $\boldsymbol{u}_k = \boldsymbol{v}_k - \sum_{j<k} \text{proj}_{\boldsymbol{u}_j}(\boldsymbol{v}_k)$. Numerically unstable for ill-conditioned $\mathbf{A}$ (Error $\propto \kappa(\mathbf{A})^2$).
- **Modified (MGS):** Orthogonalize sequentially against partially updated vectors. Better stability (Error $\propto \kappa(\mathbf{A})$).

Remark 3.6 **(Orthogonalization hierarchy)** Householder QR is the most stable ($O(\varepsilon_{\text{mach}})$ loss of orthogonality) and is used by `np.linalg.qr`. MGS is used when $\mathbf{Q}$ columns are needed one by one (e.g., Arnoldi/GMRES).

Remark 3.7 **(Orthogonalization method comparison)**

- **Classical Gram-Schmidt:** Best for deriving the idea; weakest numerically.
- **Modified Gram-Schmidt:** Better when basis vectors must be generated sequentially.
- **Householder QR:** Default dense QR method; most stable for least squares.

Ex 3.4

1. Apply hand CGS to columns of $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 0 & 1 \end{pmatrix}$ and compare the resulting basis with Def 3.2.
2. Construct a nearly singular Vandermonde matrix. Compare loss of orthogonality ($\|\mathbf{Q}^T\mathbf{Q} - \mathbf{I}\|_F$) for CGS, MGS, and Householder.
3. Explain the experiment using Remark 3.4.

3.1.4 Orthogonal Projection

Projection is the geometric operation behind least squares. If $\boldsymbol{b}$ is not in a subspace S, the best approximation from S is the vector in S closest to $\boldsymbol{b}$. The error vector must be orthogonal to S; otherwise one could move within S and reduce the error.

Def 3.6 **(Orthogonal Projector)** For a subspace S with orthonormal basis $\mathbf{Q}$ from Def 3.2: $\mathbf{P} = \mathbf{Q}\mathbf{Q}^T$.

- **Properties:** $\mathbf{P}^2 = \mathbf{P}$ (Idempotent), $\mathbf{P}^T = \mathbf{P}$ (Symmetric).
- **Geometric View:** $\mathbf{P}\boldsymbol{x}$ is the closest vector in S to $\boldsymbol{x}$. Residual $\boldsymbol{r} = \boldsymbol{x} - \mathbf{P}\boldsymbol{x}$ lies in $S^\perp = \mathcal{N}(\mathbf{Q}^T)$.

Thm 3.2 **(Closest Point Theorem)** Let S be a subspace of $\mathbb{R}^m$ and let $\mathbf{P}$ be the orthogonal projector onto S from Def 3.6. For every $\boldsymbol{b} \in \mathbb{R}^m$, $\mathbf{P}\boldsymbol{b}$ is the unique vector in S closest to $\boldsymbol{b}$ in the Euclidean norm:

$$\|\boldsymbol{b} - \mathbf{P}\boldsymbol{b}\|_2 \leq \|\boldsymbol{b} - \boldsymbol{s}\|_2 \qquad \text{for every } \boldsymbol{s} \in S.$$

Moreover, the residual $\boldsymbol{b} - \mathbf{P}\boldsymbol{b}$ is orthogonal to S.

Ex 3.5

1. Use Def 3.6 to show that $\mathbf{P}^2 = \mathbf{P}$ and $\mathbf{P}^T = \mathbf{P}$.
2. Show that the residual $\boldsymbol{x} - \mathbf{P}\boldsymbol{x}$ is orthogonal to every vector in the target subspace.
3. Project $\boldsymbol{x} = (1, 2, 3)^T$ onto the span of $(1, 1, 1)^T$.
4. Prove Thm 3.2: write any $\boldsymbol{s} \in S$ as $\boldsymbol{s} = \mathbf{P}\boldsymbol{b} + \boldsymbol{w}$ with $\boldsymbol{w} \in S$, then use the Pythagorean theorem.
5. Relate this residual orthogonality to Thm 3.7.

3.2 Vector and Matrix Norms

Foundations for measuring approximation errors and convergence.

3.2.1 Vector Norms

Def 3.7 **(Vector Norm)** $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ satisfies:

1. **Definiteness:** $\|\boldsymbol{x}\| \geq 0$, with equality iff $\boldsymbol{x} = \mathbf{0}$.
2. **Homogeneity:** $\|c\boldsymbol{x}\| = |c|\|\boldsymbol{x}\|$.
3. **Triangle Inequality:** $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$.

Def 3.8 **(ℓ^p Norms)** $\|\boldsymbol{x}\|_p = (\sum |x_i|^p)^{1/p}$.

- ℓ_1**:** $\sum |x_i|$ (Sparsity).

- ℓ_2: $\sqrt{\boldsymbol{x}^T\boldsymbol{x}}$ (Energy).
- ℓ_∞: $\max|x_i|$ (Tolerance).

Thm 3.3 **(Fundamental Inequalities)**

- **Cauchy-Schwarz:** $|\boldsymbol{x}^T\boldsymbol{y}| \leq \|\boldsymbol{x}\|_2\|\boldsymbol{y}\|_2$.
- **Hölder:** $|\boldsymbol{x}^T\boldsymbol{y}| \leq \|\boldsymbol{x}\|_p\|\boldsymbol{y}\|_q$ where $1/p + 1/q = 1$.

Remark 3.8 **Equivalence:** On $\mathbb{R}^n$, all norms are equivalent: $c_1\|\boldsymbol{x}\|_\beta \leq \|\boldsymbol{x}\|_\alpha \leq c_2\|\boldsymbol{x}\|_\beta$. Convergence in one implies convergence in all.

Ex 3.6

1. Prove the triangle inequality for $\|\cdot\|_2$ using Cauchy-Schwarz.
2. Show that $\|\boldsymbol{x}\|_\infty = \lim_{p\to\infty}\|\boldsymbol{x}\|_p$.

3.2.2 Matrix Norms

Measure how much a matrix amplifies vectors.

Def 3.9 **(Induced (Operator) Norm)** $\|\mathbf{A}\| = \max_{\boldsymbol{x}\neq\mathbf{0}}\frac{\|\mathbf{A}\boldsymbol{x}\|}{\|\boldsymbol{x}\|}$.

- $\|\mathbf{A}\|_1$: Max absolute column sum.
- $\|\mathbf{A}\|_\infty$: Max absolute row sum.
- $\|\mathbf{A}\|_2$ **(Spectral):** $\sigma_{\max}(\mathbf{A}) = \sqrt{\lambda_{\max}(\mathbf{A}^T\mathbf{A})}$.

Def 3.10 **(Frobenius Norm)** $\|\mathbf{A}\|_F = \sqrt{\sum a_{ij}^2} = \sqrt{\mathrm{Tr}(\mathbf{A}^T\mathbf{A})}$.

- **Relation:** $\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F \leq \sqrt{\mathrm{rank}(\mathbf{A})}\|\mathbf{A}\|_2$.

Remark 3.9 **Consistency and Submultiplicativity:** Matrix norms are useful when they satisfy $\|\mathbf{A}\boldsymbol{x}\| \leq \|\mathbf{A}\|\|\boldsymbol{x}\|$. All induced norms are additionally submultiplicative: $\|\mathbf{AB}\| \leq \|\mathbf{A}\|\|\mathbf{B}\|$.

Ex 3.7

1. Compute $\|\mathbf{A}\|_1, \|\mathbf{A}\|_\infty, \|\mathbf{A}\|_2, \|\mathbf{A}\|_F$ for $\mathbf{A} = \begin{pmatrix} 1 & -2 \\ -3 & 4 \end{pmatrix}$.
2. **Low-Rank Approximation:** The Frobenius norm is central to the Eckart-Young-Mirsky theorem. How is it related to singular values?
3. Find a 2×2 example where the max norm ($\max|a_{ij}|$) is not submultiplicative.

3.3 Stability and Condition Number

Sensitivity of problems vs. robustness of algorithms.

Conditioning asks whether the mathematical problem is sensitive to small changes in the data. Stability asks whether the algorithm introduces only small changes to the problem. A good computation needs both: a well-conditioned problem and a stable algorithm.

3.3.1 Residual, Forward Error, and Backward Error

Def 3.11 **(Residual and Forward Error)** For a computed solution $\hat{\boldsymbol{x}}$ to the linear system in Def 2.1:

$$\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}} \qquad \text{and} \qquad \boldsymbol{e} = \hat{\boldsymbol{x}} - \boldsymbol{x}^*,$$

where $\boldsymbol{x}^*$ is the exact solution.

- **Residual:** How well the computed answer satisfies the equations.
- **Forward error:** How close the computed answer is to the exact answer.

Example **(Small residual, large error)** Let

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 10^{-8} \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 10^{-8} \end{pmatrix}.$$

The exact solution is $\boldsymbol{x}^* = (1, 1)^T$. The approximation $\hat{\boldsymbol{x}} = (1, 0)^T$ has residual

$$\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}} = (0, 10^{-8})^T,$$

which is tiny in absolute norm, but the solution error is $(0, -1)^T$. The small singular value makes one direction hard to determine accurately.

Ex 3.8

1. Return to Remark 2.4. Explain why the residual in Def 3.11 is computable but the forward error usually is not.
2. Verify every number in the small-residual example above.
3. Change the small diagonal entry from 10^{-8} to 10^{-12}. How do the residual and forward error change?

Thm 3.4 **(Residual-Error Bound)** Let $\mathbf{A}$ be nonsingular, $\boldsymbol{x}^*$ solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, and $\hat{\boldsymbol{x}}$ have residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$. If $\boldsymbol{b} \neq \mathbf{0}$, then

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}^*\|} \leq \kappa(\mathbf{A}) \frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|}.$$

Thus a small relative residual guarantees a small relative forward error only when the problem is well-conditioned.

Ex 3.9

1. Starting from $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$ and $\boldsymbol{b} = \mathbf{A}\boldsymbol{x}^*$, show that $\boldsymbol{r} = \mathbf{A}(\boldsymbol{x}^* - \hat{\boldsymbol{x}})$.
2. Use the previous step to prove $\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\| \leq \|\mathbf{A}^{-1}\|\|\boldsymbol{r}\|$.
3. Use $\|\boldsymbol{b}\| \leq \|\mathbf{A}\|\|\boldsymbol{x}^*\|$ to finish the proof of Thm 3.4.
4. Apply the bound to the small-residual example above. Is the bound sharp?

3.3.2 Why Factorize instead of Invert?

Remark 3.10 **(Explicit inverse rule)** Never compute $\mathbf{A}^{-1}$ explicitly.

- **Cost:** Computing $\mathbf{A}^{-1}$ takes $3\times$ longer than LU ($\frac{8}{3}n^3$ vs. $\frac{2}{3}n^3$).
- **Stability:** Round-off in $\mathbf{A}^{-1}$ propagates into the entire product $\mathbf{A}^{-1}\boldsymbol{b}$. Stable factorizations confine errors.
- **Efficiency:** LU allows solving for new $\boldsymbol{b}$'s in $O(n^2)$ without storing a dense inverse.

Ex 3.10

1. Use the LU factorization costs from the LU chapter to compare total flops for solving 10 systems via $\mathbf{A}^{-1}$ vs. LU when $n = 500$.
2. Construct $\mathbf{A} = \mathrm{diag}(1, 1, 10^{-12})$. Compare residuals of `inv(A) @ b` and `solve(A, b)` using Def 3.11.
3. Explain the result using the distinction between residual and forward error from Ex 3.8.

3.3.3 Numerical Stability

A property of the **algorithm**.

Def 3.12 **(Backward Stability)** An algorithm is backward stable if its computed $\hat{\boldsymbol{x}}$ is the **exact solution to a nearby problem**: $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \delta\boldsymbol{b}$, with $\|\delta\mathbf{A}\|/\|\mathbf{A}\| = O(\varepsilon_{\text{mach}})$.

Thm 3.5 **(Normwise Backward Error for Linear Systems)** For a computed vector $\hat{\boldsymbol{x}}$ with residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$, the normwise relative backward error for the linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is

$$\eta(\hat{\boldsymbol{x}}) = \frac{\|\boldsymbol{r}\|}{\|\mathbf{A}\|\|\hat{\boldsymbol{x}}\| + \|\boldsymbol{b}\|}.$$

This is the smallest relative perturbation size, measured normwise in both $\mathbf{A}$ and $\boldsymbol{b}$, that makes $\hat{\boldsymbol{x}}$ an exact solution of a nearby system.

Ex 3.11

1. Suppose $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \delta\boldsymbol{b}$. Show that $\boldsymbol{r} = \delta\boldsymbol{b} - \delta\mathbf{A}\hat{\boldsymbol{x}}$.
2. Use the triangle inequality to prove the lower bound
$$\|\boldsymbol{r}\| \leq \|\delta\mathbf{A}\|\|\hat{\boldsymbol{x}}\| + \|\delta\boldsymbol{b}\|.$$
3. Divide by $\|\mathbf{A}\|\|\hat{\boldsymbol{x}}\| + \|\boldsymbol{b}\|$ to obtain the denominator in Thm 3.5.
4. Construct a rank-one perturbation in the direction of $\boldsymbol{r}$ to see why the bound can be attained.
5. Compare this computable backward error with the residual-error bound in Thm 3.4.

Remark 3.11 **GEPP Stability:** Gaussian Elimination with Partial Pivoting is backward stable for virtually all practical matrices. The growth factor ρ stays small, preventing exponential error amplification.

Remark 3.12 **(Stable algorithms)** Householder QR and Cholesky for SPD systems are backward stable under their usual assumptions. LU with partial pivoting is reliable in practice, though its worst-case growth factor can be large.

Ex 3.12

1. Use Def 3.12 to explain why backward stability is a natural standard when input data are already uncertain.
2. Explain why a small residual does not always guarantee a small forward error. Use Thm 3.4.
3. Compute $\eta(\hat{\boldsymbol{x}})$ from Thm 3.5 for the small-residual example above.
4. Histogram growth factors for 100 random matrices via `scipy.linalg.lu`.

3.3.4 Condition Number

A property of the **problem**.

Def 3.13 **(Condition Number)** $\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\|$.

- **Interpretation:** Worst-case error amplification. A perturbation of size ε in $\boldsymbol{b}$ can cause error up to $\kappa(\mathbf{A})\varepsilon$ in $\boldsymbol{x}$.

- **Rule of Thumb:** You expect at most $16 - \log_{10}(\kappa(\mathbf{A}))$ correct digits in double precision.

Proof

Proof. Let $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ be the original system and $\mathbf{A}(\boldsymbol{x} + \delta\boldsymbol{x}) = \boldsymbol{b} + \delta\boldsymbol{b}$ the perturbed system. Subtracting gives $\mathbf{A}\delta\boldsymbol{x} = \delta\boldsymbol{b}$, so $\delta\boldsymbol{x} = \mathbf{A}^{-1}\delta\boldsymbol{b}$. Taking norms:

$$\|\delta\boldsymbol{x}\| \leq \|\mathbf{A}^{-1}\|\|\delta\boldsymbol{b}\|.$$

Also, from $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, we have $\|\boldsymbol{b}\| \leq \|\mathbf{A}\|\|\boldsymbol{x}\|$, which implies $1/\|\boldsymbol{x}\| \leq \|\mathbf{A}\|/\|\boldsymbol{b}\|$. Combining these inequalities for the relative error:

$$\begin{aligned}\frac{\|\delta\boldsymbol{x}\|}{\|\boldsymbol{x}\|} &\leq \|\mathbf{A}^{-1}\|\|\delta\boldsymbol{b}\|\frac{\|\mathbf{A}\|}{\|\boldsymbol{b}\|} \\ &= (\|\mathbf{A}\|\|\mathbf{A}^{-1}\|)\frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|} \\ &= \kappa(\mathbf{A})\frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|}.\end{aligned}$$

Thus, the condition number $\kappa(\mathbf{A})$ acts as the amplification factor for relative perturbations in the right-hand side. □

Ex 3.13

1. Reproduce the perturbation argument above without looking.
2. Compute $\kappa_2(H_n)$ for Hilbert matrices $n = 3, \ldots, 10$. How fast does it grow?
3. Use Def 3.12, Def 3.13, and Thm 3.4 to explain why a stable algorithm on an ill-conditioned problem may still lose many digits.
4. **The Neumann Series:** Use it to prove the perturbation bound: $\frac{\|\delta\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leq \frac{\kappa(\mathbf{A})}{1-\ldots}$.

3.3.5 Solver Selection Hierarchy

Thm 3.6 **(Solver Selection by Structure)** Choose a solver from the most specific trustworthy structure in the problem:

1. If $\mathbf{A}$ is SPD and dense, use Cholesky; if it is SPD and large sparse, use CG with a preconditioner.
2. If $\mathbf{A}$ is general, square, dense, and nonsingular, use LU with partial pivoting.
3. If the problem is full-rank least squares, use Householder QR.
4. If the problem is rank-deficient, nearly rank-deficient, or requires numerical rank information, use the SVD.

5. If $\mathbf{A}$ is large, sparse, nonsymmetric, and only matrix-vector products are practical, use GMRES or another nonsymmetric Krylov method.

The more structure a solver exploits, the cheaper it can be; the less trustworthy the structure, the more robust the solver must be.

Ex 3.14

1. For each case in Thm 3.6, identify the mathematical structure being used: symmetry, positive definiteness, sparsity, full column rank, or numerical rank.
2. Explain why Cholesky is inappropriate if the SPD assumption fails.
3. Explain why QR is preferred to normal equations for full-rank least squares, using Ex 3.16.
4. Explain why the SVD is the natural fallback when rank is uncertain, using Thm 4.6.
5. Explain why CG and GMRES fit the Krylov approximation principle from Thm 5.1.

Def 3.14 **(Iterative Refinement)** The accuracy of a computed solution $\hat{\boldsymbol{x}}$ can be improved by iterative refinement:

1. Compute the residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$.
2. Solve $\mathbf{A}\boldsymbol{\delta} = \boldsymbol{r}$.
3. Update the estimate: $\hat{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}} + \boldsymbol{\delta}$.

Each step recovers digits lost due to ill-conditioning, up to the limits of machine precision.

Remark 3.13 **(Limitations)** Iterative refinement helps when the residual can be computed accurately and the correction equation can be solved reliably. It cannot overcome a problem whose condition number is so large that the desired digits are not present in the data.

Ex 3.15

1. Solve $H_{10}\boldsymbol{x} = H_{10}\mathbf{1}$. Apply one step of iterative refinement from Def 3.14. How many digits are recovered?
2. Compare forward errors of `solve` (LU) vs `lstsq` (QR) on a Hilbert system. Interpret the result using Ex 3.13.
3. Choose a solver for each case in Thm 3.6: SPD, dense general square, full-rank least squares, rank-deficient least squares, large sparse SPD, and large sparse nonsymmetric. Justify each choice by citing one earlier result or exercise.

3.4 Overdetermined Systems and Least Squares

Approximating the solution to an overdetermined system where $m > n$ and no exact solution exists.

When $\boldsymbol{b}$ is not in the column space of $\mathbf{A}$, the equation $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has no exact solution. Least squares replaces the impossible goal "make the residual zero" with the geometric goal "make the residual as short as possible."

3.4.1 Least Squares Problem

Def 3.15 **(Overdetermined System)** $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with $\mathbf{A} \in \mathbb{R}^{m\times n}, m > n$. Usually $\boldsymbol{b} \notin \mathcal{R}(\mathbf{A})$.

Def 3.16 **(Least Squares Problem)** Find $\hat{\boldsymbol{x}}$ that minimizes the Euclidean residual:

$$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}\in\mathbb{R}^n} \|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2.$$

Geometry: $\mathbf{A}\hat{\boldsymbol{x}}$ is the orthogonal projection of $\boldsymbol{b}$ onto $\mathcal{R}(\mathbf{A})$.

Thm 3.7 **(Projection Characterization)** For the least squares problem in Def 3.16, Thm 3.2 applied to $S = \mathcal{R}(\mathbf{A})$ implies that the solution $\hat{\boldsymbol{x}}$ is characterized by

$$\mathbf{A}\hat{\boldsymbol{x}} = \text{proj}_{\mathcal{R}(\mathbf{A})}(\boldsymbol{b}), \qquad \boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}} \perp \mathcal{R}(\mathbf{A}).$$

Equivalently, $\mathbf{A}^T\boldsymbol{r} = \mathbf{0}$.

Example **(Fitting a line)** For data (t_i, y_i), fitting $y \approx c_0 + c_1 t$ gives

$$\mathbf{A} = \begin{pmatrix} 1 & t_1 \\ 1 & t_2 \\ \vdots & \vdots \\ 1 & t_m \end{pmatrix}, \qquad \boldsymbol{x} = \begin{pmatrix} c_0 \\ c_1 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}.$$

The residuals are the vertical errors between the data and the fitted line.

3.4.2 Normal Equations

Thm 3.8 **(Normal Equations)** Using the orthogonality condition in Thm 3.7, the minimizer satisfies $\mathbf{A}^T\mathbf{A}\hat{\boldsymbol{x}} = \mathbf{A}^T\boldsymbol{b}$.

- **Condition:** $\mathbf{A}^T\boldsymbol{r} = \mathbf{0}$ (Residual must be normal to the column space).
- **Unique Solution:** $\hat{\boldsymbol{x}} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\boldsymbol{b}$ if $\mathbf{A}$ has full column rank.

Ex 3.16

1. Starting from Thm 3.7, derive the normal equations in Thm 3.8.
2. Show that $\mathbf{A}^T\boldsymbol{r} = \mathbf{0}$ is equivalent to $\boldsymbol{r} \in \mathcal{N}(\mathbf{A}^T)$.
3. Use Thm 2.4 to explain why $\boldsymbol{r} \in \mathcal{N}(\mathbf{A}^T)$ means $\mathbf{A}\hat{\boldsymbol{x}}$ is the projection of $\boldsymbol{b}$ onto $\mathcal{R}(\mathbf{A})$.
4. Fit a line $y = mx + b$ to points $(1, 2), (2, 3), (3, 5), (4, 4)$ via normal equations.
5. Prove $\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2$ using singular values. If $\kappa_2(\mathbf{A}) = 10^6$, estimate the number of digits that may be lost in double precision.
6. Explain why the normal equations are useful for derivation but should not be the default numerical algorithm. Your answer should mention $\mathbf{A}^T\boldsymbol{r} = \mathbf{0}$, conditioning, QR, and SVD.

3.4.3 QR-Based Least Squares

The numerically superior approach.

Thm 3.9 **(Solving via QR)** If $\mathbf{A} = \mathbf{QR}$ (thin QR), the least squares solution satisfies:

$$\mathbf{R}\hat{\boldsymbol{x}} = \mathbf{Q}^T\boldsymbol{b}.$$

This avoids forming $\mathbf{A}^T\mathbf{A}$, the instability diagnosed in Ex 3.16.

Proof *Proof.* The goal is to minimize $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2$. Let $\mathbf{A} = \mathbf{QR}$ be the thin QR factorization. Using the orthogonal projector $\mathbf{P} = \mathbf{QQ}^T$ and its complement $\mathbf{I} - \mathbf{P}$:

$$\begin{aligned}\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 &= \|\mathbf{QR}\boldsymbol{x} - \boldsymbol{b}\|_2^2 \\ &= \|\mathbf{QR}\boldsymbol{x} - (\mathbf{QQ}^T\boldsymbol{b} + (\mathbf{I} - \mathbf{QQ}^T)\boldsymbol{b})\|_2^2 \\ &= \|(\mathbf{QR}\boldsymbol{x} - \mathbf{QQ}^T\boldsymbol{b}) - (\mathbf{I} - \mathbf{QQ}^T)\boldsymbol{b}\|_2^2.\end{aligned}$$

Since $\mathbf{Q}(\mathbf{R}\boldsymbol{x} - \mathbf{Q}^T\boldsymbol{b}) \in \mathcal{R}(\mathbf{Q})$ and $(\mathbf{I} - \mathbf{QQ}^T)\boldsymbol{b} \in \mathcal{R}(\mathbf{Q})^\perp$, the Pythagorean theorem gives:

$$\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 = \|\mathbf{Q}(\mathbf{R}\boldsymbol{x} - \mathbf{Q}^T\boldsymbol{b})\|_2^2 + \|(\mathbf{I} - \mathbf{QQ}^T)\boldsymbol{b}\|_2^2.$$

The first term is minimized (set to zero) when $\mathbf{R}\boldsymbol{x} = \mathbf{Q}^T\boldsymbol{b}$, and the second term is independent of $\boldsymbol{x}$. Thus $\hat{\boldsymbol{x}} = \mathbf{R}^{-1}\mathbf{Q}^T\boldsymbol{b}$ is the unique minimizer. □

Remark 3.14 **(Least squares solver hierarchy)**

- **Normal equations:** Fast and simple, but least stable; condition number is squared.
- **QR:** Standard method for full-rank least squares; stable and efficient.
- **SVD:** Most robust; handles rank deficiency and reveals numerical rank.

3.4.4 Rank Deficiency and Regularization

If $\mathbf{A}$ does not have full column rank, there may be infinitely many least squares minimizers. The SVD selects the minimizer with smallest Euclidean norm. Regularization modifies the problem to penalize large coefficients and improve conditioning.

Thm 3.10 **(Minimum-Norm Least Squares)** For any $\mathbf{A}$, the vector

$$\hat{\boldsymbol{x}} = \mathbf{A}^{\dagger}\boldsymbol{b}$$

is the least squares minimizer with smallest Euclidean norm.

Ex 3.17

1. Write $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$.
2. Transform the least squares problem into SVD coordinates.
3. Identify which coordinates are determined by nonzero singular values.
4. Show that setting nullspace coordinates to zero gives the minimum-norm solution in Thm 3.10.

Def 3.17 **(Ridge-Regularized Least Squares)** For $\lambda > 0$, ridge regression solves

$$\hat{\boldsymbol{x}}_\lambda = \arg\min_{\boldsymbol{x}} \left(\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \lambda \|\boldsymbol{x}\|_2^2 \right).$$

The normal equations become

$$(\mathbf{A}^T\mathbf{A} + \lambda \mathbf{I})\hat{\boldsymbol{x}}_\lambda = \mathbf{A}^T\boldsymbol{b}.$$

Def 3.18 **(Weighted Least Squares)** Given positive weights w_i, weighted least squares solves

$$\min_{\boldsymbol{x}} \|\mathbf{W}^{1/2}(\mathbf{A}\boldsymbol{x} - \boldsymbol{b})\|_2^2, \qquad \mathbf{W} = \operatorname{diag}(w_1, \ldots, w_m).$$

Large w_i forces the fit to match observation i more strongly.

Thm 3.11 **(Weighted Normal Equations)** The weighted least squares problem in Def 3.18 satisfies

$$\mathbf{A}^T\mathbf{W}\mathbf{A}\hat{\boldsymbol{x}} = \mathbf{A}^T\mathbf{W}\boldsymbol{b}.$$

Proof *Proof.* Apply the ordinary normal equations from Thm 3.8 to the transformed system

$$\mathbf{W}^{1/2}\mathbf{A}\boldsymbol{x} \approx \mathbf{W}^{1/2}\boldsymbol{b}.$$

This gives $(\mathbf{W}^{1/2}\mathbf{A})^T(\mathbf{W}^{1/2}\mathbf{A})\hat{\boldsymbol{x}} = (\mathbf{W}^{1/2}\mathbf{A})^T\mathbf{W}^{1/2}\boldsymbol{b}$, which simplifies to the stated equation. □

Ex 3.18

1. Return to Ex 3.16. Solve the same line-fitting problem using QR. Compare coefficients, residual norms, and the conditioning of the two linear systems solved.
2. **Regularization:** If $\mathbf{A}$ is rank-deficient, $\|\hat{\boldsymbol{x}}\|$ can explode. Add a penalty term $\lambda\|\boldsymbol{x}\|_2^2$ (Ridge Regression). How do the normal equations change?
3. Derive Thm 3.11 by differentiating the weighted objective directly.

3.4.5 Polynomial Regression

Def 3.19 **(Vandermonde Matrix)** For nodes $t_1, \ldots, t_m$, the degree-n design matrix is:

$$V_{ij} = t_i^{j-1}.$$

Polynomial fitting $p(t) = \sum c_j t^j$ is solved as $\mathbf{V}\boldsymbol{c} \approx \mathbf{f}$.

Ex 3.19

1. Fit a degree-2 polynomial to 6 data points. Verify the residual and R^2 score.
2. **Numerical Stability:** Construct a Vandermonde matrix with $n = 10$. Compare the errors of `lstsq(A, b)` vs `solve(A.T @ A, A.T @ b)`. Relate the outcome to Ex 3.16.

3.5 Chebyshev Polynomials

Chebyshev polynomials are the polynomial analogue of Fourier modes on $[-1, 1]$. They oscillate evenly, stay bounded, and give node sets that control interpolation error. The central idea is minimax: among polynomials of a fixed degree, Chebyshev polynomials spread error as evenly as possible instead of allowing large boundary spikes.

3.5.1 Failure of the Monomial Basis

Polynomial approximation has two separate failure modes. The first is numerical: the monomial basis $1, x, x^2, \ldots$ leads to ill-conditioned Vandermonde matrices. The second is analytical: even exact interpolation at equally spaced nodes can oscillate badly near the interval endpoints.

Def 3.20 **(Interpolation Error Polynomial)** Given distinct interpolation nodes $x_1, \dots, x_n$, define

$$\omega_n(x) = \prod_{j=1}^{n} (x - x_j).$$

This polynomial is zero at every node. Its size between the nodes controls how much interpolation can amplify the derivative of the target function.

Thm 3.12 **(Interpolation Error Formula)** Let p_{n-1} interpolate a function f at n distinct nodes $x_1, \dots, x_n$. If f has n continuous derivatives on $[-1, 1]$, then for each $x \in [-1, 1]$ there exists a point $\xi_x \in [-1, 1]$ such that

$$f(x) - p_{n-1}(x) = \frac{f^{(n)}(\xi_x)}{n!} \omega_n(x).$$

Therefore,

$$\|f - p_{n-1}\|_\infty \leq \frac{\|f^{(n)}\|_\infty}{n!} \|\omega_n\|_\infty.$$

Ex 3.20

1. For three nodes x_1, x_2, x_3, write out $\omega_3(x)$ and verify that it vanishes at each node.
2. Use Thm 3.12 to explain why node choice matters even when $f^{(n)}$ is fixed.
3. Build the degree-14 Vandermonde matrix on 15 equally spaced nodes in $[-1, 1]$. Compute $\kappa_2(\mathbf{V})$.
4. Plot Runge's function $f(x) = 1/(1 + 25x^2)$ and its degree-10 interpolant at equally spaced nodes. Compare the largest error in the middle of the interval with the largest error near the endpoints.

3.5.2 Chebyshev Polynomials (T_k)

Def 3.21 **(Chebyshev Polynomial)** For $x \in [-1, 1]$, the degree-k Chebyshev polynomial is

$$T_k(x) = \cos(k \arccos x).$$

Equivalently, if $x = \cos\theta$, then

$$T_k(\cos\theta) = \cos(k\theta).$$

Thm 3.13 **(Basic Chebyshev Identities)** The polynomials from Def 3.21 satisfy

$$T_0(x) = 1, \qquad T_1(x) = x, \qquad T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x).$$

They are bounded by 1 on $[-1, 1]$:

$$|T_k(x)| \leq 1.$$

They are orthogonal with respect to the weight $(1 - x^2)^{-1/2}$:

$$\int_{-1}^{1} T_j(x)T_k(x)\frac{dx}{\sqrt{1-x^2}} = 0, \qquad j \neq k.$$

Ex 3.21

1. Use $\cos((k+1)\theta) + \cos((k-1)\theta) = 2\cos\theta\cos(k\theta)$ to prove the three-term recurrence in Thm 3.13.
2. Compute T_2, T_3, and T_4 from the recurrence.
3. Prove $|T_k(x)| \leq 1$ on $[-1, 1]$ directly from Def 3.21.
4. Substitute $x = \cos\theta$ into the weighted integral and reduce the orthogonality statement to Fourier cosine orthogonality on $[0, \pi]$.
5. Plot $T_0, \ldots, T_5$. Mark the points where each polynomial reaches ± 1.

3.5.3 Zeros and Nodes

Thm 3.14 **(Zeros and Extrema)** The zeros of T_n are

$$x_j = \cos\left(\frac{(2j-1)\pi}{2n}\right), \qquad j = 1, \ldots, n.$$

The extrema of T_n occur at

$$y_j = \cos\left(\frac{j\pi}{n}\right), \qquad j = 0, \ldots, n,$$

and satisfy $T_n(y_j) = (-1)^j$.

Def 3.22 **(Chebyshev Nodes)** The Chebyshev interpolation nodes are the zeros of T_n:

$$x_j = \cos\left(\frac{(2j-1)\pi}{2n}\right), \qquad j = 1, \ldots, n.$$

These nodes cluster near ± 1. The clustering is not cosmetic; it reduces the size of the interpolation error polynomial from Def 3.20.

Thm 3.15 **(Chebyshev Error Polynomial)** If $x_1, \ldots, x_n$ are the Chebyshev nodes from Def 3.22, then

$$\omega_n(x) = \prod_{j=1}^{n}(x - x_j) = 2^{1-n}T_n(x).$$

Consequently,

$$\|\omega_n\|_\infty = 2^{1-n}.$$

Ex 3.22

1. Starting from $T_n(\cos\theta) = \cos(n\theta)$, solve $\cos(n\theta) = 0$ and prove the zero formula in Thm 3.14.
2. Solve $\cos(n\theta) = \pm 1$ and prove the extrema formula in Thm 3.14.
3. Explain why the Chebyshev nodes cluster near ± 1 by comparing the spacing of equally spaced θ values under $x = \cos\theta$.
4. Show that T_n has leading coefficient 2^{n-1} for $n \geq 1$.
5. Use the previous step and the fact that T_n has zeros $x_1, \ldots, x_n$ to prove Thm 3.15.

3.5.4 The Minimax Property

Thm 3.16 **(Chebyshev Minimax Polynomial)** Among all monic polynomials q of degree n on $[-1, 1]$,

$$2^{1-n}T_n(x)$$

has the smallest possible infinity norm:

$$\|2^{1-n}T_n\|_\infty = 2^{1-n} \leq \|q\|_\infty.$$

Ex 3.23

1. Let $m(x) = 2^{1-n}T_n(x)$. Use Thm 3.13 to show that m is monic and $\|m\|_\infty = 2^{1-n}$.
2. Suppose another monic polynomial q satisfies $\|q\|_\infty < 2^{1-n}$. Define $r = q - m$. What is the degree of r?
3. Evaluate the signs of m at the $n + 1$ extrema from Thm 3.14.

4. Use the assumption $\|q\|_\infty < 2^{1-n}$ to show that r changes sign between each pair of adjacent extrema.

5. Conclude that r has at least n roots, contradicting its degree. This proves Thm 3.16.

6. Combine Thm 3.12 and Thm 3.15 to explain why Chebyshev nodes are the natural interpolation nodes.

3.5.5 Chebyshev Series

Def 3.23 **(Chebyshev Series)** A Chebyshev expansion writes a function on $[-1, 1]$ as

$$f(x) \sim \sum_{k=0}^{\infty} c_k T_k(x).$$

Using $x = \cos\theta$, this is exactly a cosine series for the even function $f(\cos\theta)$.

Thm 3.17 **(Coefficient Formula)** For $k \geq 1$,

$$c_k = \frac{2}{\pi} \int_0^\pi f(\cos\theta) \cos(k\theta)\, d\theta,$$

and

$$c_0 = \frac{1}{\pi} \int_0^\pi f(\cos\theta)\, d\theta.$$

Thm 3.18 **(Spectral Accuracy)** If f is analytic on and near $[-1, 1]$, then its Chebyshev coefficients decay geometrically:

$$|c_k| \leq C\rho^{-k} \qquad \text{for some } \rho > 1.$$

The truncated series

$$p_n(x) = \sum_{k=0}^{n} c_k T_k(x)$$

therefore satisfies

$$\|f - p_n\|_\infty = O(\rho^{-n}).$$

This is called spectral accuracy.

Remark 3.15 **(Fourier connection)** Chebyshev methods are Fourier cosine methods after the change of variables $x = \cos\theta$. This is why fast cosine transforms can compute Chebyshev coefficients efficiently.

Remark 3.16 **(Interpolation vs. expansion)** Chebyshev interpolation chooses values at Chebyshev nodes. Chebyshev series choose coefficients in the Chebyshev basis. Both work well for the same reason: the basis oscillates evenly and avoids the endpoint instability of monomials.

Ex 3.24

1. Use Def 3.23 to rewrite a Chebyshev expansion as a cosine series in θ.
2. Derive the coefficient formula in Thm 3.17 from Fourier cosine orthogonality.
3. Compute degree-n Chebyshev fits to e^x using `np.polynomial.chebyshev.chebfit`. Plot $\|f - p_n\|_\infty$ versus n on a semilog scale.
4. Repeat the previous experiment for $f(x) = |x|$. Compare the coefficient decay with the analytic case in Thm 3.18.
5. Use `chebder` to differentiate a Chebyshev approximation to $f(x) = \sin(\pi x)$. Compare the derivative error with a finite-difference approximation using the same number of sample points.

4 Eigenvalues and the SVD

4.1 Eigenvalues and Eigenvectors

Reveals the scaling directions of a linear transformation. Essential for stability analysis, PCA, and spectral methods.

Eigenvectors are directions preserved by a linear map. Along an eigenvector, the matrix acts like multiplication by a scalar. This reduces a matrix transformation to independent one-dimensional actions, when enough eigenvectors exist.

4.1.1 Definitions and Basic Properties

Def 4.1 **(Eigenvalues and Eigenvectors)** For $\mathbf{A} \in \mathbb{R}^{n\times n}$, a scalar $\lambda \in \mathbb{C}$ is an **eigenvalue** if $\exists$ nonzero $\boldsymbol{v} \in \mathbb{C}^n$ such that:

$$\mathbf{A}\boldsymbol{v} = \lambda \boldsymbol{v}.$$

- **Spectrum $\sigma(\mathbf{A})$:** The set of all eigenvalues.
- **Invariance:** Multiplying $\boldsymbol{v}$ by $\mathbf{A}$ only scales its length; its direction remains fixed.

Def 4.2 **(Characteristic Polynomial)** $p_{\mathbf{A}}(\lambda) = \det(\mathbf{A} - \lambda \mathbf{I})$. Eigenvalues are the roots of $p_{\mathbf{A}}(\lambda) = 0$.

Remark 4.1 **(Characteristic polynomial as theory)** The characteristic polynomial is useful for proofs and small hand examples. Numerical algorithms do not compute eigenvalues by forming $p_{\mathbf{A}}$ and finding its roots; polynomial coefficients are extremely sensitive to perturbations.

Ex 4.1

1. Verify $\boldsymbol{v} = (1, 1)^T$ is an eigenvector of $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ and find λ.
2. Why is the zero vector excluded from the definition of an eigenvector?

Thm 4.1 **(Similar Matrices)** If $\mathbf{B} = \mathbf{V}^{-1}\mathbf{A}\mathbf{V}$ (Similarity Transform), then $\sigma(\mathbf{B}) = \sigma(\mathbf{A})$. Eigenvalues are invariant under change of basis.

Def 4.3 **(Diagonalization)** A matrix $\mathbf{A}$ is diagonalizable if there exists an invertible matrix $\mathbf{V}$ such that

$$\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^{-1},$$

where $\mathbf{D}$ is diagonal. The columns of $\mathbf{V}$ are eigenvectors and the diagonal entries of $\mathbf{D}$ are eigenvalues.

Remark 4.2 **(Multiplicity and defectiveness)** Repeated eigenvalues do not guarantee enough eigenvectors. A matrix can have an eigenvalue of algebraic multiplicity 2 but only one independent eigenvector; such a matrix is **defective** and cannot be diagonalized.

Example **(Defective matrix)**

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$$

has characteristic polynomial $(1 - \lambda)^2$. Solving $(\mathbf{A} - \mathbf{I})\boldsymbol{v} = \mathbf{0}$ gives only one independent eigenvector, so $\mathbf{A}$ is not diagonalizable.

Thm 4.2 **(Key Properties)**

1. **Trace:** $\sum \lambda_i = \operatorname{tr}(\mathbf{A})$.
2. **Determinant:** $\prod \lambda_i = \det(\mathbf{A})$.
3. **Singularity:** $0 \in \sigma(\mathbf{A}) \iff \mathbf{A}$ is singular.
4. **Inversion:** If $\mathbf{A}$ is invertible, eigenvalues of $\mathbf{A}^{-1}$ are $1/\lambda_i$.

4.1.2 Symmetric and Hermitian Matrices

Symmetric and Hermitian matrices possess special spectral properties that simplify analysis and computation.

The symmetric case is the cleanest eigenvalue theory. Orthogonal eigenvectors mean that diagonalization is also a numerically safe change of basis: $\mathbf{Q}^{-1} = \mathbf{Q}^T$, so the transformation does not amplify Euclidean errors.

Thm 4.3 **(Spectral Theorem (Real Symmetric))** If $\mathbf{A} = \mathbf{A}^T$:

1. All eigenvalues λ_i are **real**.
2. Eigenvectors for distinct λ are **orthogonal**.
3. $\mathbf{A}$ is **orthogonally diagonalizable**: $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^T$.

Def 4.4 **(Hermitian Matrix)** $\mathbf{A} = \mathbf{A}^*$ (conjugate transpose). The complex analog of symmetric matrices.

- **Spectral Theorem (Hermitian):** Unitarily diagonalizable ($\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^*$) with real eigenvalues.

Ex 4.2

1. Prove eigenvalues of $\mathbf{A}^2$ are λ_i^2.
2. Diagonalize $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 1 & 4 \end{pmatrix}$ by finding an orthonormal eigenbasis.

4.1.3 Iterative Algorithms

In practice, we compute eigenvalues via iteration, not by finding roots of the characteristic polynomial.

Most large eigenvalue computations do not compute the full spectrum. They target a few eigenvalues: the largest, smallest, or those closest to a shift. Matrix-vector products and linear solves are the basic primitives.

Def 4.5 **(Power Iteration)** Repeatedly apply $\mathbf{A}$ to a unit vector: $\boldsymbol{v}^{(k)} = \frac{\mathbf{A}\boldsymbol{v}^{(k-1)}}{\|\mathbf{A}\boldsymbol{v}^{(k-1)}\|}$.

- **Convergence:** $\boldsymbol{v}^{(k)} \to$ dominant eigenvector (for $|\lambda_1| > |\lambda_2|$).
- **Rate:** $O(|\lambda_2/\lambda_1|^k)$.

Remark 4.3 **(Power iteration failure modes)** Power iteration can fail or converge slowly if $|\lambda_1| \approx |\lambda_2|$, if the starting vector has tiny component in the dominant eigenvector direction, or if the dominant eigenvalue is not unique in magnitude.

Proof *Proof.* Assume $\mathbf{A}$ is diagonalizable with eigenvectors $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$ and eigenvalues $|\lambda_1| > |\lambda_2| \geq \ldots$. Let $\boldsymbol{x}^{(0)} = \sum_{i=1}^{n} c_i \boldsymbol{u}_i$ with $c_1 \neq 0$. Applying $\mathbf{A}$ k times:

$$\begin{aligned} \mathbf{A}^k \boldsymbol{x}^{(0)} &= \sum_{i=1}^{n} c_i \lambda_i^k \boldsymbol{u}_i \\ &= c_1 \lambda_1^k \left(\boldsymbol{u}_1 + \sum_{i=2}^{n} \frac{c_i}{c_1} \left(\frac{\lambda_i}{\lambda_1} \right)^k \boldsymbol{u}_i \right). \end{aligned}$$

Since $|\lambda_i/\lambda_1| < 1$ for all $i > 1$, the summation term decays to zero as $k \to \infty$. Thus $\mathbf{A}^k \boldsymbol{x}^{(0)}$ aligns with $\boldsymbol{u}_1$. The normalization at each step prevents overflow while preserving this alignment. □

Def 4.6 **(Inverse Iteration)** Apply power iteration to $(\mathbf{A} - \sigma \mathbf{I})^{-1}$.

- **Use:** Finds the eigenvalue closest to the shift σ.
- **Efficiency:** Factorize $(\mathbf{A} - \sigma \mathbf{I})$ once, then solve at each step ($O(n^2)$).

Remark 4.4 **(Shifted inverse iteration)** If λ_i is an eigenvalue of $\mathbf{A}$, then $(\lambda_i - \sigma)^{-1}$ is an eigenvalue of $(\mathbf{A} - \sigma \mathbf{I})^{-1}$. The eigenvalue closest to σ becomes largest in magnitude after inversion.

Def 4.7 **(Rayleigh Quotient Iteration (RQI))** Update the shift σ_k to the current Rayleigh quotient $(\boldsymbol{v}^{(k)})^T \mathbf{A} \boldsymbol{v}^{(k)}$ at each step.

- **Convergence: Cubic** for symmetric matrices (digits triple each step).

Def 4.8 **(Eigenvalue Method Selection)**

Goal	Typical method
Largest eigenvalue of large matrix	Power iteration
Eigenvalue near a shift	Inverse iteration
Fast local refinement, symmetric case	Rayleigh quotient iteration
Full dense spectrum	QR algorithm
Few extreme eigenvalues, symmetric sparse matrix	Lanczos

Ex 4.3

1. Implement power iteration to find $\lambda_{\max}$ of a tridiagonal matrix.
2. Use RQI on a random 4×4 symmetric matrix. Observe the rate of convergence.

4.1.4 Schur Decomposition and the QR Algorithm

The Schur decomposition provides the theoretical basis for the QR algorithm, the standard iterative method for computing the eigenvalues and eigenvectors of dense, general matrices.

Diagonalization may fail for defective matrices, but Schur decomposition always exists over $\mathbb{C}$. This is why practical dense eigensolvers target Schur form rather than diagonal form.

Thm 4.4 **(Schur Decomposition)** Every square $\mathbf{A}$ satisfies $\mathbf{A} = \mathbf{QTQ}^*$, where $\mathbf{Q}$ is unitary and $\mathbf{T}$ is upper triangular.

- **Schur Vectors:** Columns of $\mathbf{Q}$ provide an orthonormal basis for nested invariant subspaces.

Def 4.9 **(QR Iteration)** Iteratively compute QR factorizations and swap the factors. The goal is to transform $\mathbf{A}$ into an upper triangular (Schur) form, where the eigenvalues appear on the diagonal.

1. $\mathbf{A}_k = \mathbf{Q}_k\mathbf{R}_k$
2. $\mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k = \mathbf{Q}_k^T\mathbf{A}_k\mathbf{Q}_k$

Convergence: $\mathbf{A}_k$ converges to the Schur form $\mathbf{T}$.

Remark 4.5 **(QR iteration performance)** Raw QR iteration requires $O(n^3)$ operations per step. Practical implementations, such as those in standard numerical libraries, typically use:

1. **Hessenberg Reduction:** Pre-process $\mathbf{A}$ to upper Hessenberg form in $O(n^3)$ operations.
2. **Triangularization:** Subsequent QR steps on a Hessenberg matrix cost only $O(n^2)$.
3. **Shifts:** Techniques to accelerate convergence to quadratic or cubic rates.

Ex 4.4

1. For $\mathbf{A} = \begin{pmatrix} 3 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{pmatrix}$, identify the Schur vectors and eigenvalues by inspection.
2. Implement 50 steps of basic QR iteration on a random 3×3 symmetric matrix. Check the off-diagonals.

4.2 Singular Value Decomposition

The Singular Value Decomposition (SVD) is a general matrix factorization that decomposes any $m \times n$ matrix into a product of orthogonal transformations and a diagonal scaling matrix.

The SVD is the most complete description of what a matrix does geometrically. It finds orthonormal input directions, tells how much each direction is stretched, and gives the corresponding orthonormal output directions.

4.2.1 The SVD Theorem

Thm 4.5 **(Singular Value Decomposition (SVD))** For $\mathbf{A} \in \mathbb{R}^{m\times n}$, there exist orthogonal matrices $\mathbf{U} \in \mathbb{R}^{m\times m}, \mathbf{V} \in \mathbb{R}^{n\times n}$ and diagonal $\mathbf{\Sigma} \in \mathbb{R}^{m\times n}$ such that:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T.$$

- **Singular Values (σ_i):** Non-negative, ordered $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.
- **Modes:** $\mathbf{A} = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$ (Outer product expansion).

Remark 4.6 **Geometric Interpretation:** $\mathbf{A}$ rotates the input space ($\mathbf{V}^T$), scales along the axes ($\mathbf{\Sigma}$), and rotates the result to the output space ($\mathbf{U}$).

Thm 4.6 **(SVD Coordinates)** Let $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ be the SVD from Thm 4.5. If $\boldsymbol{x} = \mathbf{V}\boldsymbol{y}$ and $\boldsymbol{c} = \mathbf{U}^T\boldsymbol{b}$, then the equation $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ becomes

$$\mathbf{\Sigma}\boldsymbol{y} = \boldsymbol{c}.$$

Thus the action of $\mathbf{A}$ is a set of independent scalar equations $\sigma_i y_i = c_i$ in orthonormal coordinates. Small singular values correspond to directions where solving requires division by small numbers.

Ex 4.5

1. Substitute $\boldsymbol{x} = \mathbf{V}\boldsymbol{y}$ into $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ and use Thm 4.5 to obtain $\mathbf{U}\mathbf{\Sigma}\boldsymbol{y} = \boldsymbol{b}$.
2. Multiply by $\mathbf{U}^T$ and use orthogonality of $\mathbf{U}$ to prove Thm 4.6.
3. For a square full-rank matrix, use the scalar equations $\sigma_i y_i = c_i$ to derive $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$.
4. Explain why a tiny nonzero σ_i makes the system numerically close to rank deficient.
5. Use the same coordinate picture to explain the pseudoinverse in Def 4.11.

Remark 4.7 **(Full vs. thin SVD)** If $m \geq n$, the full SVD has $\mathbf{U} \in \mathbb{R}^{m\times m}$ and $\mathbf{\Sigma} \in \mathbb{R}^{m\times n}$. The thin SVD keeps only the first n left singular vectors:

$$\mathbf{A} = \mathbf{U}_1\mathbf{\Sigma}_1\mathbf{V}^T, \qquad \mathbf{U}_1 \in \mathbb{R}^{m\times n}, \quad \mathbf{\Sigma}_1 \in \mathbb{R}^{n\times n}.$$

For most computations, the thin SVD contains all nonzero singular information and avoids storing unnecessary columns.

Def 4.10 **(Numerical Rank)** The numerical rank of $\mathbf{A}$ is the number of singular values considered meaningfully nonzero relative to a tolerance:

$$\sigma_i > \tau.$$

A common scale-aware choice is $\tau = \varepsilon_{\text{mach}} \max(m, n)\sigma_1$.

Remark 4.8 **(Conditioning through singular values)** As shown in Ex 4.5, for a full-rank square matrix,

$$\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}}.$$

Small singular values mark directions where $\mathbf{A}$ nearly collapses information. Solving a system must divide by these small values, amplifying noise and rounding error.

4.2.2 The Four Fundamental Subspaces

The SVD provides optimal orthonormal bases for the spaces defined by $\mathbf{A}$ (rank r):

1. **Column Space ($\mathcal{R}(\mathbf{A})$):** First r columns of $\mathbf{U}$.
2. **Null Space ($\mathcal{N}(\mathbf{A})$):** Last $n - r$ columns of $\mathbf{V}$.
3. **Row Space ($\mathcal{R}(\mathbf{A}^T)$):** First r columns of $\mathbf{V}$.
4. **Left Null Space ($\mathcal{N}(\mathbf{A}^T)$):** Last $m - r$ columns of $\mathbf{U}$.

Thm 4.7 **(Relationship to Eigendecomposition)**

- Right singular vectors ($\boldsymbol{v}_i$) are eigenvectors of $\mathbf{A}^T\mathbf{A}$.
- Left singular vectors ($\boldsymbol{u}_i$) are eigenvectors of $\mathbf{A}\mathbf{A}^T$.
- Singular values $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^T\mathbf{A})}$.

Remark 4.9 **(SVD via normal equations)** The relation to eigenvalues explains why the SVD exists, but forming $\mathbf{A}^T\mathbf{A}$ squares the condition number. Numerical SVD algorithms avoid this loss of accuracy.

Ex 4.6

1. Use Thm 4.7 to show that $\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2$ for full-rank $\mathbf{A}$.
2. Compare this result with Ex 3.16.
3. Explain why this makes $\mathbf{A}^T\mathbf{A}$ useful for theory but dangerous as a default computational route.

4.2.3 Low-Rank Approximation

Thm 4.8 **(Eckart-Young-Mirsky Theorem)** The best rank-k approximation ($k < r$) of $\mathbf{A}$ in spectral and Frobenius norms is the truncated SVD:

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T.$$

- **Error:** $\|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}$.

Remark 4.10 **Applications:** Truncated SVD provides the computational basis for Principal Component Analysis (PCA), image compression, and latent semantic analysis.

Remark 4.11 **(Singular value decay)** If the singular values decay rapidly, the matrix is well approximated by a low-rank matrix. If they decay slowly, no low-dimensional linear summary captures the matrix accurately.

Example **(Image compression)** A grayscale image can be treated as a matrix of pixel intensities. Keeping only the largest k singular values stores

$$k(m + n + 1)$$

numbers instead of mn. The approximation error in spectral norm is exactly the next omitted singular value σ_{k+1}.

4.2.4 The Pseudoinverse

Def 4.11 **(Moore-Penrose Pseudoinverse)** $\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^T$, where $(\mathbf{\Sigma}^+)_{ii} = 1/\sigma_i$ if $\sigma_i > 0$, else 0.

- **Minimum-Norm Solution:** For any system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, the vector $\hat{\boldsymbol{x}} = \mathbf{A}^+\boldsymbol{b}$ is the shortest vector minimizing $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2$.

Remark 4.12 **(Why the pseudoinverse is the fallback)** In SVD coordinates, solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ means dividing by singular values. The pseudoinverse divides only by nonzero singular values and ignores null-space directions. This gives the minimum-norm least squares solution even when $\mathbf{A}$ is rectangular or rank deficient.

Remark 4.13 **(Regularization by truncation)** If small singular values are dominated by noise, dividing by them can make the solution explode. Truncated SVD sets small singular values to zero before applying the pseudoinverse. This trades bias for stability.

Ex 4.7

1. Compute the SVD of $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}$ by hand and verify the coordinate equation in Thm 4.6.
2. **Compression:** Load a 512×512 image. Plot the relative error $\|\mathbf{A} - \mathbf{A}_k\|_F / \|\mathbf{A}\|_F$ vs. k. At what k is the image recognizable?
3. Use the pseudoinverse to find the least squares solution to a rank-deficient 3×3 system. Explain each zero singular value using Def 4.10.

4.3 Lanczos Algorithm

Iterative method for finding extreme eigenpairs of large, symmetric matrices without explicit $O(n^2)$ formation.

Remark 4.14 **(Symmetric assumption)** Lanczos applies to symmetric or Hermitian matrices. For nonsymmetric matrices, use Arnoldi or GMRES-type constructions.

4.3.1 Krylov Subspaces

Def 4.12 **(Krylov Subspace)** $\mathcal{K}_k(\mathbf{A}, \boldsymbol{v}) = \text{span}\{\boldsymbol{v}, \mathbf{A}\boldsymbol{v}, \mathbf{A}^2\boldsymbol{v}, \ldots, \mathbf{A}^{k-1}\boldsymbol{v}\}$. $\mathbf{A}^j\boldsymbol{v}$ amplifies components along dominant eigenvectors, concentrating information about the spectral edges.

Remark 4.15 **(Power iteration with memory)** Power iteration keeps only the newest vector. Lanczos keeps an orthonormal basis for the whole Krylov subspace, so it can approximate several eigenvalues at once.

4.3.2 The Lanczos Recurrence

Thm 4.9 **(Lanczos Relation)** The process builds an orthonormal basis $V_k = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k]$ for $\mathcal{K}_k$ satisfying:

$$\mathbf{A}V_k = V_k T_k + \beta_k \boldsymbol{v}_{k+1} \boldsymbol{e}_k^T.$$

- T_k**:** Symmetric **tridiagonal** matrix.
- $\alpha_j = \boldsymbol{v}_j^T \mathbf{A} \boldsymbol{v}_j$**:** Rayleigh quotients (diagonal).
- $\beta_j = \|$**residual**$\|$**:** Orthogonalization coefficients (off-diagonal).

Prop 4.10 **(Galerkin Projection)** In exact arithmetic,

$$T_k = V_k^T \mathbf{A} V_k.$$

Thus Lanczos replaces the large symmetric matrix $\mathbf{A}$ by a small symmetric tridiagonal projection.

Proof *Proof.* Left multiply the Lanczos relation in Thm 4.9 by V_k^T:

$$V_k^T \mathbf{A} V_k = V_k^T V_k T_k + \beta_k V_k^T \boldsymbol{v}_{k+1} \boldsymbol{e}_k^T.$$

Since the Lanczos vectors are orthonormal, $V_k^T V_k = \mathbf{I}$ and $V_k^T \boldsymbol{v}_{k+1} = \mathbf{0}$. Hence $V_k^T \mathbf{A} V_k = T_k$. □

Remark 4.16 **Performance Advantage:** Unlike Gram-Schmidt, which orthogonalizes against all previous vectors, Lanczos only requires the **two previous vectors** (three-term recurrence). This makes it $O(k \cdot \text{nnz})$ for sparse matrices.

4.3.3 Ritz Values and Vectors

Approximations to the true eigenpairs from the Krylov subspace.

Def 4.13 **(Ritz Pair)**

- **Ritz Value (θ_i):** Eigenvalue of T_k.
- **Ritz Vector ($\tilde{\boldsymbol{u}}_i$):** $V_k \boldsymbol{s}_i$, where $\boldsymbol{s}_i$ is an eigenvector of T_k.

Thm 4.11 **(Residual Estimate)** The residual of the i-th Ritz pair $(\theta_i, \tilde{\boldsymbol{u}}_i)$ satisfies:

$$\|\mathbf{A}\tilde{\boldsymbol{u}}_i - \theta_i \tilde{\boldsymbol{u}}_i\|_2 = \beta_k |s_i(k)|.$$

Significance: Convergence of a Ritz pair can be monitored for $O(k)$ cost without forming the full n-vector $\tilde{\boldsymbol{u}}_i$.

Proof *Proof.* Starting from the Lanczos relation $\mathbf{A}V_k = V_k T_k + \beta_k \boldsymbol{v}_{k+1} \boldsymbol{e}_k^T$, multiply by the i-th eigenvector $\boldsymbol{s}_i$ of T_k on the right:

$$\begin{aligned} \mathbf{A} V_k \boldsymbol{s}_i &= V_k T_k \boldsymbol{s}_i + \beta_k \boldsymbol{v}_{k+1} \boldsymbol{e}_k^T \boldsymbol{s}_i \\ \mathbf{A}\tilde{\boldsymbol{u}}_i &= V_k(\theta_i \boldsymbol{s}_i) + \beta_k \boldsymbol{v}_{k+1} s_i(k). \end{aligned}$$

where $s_i(k)$ is the k-th entry of $\boldsymbol{s}_i$. Rearranging:

$$\mathbf{A}\tilde{\boldsymbol{u}}_i - \theta_i \tilde{\boldsymbol{u}}_i = \beta_k \boldsymbol{v}_{k+1} s_i(k).$$

Taking the L_2 norm and using the fact that $\|\boldsymbol{v}_{k+1}\|_2 = 1$:

$$\begin{aligned} \|\mathbf{A}\tilde{\boldsymbol{u}}_i - \theta_i \tilde{\boldsymbol{u}}_i\|_2 &= |\beta_k s_i(k)| \|\boldsymbol{v}_{k+1}\|_2 \\ &= \beta_k |s_i(k)|. \end{aligned}$$

□

Remark 4.17 **Ghost Eigenvalues:** In floating-point, the three-term recurrence loses orthogonality, leading to "ghost" (spurious) copies of eigenvalues. **(Reorthogonalization)** Use full or selective reorthogonalization to maintain basis integrity.

Remark 4.18 **Connection to Conjugate Gradient:** The Lanczos algorithm applied to a symmetric system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is mathematically equivalent to the Conjugate Gradient method. Both algorithms build the same Krylov subspace and produce identical iterates in exact arithmetic.

Ex 4.8

1. Carry out 2 steps of Lanczos for $\mathbf{A} = \mathrm{diag}(5, 3, 1)$ starting with $\boldsymbol{v}_1 = \frac{1}{\sqrt{3}}\mathbf{1}$.
2. Verify Prop 4.10 numerically for a random symmetric matrix.
3. Implement Lanczos on a 50×50 Hilbert matrix. Count spurious eigenvalues without reorthogonalization.

4.4 Jacobi Eigenvalue Algorithm

Iterative algorithm for symmetric matrices using plane rotations to zero off-diagonal entries.

4.4.1 Givens Rotations

Def 4.14 **(Givens Rotation)** $G(p, q, \theta)$ is an identity matrix with a 2×2 rotation block at $(p,p), (p,q), (q,p), (q,$

- **Similarity Transform:** $\mathbf{A}' = G^T\mathbf{A}G$.
- **Effect:** Modifies only rows/columns p and q. Preserves eigenvalues ($\mathbf{A}'$ is orthogonally similar to $\mathbf{A}$).

Thm 4.12 **(Optimal Rotation Angle)** To set $A'_{pq} = 0$, let $\tau = (A_{qq} - A_{pp})/(2A_{pq})$. The optimal tangent $t = \tan\theta$ is:

$$t = \frac{\mathrm{sign}(\tau)}{|\tau| + \sqrt{1+\tau^2}}, \quad c = \frac{1}{\sqrt{1+t^2}}, \quad s = ct.$$

Choosing the smaller root $|t| \leq 1$ ensures numerical stability and minimal perturbation.

4.4.2 The Jacobi Algorithm

The Jacobi eigenvalue algorithm repeatedly applies orthogonal similarity transformations

$$\mathbf{A}^{(k+1)} = \mathbf{G}_k^T \mathbf{A}^{(k)} \mathbf{G}_k$$

to a symmetric matrix. Each rotation zeros one off-diagonal entry while preserving eigenvalues. As the off-diagonal entries converge to zero, $\mathbf{A}^{(k)}$ converges to a diagonal matrix whose diagonal entries are the eigenvalues.

Def 4.15 **(Jacobi Eigenvalue Algorithm) Input:** Symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ and tolerance `tol`.
Initialize: $\mathbf{A}^{(0)} = \mathbf{A}$, $\mathbf{V}^{(0)} = \mathbf{I}$.
Repeat:

1. Choose an off-diagonal pivot (p, q) with $p < q$.
2. Compute $c = \cos\theta$ and $s = \sin\theta$ using the stable tangent formula so that $(\mathbf{G}^T \mathbf{A}^{(k)} \mathbf{G})_{pq} = 0$.
3. Update the matrix by orthogonal similarity:
$$\mathbf{A}^{(k+1)} = \mathbf{G}^T \mathbf{A}^{(k)} \mathbf{G}.$$
4. Accumulate eigenvectors:
$$\mathbf{V}^{(k+1)} = \mathbf{V}^{(k)} \mathbf{G}.$$

Stop: when $\sigma_{\text{off}}(\mathbf{A}^{(k)}) \leq \texttt{tol} \|\mathbf{A}^{(k)}\|_F$.
Output: Diagonal entries of $\mathbf{A}^{(k)}$ approximate eigenvalues; columns of $\mathbf{V}^{(k)}$ approximate eigenvectors.

Remark 4.19 **(Pivot choices)**

- **Classical Jacobi:** choose the largest off-diagonal entry $|A_{pq}|$.
- **Cyclic Jacobi:** sweep through all pairs (p, q) in a fixed order.

The largest-entry rule gives the clearest convergence proof. Cyclic sweeps are simpler to implement and often used in practice.

Remark 4.20 **(Why eigenvectors accumulate)** After k rotations,

$$\mathbf{A}^{(k)} = (\mathbf{V}^{(k)})^T \mathbf{A} \mathbf{V}^{(k)}.$$

If $\mathbf{A}^{(k)}$ is nearly diagonal, then $\mathbf{A}\mathbf{V}^{(k)} \approx \mathbf{V}^{(k)}\mathbf{\Lambda}$, so the columns of $\mathbf{V}^{(k)}$ are approximate eigenvectors of the original matrix.

4.4.3 Convergence and Cost

Thm 4.13 **(Monotone Convergence)** Let $\sigma_{\text{off}}^2(\mathbf{A}) = \sum_{p<q} A_{pq}^2$ be the Frobenius norm of the off-diagonal entries. Each rotation $G(p, q, \theta)$ reduces this norm:

$$\sigma_{\text{off}}^2(\mathbf{A}') = \sigma_{\text{off}}^2(\mathbf{A}) - A_{pq}^2.$$

Convergence: Off-diagonals decrease monotonically. Convergence is ultimately **quadratic** (errors square each sweep).

Proof *Proof.* The orthogonal transform $\mathbf{A}' = G^T \mathbf{A} G$ preserves the Frobenius norm: $\|\mathbf{A}'\|_F^2 = \|\mathbf{A}\|_F^2$. Let $d(\mathbf{A}) = \sum A_{ii}^2$ be the sum of squares of diagonal entries. Then:

$$\|\mathbf{A}\|_F^2 = d(\mathbf{A}) + 2\sigma_{\text{off}}^2(\mathbf{A}).$$

A Jacobi rotation $G(p, q, \theta)$ only affects rows and columns p and q. For the 2×2 submatrix at $\{p, q\}$, the transformation is an eigendecomposition, so:

$$\begin{aligned}(A'_{pp})^2 + (A'_{qq})^2 &= A_{pp}^2 + A_{qq}^2 + 2A_{pq}^2 \\ d(\mathbf{A}') &= d(\mathbf{A}) + 2A_{pq}^2.\end{aligned}$$

Substituting this into the norm conservation $\|\mathbf{A}'\|_F^2 = \|\mathbf{A}\|_F^2$:

$$\begin{aligned}d(\mathbf{A}') + 2\sigma_{\text{off}}^2(\mathbf{A}') &= d(\mathbf{A}) + 2\sigma_{\text{off}}^2(\mathbf{A}) \\ (d(\mathbf{A}) + 2A_{pq}^2) + 2\sigma_{\text{off}}^2(\mathbf{A}') &= d(\mathbf{A}) + 2\sigma_{\text{off}}^2(\mathbf{A}) \\ \sigma_{\text{off}}^2(\mathbf{A}') &= \sigma_{\text{off}}^2(\mathbf{A}) - A_{pq}^2.\end{aligned}$$

Thus each rotation reduces the off-diagonal mass by exactly A_{pq}^2. □

Remark 4.21 **Performance vs. Accuracy:**

- **Cost:** $O(n^3)$ per sweep. Typically 5–15 sweeps for convergence. More expensive than QR.
- **Accuracy:** Unlike QR, Jacobi achieves **high relative accuracy** on small eigenvalues.

Ex 4.9

1. Apply one Jacobi rotation to zero A_{12} in $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$.
2. Show that after zeroing A_{pq}, the new diagonal entries A'_{pp} and A'_{qq} are the eigenvalues of the 2×2 submatrix at $\{p, q\}$.
3. Perform one full cyclic sweep on $\mathbf{A} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 3 \end{pmatrix}$ by hand.

5 Iterative Methods for Linear Systems

5.1 Iterative Methods for Linear Systems

Alternative to direct solvers for large, sparse systems where matrix factorization is impractical due to memory or compute limits.

Direct methods factor the matrix. Iterative methods repeatedly improve an approximate solution using cheap operations, usually matrix-vector products. They are preferred when $\mathbf{A}$ is large, sparse, or available only as a function that computes $\mathbf{A}\boldsymbol{v}$.

5.1.1 Direct vs. Iterative Solvers

Remark 5.1 **(Sparse fill-in)** Even if $\mathbf{A}$ is sparse, its LU or Cholesky factors are typically much denser (**fill-in**). For a 2D Poisson problem on an $n \times n$ grid:

- $\mathbf{A}$ has $O(n^2)$ nonzeros.
- LU factors have $O(n^3)$ nonzeros.

Iterative methods exploit sparsity by using only matrix-vector products ($O(\text{nnz})$).

Remark 5.2 **(Matrix-free viewpoint)** Many scientific codes never assemble $\mathbf{A}$ explicitly. They only provide a routine that maps $\boldsymbol{v} \mapsto \mathbf{A}\boldsymbol{v}$. Krylov methods such as CG and GMRES are designed for this setting.

Def 5.1 **(Krylov Subspace)** Given a matrix $\mathbf{A}$ and a vector $\boldsymbol{v}$, the k-th Krylov subspace is

$$\mathcal{K}_k(\mathbf{A}, \boldsymbol{v}) = \operatorname{span}\{\boldsymbol{v}, \mathbf{A}\boldsymbol{v}, \mathbf{A}^2\boldsymbol{v}, \ldots, \mathbf{A}^{k-1}\boldsymbol{v}\}.$$

It is the smallest subspace that contains $\boldsymbol{v}$ and is built by repeated multiplication by $\mathbf{A}$.

Thm 5.1 **(Krylov Approximation Principle)** Let $\boldsymbol{r}^{(0)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)}$. Any iterate of the form

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + p_{k-1}(\mathbf{A})\boldsymbol{r}^{(0)},$$

where p_{k-1} is a polynomial of degree at most $k-1$, lies in the affine space

$$\boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)}).$$

Thus a matrix-free method can improve $\boldsymbol{x}^{(0)}$ by choosing increasingly good corrections from Krylov subspaces using only matrix-vector products.

Ex 5.1

1. Write $p_{k-1}(t) = c_0 + c_1 t + \cdots + c_{k-1}t^{k-1}$ and expand $p_{k-1}(\mathbf{A})\boldsymbol{r}^{(0)}$.
2. Use Def 5.1 to show that this correction belongs to $\mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$.

3. List $\mathcal{K}_1$, $\mathcal{K}_2$, and $\mathcal{K}_3$ explicitly. What new vector must be computed to move from $\mathcal{K}_k$ to $\mathcal{K}_{k+1}$?

4. Explain why Thm 5.1 makes Krylov methods compatible with the matrix-free viewpoint above.

5. Preview the later chapters: CG chooses the best correction in an energy norm for SPD systems, while GMRES chooses the correction with smallest residual norm.

Def 5.2 **(Stationary Iteration)** An iteration of the form $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$, where:

- $\mathbf{T} = \mathbf{I} - \mathbf{M}^{-1}\mathbf{A}$ is the iteration matrix.
- $\boldsymbol{c} = \mathbf{M}^{-1}\boldsymbol{b}$ is the updated right-hand side.
- $\mathbf{M}$ is the **splitting matrix** (ideally $\mathbf{M} \approx \mathbf{A}$ and $\mathbf{M}^{-1}$ is cheap).

Stationary iterations are fixed-point iterations. Instead of solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ directly, we rewrite the problem as $\boldsymbol{x} = \mathbf{T}\boldsymbol{x} + \boldsymbol{c}$ and repeatedly apply the map. The exact solution is a fixed point of this map.

Remark 5.3 **(Splitting form)** If $\mathbf{A} = \mathbf{M} - \mathbf{N}$, then

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \quad \Longleftrightarrow \quad \mathbf{M}\boldsymbol{x} = \mathbf{N}\boldsymbol{x} + \boldsymbol{b}.$$

The iteration is

$$\boldsymbol{x}^{(k+1)} = \mathbf{M}^{-1}\mathbf{N}\boldsymbol{x}^{(k)} + \mathbf{M}^{-1}\boldsymbol{b}.$$

The art is choosing $\mathbf{M}$ easy to invert while making the iteration converge quickly.

5.1.2 Convergence and Spectral Radius

Def 5.3 **(Spectral Radius)** $\rho(\mathbf{B}) = \max\{|\lambda| : \lambda \in \sigma(\mathbf{B})\}$.

Remark 5.4 **(Spectral radius intuition)** The eigenvalue of largest magnitude controls the long-run behavior of repeated multiplication. If every eigenvalue of $\mathbf{T}$ lies inside the unit disk, powers of $\mathbf{T}$ decay and the iteration converges.

Thm 5.2 **(Gelfand's Formula)** For any submultiplicative norm, $\rho(\mathbf{B}) = \lim_{k\to\infty} \|\mathbf{B}^k\|^{1/k}$. For large k, $\|\mathbf{B}^k\| \approx \rho(\mathbf{B})^k$.

Thm 5.3 **(Fundamental Convergence Condition)** The iteration $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$ converges for any $\boldsymbol{x}^{(0)}$ iff $\rho(\mathbf{T}) < 1$.

Proof *Proof.* Let $\boldsymbol{x}^*$ be the exact solution satisfying $\boldsymbol{x}^* = \mathbf{T}\boldsymbol{x}^* + \boldsymbol{c}$. Subtracting this from the iteration formula gives the error evolution:

$$\begin{aligned} \boldsymbol{x}^{(k+1)} - \boldsymbol{x}^* &= (\mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}) - (\mathbf{T}\boldsymbol{x}^* + \boldsymbol{c}) \\ \boldsymbol{e}^{(k+1)} &= \mathbf{T}\boldsymbol{e}^{(k)}. \end{aligned}$$

Applying this relation recursively from $k = 0$:

$$\boldsymbol{e}^{(k)} = \mathbf{T}^k \boldsymbol{e}^{(0)}.$$

The error $\boldsymbol{e}^{(k)} \to \mathbf{0}$ for any $\boldsymbol{e}^{(0)}$ iff $\mathbf{T}^k \to \mathbf{0}$ as $k \to \infty$, which is guaranteed iff $\rho(\mathbf{T}) < 1$ by Gelfand's formula. □

Remark 5.5 **(Iteration count)** To reduce error by 10^{-p}, the required iterations are $k \approx \frac{p}{\log_{10}(1/\rho(\mathbf{T}))}$. As $\rho(\mathbf{T}) \to 1^-$, the cost explodes.

5.1.3 Stopping Criteria

Iterative methods need a stopping rule. The true error $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|$ is usually unknown, so algorithms monitor the residual $\boldsymbol{r}^{(k)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(k)}$.

Remark 5.6 **(Common stopping test)** Stop when

$$\frac{\|\boldsymbol{r}^{(k)}\|}{\|\boldsymbol{b}\|} \leq \texttt{tol}.$$

This is a relative residual test. It is cheap and scale-aware, but it is not the same as a relative error test unless the problem is well-conditioned.

Remark 5.7 **(Preconditioning preview)** Most useful iterative solvers are preconditioned. A preconditioner $\mathbf{M} \approx \mathbf{A}$ should be cheap to solve with and should make the transformed system easier for the iterative method. Good preconditioning changes the problem from "possible but slow" to practical.

Ex 5.2

1. For $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, find $\rho(\mathbf{T})$ for the splitting $\mathbf{M} = 3\mathbf{I}$.
2. If $\rho(\mathbf{T}) = 0.99$, how many iterations are needed to gain 4 digits of precision?
3. Start $\boldsymbol{x}^{(0)} = \mathbf{0}$ and solve $\mathbf{A}\boldsymbol{x} = (4, 4)^T$ via 3 steps of this iteration.

5.1.4 Fill-in and Sparse Storage

Ex 5.3

1. **Fill-in Demo:** Generate a large sparse tridiagonal matrix. Compare nonzeros in $\mathbf{A}$ vs. LU factors.

2. **Timing:** Use `scipy.sparse.linalg.spsolve` vs. dense `np.linalg.solve` for $n = 2000$. Record memory and time.

5.2 Jacobi and Gauss-Seidel Methods

Standard stationary iterations based on diagonal and triangular splittings.

5.2.1 Jacobi Iteration

Decomposes $\mathbf{A}$ into diagonal $\mathbf{D}$ and remainder $\mathbf{R}$: $\mathbf{A} = \mathbf{D} + \mathbf{R}$.

Def 5.4 **(Jacobi Scheme)**

$$\boldsymbol{x}^{(k+1)} = \mathbf{D}^{-1}(\boldsymbol{b} - \mathbf{R}\boldsymbol{x}^{(k)})$$

- **Component form:** $x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$.
- **Concurrency:** All components update independently using values from the *previous* step.

Remark 5.8 **(Jacobi performance)** Jacobi iteration is **inherently parallel**, as all components can be updated simultaneously. However, it often exhibits slow convergence and requires double buffering of the solution vector.

5.2.2 Gauss-Seidel Iteration

Splits $\mathbf{A} = \mathbf{L}_* + \mathbf{U}_*$, where $\mathbf{L}_*$ includes the diagonal.

Def 5.5 **(Gauss-Seidel Scheme)**

$$\mathbf{L}_* \boldsymbol{x}^{(k+1)} = \boldsymbol{b} - \mathbf{U}_* \boldsymbol{x}^{(k)}$$

- **Component form:** $x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right)$.
- **Update rule:** Use updated values *immediately* within the same sweep.

Remark 5.9 **(Gauss-Seidel performance)** Sequential dependencies restrict efficient parallelization, but Gauss-Seidel typically converges **twice as fast** as Jacobi and allows for in-place updates.

5.2.3 Convergence Conditions

Thm 5.4 **(Stationary Iteration Matrices)** For $\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$, where $\mathbf{L}$ and $\mathbf{U}$ are the strict lower and upper triangular parts,

$$\mathbf{T}_J = -\mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})$$

is the Jacobi iteration matrix, and

$$\mathbf{T}_{GS} = -(\mathbf{D} + \mathbf{L})^{-1}\mathbf{U}$$

is the Gauss-Seidel iteration matrix. By Thm 5.3, each method converges for every initial guess exactly when the corresponding spectral radius is less than 1.

Ex 5.4

1. Rewrite Def 5.4 in the form $\boldsymbol{x}^{(k+1)} = \mathbf{T}_J\boldsymbol{x}^{(k)} + \boldsymbol{c}_J$.
2. Rewrite Def 5.5 in the form $\boldsymbol{x}^{(k+1)} = \mathbf{T}_{GS}\boldsymbol{x}^{(k)} + \boldsymbol{c}_{GS}$.
3. Apply Thm 5.3 to prove the convergence criterion in Thm 5.4.
4. Compute $\mathbf{T}_J$ and $\mathbf{T}_{GS}$ for $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$.

Thm 5.5 **(Diagonal Dominance)** If $\mathbf{A}$ is **strictly diagonally dominant** ($|a_{ii}| > \sum_{j \neq i} |a_{ij}|$), both Jacobi and Gauss-Seidel converge for any $\boldsymbol{x}^{(0)}$.

Ex 5.5

1. For Jacobi, use strict diagonal dominance to show that the infinity-norm row sums of $\mathbf{T}_J$ are less than 1.
2. Conclude $\rho(\mathbf{T}_J) \leq \|\mathbf{T}_J\|_\infty < 1$.
3. Repeat the argument for Gauss-Seidel by isolating the largest component of the error equation.

Thm 5.6 **(SPD Convergence)** If $\mathbf{A}$ is **SPD**, Gauss-Seidel is guaranteed to converge. Jacobi may still diverge.

Ex 5.6

1. For $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, compute one step of Jacobi and Gauss-Seidel from $\mathbf{0}$.
2. Compare $\rho(\mathbf{T}_J)$ and $\rho(\mathbf{T}_{GS})$. Does $\rho_{GS} \approx \rho_J^2$ hold?

5.2.4 Successive Over-Relaxation (SOR)

Accelerates Gauss-Seidel by overshooting the update.

Def 5.6 **(SOR Update)**

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \omega\,[\text{GS update}]$$

Thm 5.7 **(SOR Convergence Window)** For SPD systems, SOR converges for relaxation parameters

$$0 < \omega < 2.$$

For model Poisson problems, well-chosen $\omega > 1$ can reduce the iteration count substantially compared with Gauss-Seidel.

Remark 5.10 **(SOR acceleration)** For the 1D Poisson problem on a grid with spacing h:

- Jacobi and Gauss-Seidel require $O(n^2)$ iterations.
- Optimal SOR requires only $O(n)$ iterations.

SOR effectively improves the convergence rate from $O(h^2)$ to $O(h)$.

Ex 5.7

1. **Optimal Tuning:** For $\rho_J = 0.95$, calculate the optimal $\omega^* = 2/(1+\sqrt{1-\rho_J^2})$.
2. **Convergence Sweep:** Implement SOR and plot iterations to convergence vs. $\omega \in [1, 2)$. Identify the "cliff" at ω^*.
3. Check numerically that choices $\omega \leq 0$ or $\omega \geq 2$ fail on an SPD Poisson matrix, as predicted by Thm 5.7.

5.2.5 Stopping Criteria

Def 5.7 **(Iterative Stopping Tests)** Common stopping tests include

1. absolute residual: $\|\boldsymbol{r}^{(k)}\| < \varepsilon$,
2. relative residual: $\|\boldsymbol{r}^{(k)}\|/\|\boldsymbol{b}\| < \varepsilon$,
3. increment: $\|\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}\| < \varepsilon$.

Residual tests are tied to the equation; increment tests are cheap but can stop prematurely if convergence is slow.

Ex 5.8

1. Why is a small residual not enough to guarantee a small error $\boldsymbol{e}^{(k)}$? Use Thm 3.4.
2. Implement a robust solver that combines relative residual with a `max_iter` budget.

5.3 Conjugate Gradient Method

The Conjugate Gradient (CG) method is a standard iterative solver for large, sparse, symmetric positive definite (SPD) linear systems. It belongs to the class of **Krylov subspace methods**, which seek an approximate solution within a space spanned by powers of the matrix applied to the initial residual.

Remark 5.11 **(Assumption)** CG requires $\mathbf{A}$ to be symmetric positive definite. If $\mathbf{A}$ is nonsymmetric, use GMRES or another nonsymmetric Krylov method. If $\mathbf{A}$ is symmetric indefinite, use methods designed for indefinite systems, such as MINRES.

For SPD $\mathbf{A}$, solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is equivalent to minimizing a convex quadratic energy. CG chooses search directions that do not interfere with previous progress, so each step is optimal over a growing Krylov subspace.

5.3.1 Krylov Subspaces

Def 5.8 **(Krylov Subspace)** The k-th Krylov subspace $\mathcal{K}_k(\mathbf{A}, \boldsymbol{b})$ is defined as the span of the first k vectors in the sequence $\{\boldsymbol{b}, \mathbf{A}\boldsymbol{b}, \mathbf{A}^2\boldsymbol{b}, \dots\}$:

$$\mathcal{K}_k(\mathbf{A}, \boldsymbol{b}) = \text{span}\{\boldsymbol{b}, \mathbf{A}\boldsymbol{b}, \mathbf{A}^2\boldsymbol{b}, \dots, \mathbf{A}^{k-1}\boldsymbol{b}\}.$$

Krylov methods are efficient because they only require the ability to compute matrix-vector products $\mathbf{A}\boldsymbol{v}$, making them ideal for sparse matrices where $\mathbf{A}$ is never explicitly stored.

5.3.2 CG as a Projection Method

For SPD $\mathbf{A}$, the quadratic

$$\phi(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} - \boldsymbol{b}^T\boldsymbol{x}$$

has gradient $\nabla\phi(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x} - \boldsymbol{b}$. Therefore $\nabla\phi(\boldsymbol{x}) = 0$ exactly when $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. Solving the linear system is the same as finding the unique minimizer of ϕ.

Remark 5.12 **Error Minimization Properties:** For an SPD system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, the CG method identifies the unique vector $\boldsymbol{x}_k \in \mathcal{K}_k(\mathbf{A}, \boldsymbol{b})$ that minimizes the **A-norm of the error**:

$$\boldsymbol{x}_k = \arg\min_{\boldsymbol{y}\in\mathcal{K}_k} \|\boldsymbol{x}^* - \boldsymbol{y}\|_{\mathbf{A}},$$

where $\boldsymbol{x}^*$ is the exact solution and $\|\boldsymbol{e}\|_{\mathbf{A}} = \sqrt{\boldsymbol{e}^T\mathbf{A}\boldsymbol{e}}$. This is equivalent to minimizing the quadratic energy functional $\phi(\boldsymbol{y}) = \frac{1}{2}\boldsymbol{y}^T\mathbf{A}\boldsymbol{y} - \boldsymbol{y}^T\boldsymbol{b}$.

5.3.3 Conjugate Directions and A-Orthogonality

Remark 5.13 **The Basis of Directions:** To find this minimum efficiently, CG constructs a basis of search directions $\{\boldsymbol{p}_0, \boldsymbol{p}_1, \ldots, \boldsymbol{p}_{k-1}\}$ that are **A-orthogonal** (or conjugate), meaning $\boldsymbol{p}_i^T \mathbf{A} \boldsymbol{p}_j = 0$ for $i \neq j$.

- **Global Optimality:** Because directions are $\mathbf{A}$-orthogonal, the k-th update $\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \alpha_{k-1}\boldsymbol{p}_{k-1}$ is guaranteed to be the best possible update in the entire subspace $\mathcal{K}_k$. There is no need to revisit previous directions.
- **Short Recurrence:** Due to the symmetry of $\mathbf{A}$, the new conjugate direction $\boldsymbol{p}_k$ can be computed using only the current residual and the *single* previous direction. This leads to the $O(n)$ storage requirement.

Remark 5.14 **(Residuals vs. directions)** In exact arithmetic, CG residuals are mutually orthogonal in the Euclidean inner product, while search directions are mutually orthogonal in the $\mathbf{A}$-inner product. These two orthogonality structures are what make the short recurrence possible.

Remark 5.15 **Why not steepest descent?** Steepest descent always moves in the negative gradient direction, but successive gradients can undo previous progress by zig-zagging across narrow energy valleys. CG corrects this by choosing conjugate directions adapted to the geometry of $\mathbf{A}$.

5.3.4 The CG Algorithm

Def 5.9 **(Conjugate Gradient Algorithm) Initialize:** $\boldsymbol{x}^{(0)} = \mathbf{0}, \boldsymbol{r}^{(0)} = \boldsymbol{b}, \boldsymbol{p}^{(0)} = \boldsymbol{r}^{(0)}$.
Iteration:

1. $\alpha_k = \frac{\boldsymbol{r}^{(k)T}\boldsymbol{r}^{(k)}}{\boldsymbol{p}^{(k)T}\mathbf{A}\boldsymbol{p}^{(k)}}$ (Step length)
2. $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{p}^{(k)}$ (Update iterate)
3. $\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_k \mathbf{A}\boldsymbol{p}^{(k)}$ (Update residual)
4. $\beta_k = \frac{\boldsymbol{r}^{(k+1)T}\boldsymbol{r}^{(k+1)}}{\boldsymbol{r}^{(k)T}\boldsymbol{r}^{(k)}}$ (Gram-Schmidt weight)
5. $\boldsymbol{p}^{(k+1)} = \boldsymbol{r}^{(k+1)} + \beta_k \boldsymbol{p}^{(k)}$ (Next direction)

Remark 5.16 **Efficiency:**

- **Matvecs:** Only 1 matrix-vector product per iteration.
- **Memory:** $O(n)$ storage (requires only a few vectors). No need to store the basis.

Remark 5.17 **(Practical stopping)** A common stopping rule is

$$\frac{\|\boldsymbol{r}^{(k)}\|_2}{\|\boldsymbol{b}\|_2} \leq \texttt{tol}.$$

For ill-conditioned systems, residual decrease may not translate directly into error decrease. The condition number controls that conversion.

5.3.5 Convergence Rate

Thm 5.8 **(Convergence Bound)**

$$\frac{\|\boldsymbol{e}^{(k)}\|_{\mathbf{A}}}{\|\boldsymbol{e}^{(0)}\|_{\mathbf{A}}} \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k.$$

Remark 5.18 **The $\sqrt{\kappa}$ Advantage:** While stationary methods depend on $(\kappa-1)/(\kappa+1)$, CG depends on the **square root** of the condition number. For $\kappa = 100$, CG reduces error $5\times$ faster per step.

Remark 5.19 **Superlinear Convergence:** If eigenvalues are clustered, CG converges much faster than the worst-case bound suggests (e.g., $k \ll n$ for clustered spectra).

Remark 5.20 **(Polynomial view)** CG chooses a degree-k polynomial p_k with $p_k(0) = 1$ so that

$$\boldsymbol{e}^{(k)} = p_k(\mathbf{A})\boldsymbol{e}^{(0)}$$

is small in the $\mathbf{A}$-norm. Good convergence occurs when a low-degree polynomial can be small on the eigenvalues of $\mathbf{A}$.

5.3.6 Preconditioning

The convergence of CG depends on the condition number $\kappa(\mathbf{A})$. **Preconditioning** transforms the system using a symmetric positive definite matrix $\mathbf{M}$ that approximates $\mathbf{A}$ but is much easier to invert.

Remark 5.21 **Spectral Transformation:** The goal of the preconditioner is to cluster the eigenvalues of $\mathbf{M}^{-1}\mathbf{A}$ near 1. A perfectly preconditioned system ($\mathbf{M} = \mathbf{A}$) has all eigenvalues equal to 1 and converges in a single step. In practice, a good preconditioner reduces the effective condition number, significantly accelerating convergence.

Remark 5.22 **(Preconditioning is not optional in hard problems)** For large PDE systems, unpreconditioned CG can require too many iterations. The preconditioner is often the main algorithmic design choice.

Def 5.10 **(Common Preconditioners)**

1. **Jacobi:** $\mathbf{M} = \text{diag}(\mathbf{A})$. (Cheap, moderate speedup).
2. **Incomplete Cholesky (IC):** $\mathbf{M} \approx \mathbf{L}\mathbf{L}^T$ with sparse factors. (Robust).
3. **Multigrid:** Near $O(n)$ total complexity for PDE systems.

Ex 5.9

1. **Convergence Check:** Solve a 100×100 Poisson system via CG. Compare your error plot to the theoretical bound.
2. **Spectral Clustering:** Construct $\mathbf{A}$ with eigenvalues $\{1, \ldots, 1, 100, 200\}$. Observe superlinear convergence in the first few steps.
3. **Preconditioning:** Compare iteration counts for a sparse system with and without an IC preconditioner.

5.4 GMRES

The Generalized Minimal Residual (GMRES) method is the standard Krylov subspace solver for general (non-symmetric) linear systems $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. Unlike CG, it does not require $\mathbf{A}$ to be symmetric or positive definite.

CG relies on symmetry to get a short recurrence. GMRES works without symmetry by explicitly building an orthonormal Krylov basis and choosing the vector in that subspace with smallest residual. This makes GMRES broadly applicable, but more expensive in memory.

5.4.1 Residual Minimization and Krylov Subspaces

Remark 5.23 **The Minimization Problem:** At each step k, GMRES finds the approximate solution $\boldsymbol{x}_k$ in the affine space $\boldsymbol{x}_0 + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}_0)$ that minimizes the **Euclidean norm of the residual**:

$$\boldsymbol{x}_k = \arg \min_{\boldsymbol{x} \in \boldsymbol{x}_0 + \mathcal{K}_k} \|\boldsymbol{b} - \mathbf{A}\boldsymbol{x}\|_2.$$

Because the residual norm is minimized explicitly, it is guaranteed to be non-increasing with k.

Remark 5.24 **(CG vs. GMRES)**

- **CG:** SPD systems only; short recurrence; $O(n)$ memory.
- **GMRES:** General square systems; stores a growing basis; $O(kn)$ memory after k steps.

5.4.2 The Arnoldi Process

Remark 5.25 **Basis Construction:** To compute this minimum efficiently, GMRES uses the **Arnoldi process** to build an orthonormal basis $V_k = [\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k]$ for $\mathcal{K}_k$. This process is essentially a Gram-Schmidt orthogonalization of the Krylov sequence.

Arnoldi is the nonsymmetric analog of the Lanczos process. Each step applies $\mathbf{A}$ to the newest basis vector, then orthogonalizes the result against all previous basis vectors. The need to remember all previous basis vectors is the source of GMRES memory growth.

Def 5.11 **(Arnoldi Relation)** The k steps of the Arnoldi process yield the fundamental relation:

$$\mathbf{A}V_k = V_{k+1}\bar{H}_k,$$

where $\bar{H}_k$ is an $(k+1) \times k$ upper Hessenberg matrix. This relation expresses the action of $\mathbf{A}$ on the basis V_k in terms of the expanded basis V_{k+1}.

5.4.3 Reduction to Hessenberg Least Squares

The critical insight of GMRES is that the n-dimensional residual minimization problem can be projected onto the k-dimensional Krylov subspace, resulting in a much smaller problem.

Thm 5.9 **(Projected Least Squares)** Let $\boldsymbol{x}_k = \boldsymbol{x}_0 + V_k\boldsymbol{y}$ for some $\boldsymbol{y} \in \mathbb{R}^k$. The residual norm satisfies:

$$\|\boldsymbol{r}_k\|_2 = \|\beta\boldsymbol{e}_1 - \bar{H}_k\boldsymbol{y}\|_2,$$

where $\beta = \|\boldsymbol{r}_0\|_2$ and $\boldsymbol{e}_1$ is the first canonical basis vector. Solving the original problem thus reduces to solving a $(k+1) \times k$ least-squares problem for $\boldsymbol{y}$ using the small matrix $\bar{H}_k$.

Proof *Proof.* Let $\boldsymbol{r}_0 = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}_0$ and choose $\boldsymbol{v}_1 = \boldsymbol{r}_0/\|\boldsymbol{r}_0\|_2$, so $\boldsymbol{r}_0 = \beta V_{k+1}\boldsymbol{e}_1$. If $\boldsymbol{x}_k = \boldsymbol{x}_0 + V_k\boldsymbol{y}$, then

$$\begin{aligned}\boldsymbol{r}_k &= \boldsymbol{b} - \mathbf{A}(\boldsymbol{x}_0 + V_k\boldsymbol{y}) \\ &= \boldsymbol{r}_0 - \mathbf{A}V_k\boldsymbol{y} \\ &= V_{k+1}\left(\beta\boldsymbol{e}_1 - \bar{H}_k\boldsymbol{y}\right),\end{aligned}$$

using the Arnoldi relation $\mathbf{A}V_k = V_{k+1}\bar{H}_k$. Since V_{k+1} has orthonormal columns, it preserves the Euclidean norm, giving

$$\|\boldsymbol{r}_k\|_2 = \|\beta\boldsymbol{e}_1 - \bar{H}_k\boldsymbol{y}\|_2.$$

□

Remark 5.26 **Optimality:** By definition, GMRES finds the absolute best solution available in the growing Krylov subspace. The residual norm is non-increasing.

5.4.4 Restarted GMRES(m)

Remark 5.27 **The Memory Wall:** Full GMRES storage costs $O(kn)$ and work grows quadratically with k (due to orthogonalization). **Restarting** after m steps prevents memory explosion but can lead to stagnation.

Remark 5.28 **(Restart trade-off)** GMRES(m) discards the Krylov basis every m steps and starts again from the current residual. This controls memory, but it also discards spectral information. Small restart values can cause long plateaus in convergence.

5.4.5 Convergence and Preconditioning

Thm 5.10 **(Approximation Bound)**

$$\frac{\|\boldsymbol{r}_k\|_2}{\|\boldsymbol{r}_0\|_2} \leq \min_{p \in \mathcal{P}_k, p(0)=1} \max_{\lambda \in \sigma(\mathbf{A})} |p(\lambda)|.$$

Remark 5.29 **(Convergence is less predictable than CG)** For nonnormal matrices, eigenvalues alone may not explain GMRES behavior. The geometry of eigenvectors and transient growth can matter. In practice, preconditioning is usually essential.

Def 5.12 **(Preconditioning)**

- **Left:** $\mathbf{M}^{-1}\mathbf{A}\boldsymbol{x} = \mathbf{M}^{-1}\boldsymbol{b}$. (Minimizes preconditioned residual).
- **Right:** $\mathbf{A}\mathbf{M}^{-1}(\mathbf{M}\boldsymbol{x}) = \boldsymbol{b}$. (Minimizes true residual; standard choice).

Remark 5.30 **(Breakdown and stagnation)** Arnoldi breakdown can be good: if the Krylov subspace becomes invariant and the residual reaches zero, GMRES has found the exact solution. Stagnation is different: the method keeps iterating but the residual barely decreases, often because the restart length or preconditioner is poor.

Ex 5.10

1. **GMRES vs. CG:** Compare residual plots for a symmetric system. Why is CG usually faster?
2. **Restarting:** Test GMRES(m) on a non-symmetric system for $m = 5, 20, 50$. Observe how small m can cause "plateaus" in convergence.
3. **Arnoldi:** Implement the Arnoldi process via Modified Gram-Schmidt. Verify the Hessenberg structure of $H_k = V_k^T \mathbf{A} V_k$.

6 Numerical Analysis

6.1 Polynomial Interpolation

Constructing a polynomial that passes exactly through a given set of data points.

6.1.1 Existence and Uniqueness

Thm 6.1 **(Existence and Uniqueness)** Given $n+1$ distinct points (x_i, y_i), there exists a unique polynomial $P_n(x)$ of degree at most n such that $P_n(x_i) = y_i$ for all $i = 0, \ldots, n$.

Ex 6.1

1. Count the number of coefficients in a degree-n polynomial.
2. Write the interpolation equations as a Vandermonde linear system.
3. Use distinctness of the nodes to explain why the Vandermonde matrix is nonsingular.
4. Conclude Thm 6.1.

6.1.2 Lagrange Form

Provides an explicit construction for the interpolating polynomial.

Def 6.1 **(Lagrange Basis Polynomials)**

$$L_i(x) = \prod_{j=0, j\neq i}^{n} \frac{x - x_j}{x_i - x_j}.$$

The interpolating polynomial is $P_n(x) = \sum_{i=0}^{n} y_i L_i(x)$.

Ex 6.2

1. Show that $L_i(x_j) = 0$ when $i \neq j$.
2. Show that $L_i(x_i) = 1$.
3. Use the previous two steps to prove that $P_n(x_j) = y_j$ for every node.
4. Explain why uniqueness follows from Thm 6.1.

Remark 6.1 **Computational Cost:** Evaluation of the Lagrange form costs $O(n^2)$ operations. Adding a new point requires recomputing all basis polynomials.

6.1.3 Barycentric Interpolation

A numerically stable and efficient refinement of the Lagrange form.

Def 6.2 **(Barycentric Formula)**

$$P_n(x) = \frac{\sum_{i=0}^{n} \frac{w_i}{x-x_i} y_i}{\sum_{i=0}^{n} \frac{w_i}{x-x_i}}, \quad \text{where } w_i = \frac{1}{\prod_{j \neq i}(x_i - x_j)}.$$

Remark 6.2 **Advantages:** Evaluation costs only $O(n)$ once the weights w_i are computed ($O(n^2)$). This is the standard recommended approach for general polynomial interpolation.

6.1.4 Interpolation Error

Thm 6.2 **(Polynomial Interpolation Error)** Let P_n interpolate f at distinct nodes $x_0, \ldots, x_n$. If f has $n+1$ continuous derivatives, then for each x there exists ξ_x in the interval containing the nodes and x such that

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{j=0}^{n} (x - x_j).$$

Ex 6.3

1. Define $\omega(x) = \prod_{j=0}^{n}(x - x_j)$.
2. For a fixed x, construct $g(t) = f(t) - P_n(t) - c\omega(t)$ so that $g(x) = 0$.
3. Count the zeros of g.
4. Apply Rolle's theorem $n + 1$ times.
5. Solve for c and derive Thm 6.2.

6.1.5 Runge's Phenomenon

Equally spaced points are often a poor choice for high-degree interpolation.

Remark 6.3 **The Runge Phenomenon:** For certain functions, such as $f(x) = 1/(1+25x^2)$ on $[-1, 1]$, the interpolation error near the boundaries increases exponentially as the degree $n \to \infty$.

Thm 6.3 **(Chebyshev Nodes for Interpolation)** The Chebyshev nodes from Def 3.22,

$$x_i = \cos\left(\frac{2i+1}{2n+2}\pi\right), \quad i = 0, \ldots, n.$$

make the interpolation error polynomial small on $[-1, 1]$. By Thm 3.12 and Thm 3.15, this directly improves the worst-case interpolation error bound.

Ex 6.4

1. Translate the node formula above into the indexing convention of Def 3.22.
2. Use Thm 3.15 to compute $\|\omega_{n+1}\|_\infty$ for these nodes.
3. Compare this with the error polynomial for equally spaced nodes numerically for $n = 10, 20, 40$.
4. Explain why Chebyshev nodes reduce Runge oscillations without claiming that every function converges uniformly under every interpolation scheme.

6.1.6 Cubic Splines

For large datasets, high-degree polynomial interpolation is avoided in favor of piecewise polynomials.

Def 6.3 **(Cubic Spline)** A piecewise cubic function $S(x)$ that is C^2 continuous (continuous values, first, and second derivatives) at the internal nodes.

Remark 6.4 **(Spline linear system)** Computing the coefficients of a cubic spline requires solving a sparse tridiagonal linear system, which can be done in $O(n)$ time.

6.1.7 Exercises

Ex 6.5

1. Construct the Lagrange form for points $(-1, 1), (0, 0), (1, 1)$. Verify it is $P_2(x) = x^2$.
2. Use the barycentric formula to interpolate $f(x) = \sin(x)$ at 5 Chebyshev nodes.
3. Compare the error plots for $f(x) = 1/(1 + 25x^2)$ using 15 equally spaced nodes vs. 15 Chebyshev nodes.

6.2 Numerical Quadrature

Approximating $I = \int_a^b f(x)\,dx$ via weighted sums: $\sum_{i=1}^n w_i f(x_i)$.

Def 6.4 **(Quadrature Rule)** A quadrature rule approximates an integral by function values:

$$\int_a^b f(x)\,dx \approx \sum_{i=1}^{p} w_i f(x_i).$$

The nodes x_i choose where f is sampled, and the weights w_i choose how those samples are averaged.

Def 6.5 **(Degree of Exactness)** A quadrature rule has degree of exactness m if it integrates every polynomial of degree at most m exactly, and fails for at least one polynomial of degree $m+1$.

Remark 6.5 **(Interpolation view)** Newton-Cotes rules interpolate f at selected nodes and integrate the interpolating polynomial. The quadrature error is therefore controlled by interpolation error and smoothness of f.

6.2.1 Newton-Cotes Rules

Uses equally-spaced nodes. Higher-order rules are prone to Runge's phenomenon.

Def 6.6 **(Trapezoidal Rule)** On n panels of width $h = (b-a)/n$:

$$T_n = h\left[\frac{f(a)+f(b)}{2} + \sum_{i=1}^{n-1} f(a+ih)\right].$$

Prop 6.4 **(Trapezoid Exactness)** The one-panel trapezoidal rule is exact for polynomials of degree at most 1.

Proof *Proof.* On $[a,b]$, the trapezoidal rule integrates the line through $(a,f(a))$ and $(b,f(b))$. If f itself is constant or linear, this interpolant equals f, so the integral is exact. □

Thm 6.5 **(Trapezoidal Error)** If f has two continuous derivatives on $[a,b]$, then the composite trapezoidal rule satisfies

$$I - T_n = O(h^2).$$

Thus doubling n reduces the leading error by a factor of about 4.

Ex 6.6

1. Derive the one-panel trapezoidal rule by integrating the linear interpolant through $(a,f(a))$ and $(b,f(b))$.
2. Use Taylor expansion on one panel to show the local error is $O(h^3)$.
3. Sum over $O(1/h)$ panels to prove the global $O(h^2)$ statement in Thm 6.5.

Def 6.7 **(Simpson's Rule)** Requires n to be even. Uses piecewise quadratic interpolation:

$$S_n = \frac{h}{3}\left[f(a)+f(b)+4\sum_{\text{odd } i} f(x_i) + 2\sum_{\text{even } i} f(x_i)\right].$$

Thm 6.6 **(Simpson Exactness and Error)** Simpson's rule is exact for polynomials of degree at most 3. For sufficiently smooth f, the composite Simpson rule has global error $O(h^4)$.

Remark 6.6 **(Degree effect)** Simpson's rule uses a quadratic interpolant, but symmetry cancels the cubic error term. This is why its exactness degree is 3, not merely 2.

Ex 6.7

1. Integrate the quadratic interpolant through x_0, x_1, x_2 to derive the Simpson weights $1, 4, 1$.
2. Verify exactness for $1, x, x^2, x^3$ on a symmetric panel.
3. Use the exactness degree to explain why the leading error involves the fourth derivative.
4. Compare the observed error reduction after halving h with Thm 6.6.

6.2.2 Gaussian Quadrature

Optimizes **both** nodes and weights to achieve the highest possible degree of exactness.

Thm 6.7 **(Gauss-Legendre Quadrature)** Integrating over $[-1, 1]$ with p points:

$$\int_{-1}^{1} f(x)\,dx \approx \sum_{i=1}^{p} w_i f(x_i).$$

The nodes are the roots of the degree-p Legendre polynomial $P_p(x)$, and the rule is exact for all polynomials of degree at most $2p - 1$.

Remark 6.7 **(Gauss efficiency)** Gaussian quadrature spends nodes where orthogonality says they are most useful. For smooth, non-periodic integrands, a small number of Gauss points often beats many equally spaced Newton-Cotes points.

Ex 6.8

1. For $p = 2$, use the roots of $P_2(x) = (3x^2 - 1)/2$ to find the nodes.
2. Solve for weights that integrate 1 and x^2 exactly on $[-1, 1]$.
3. Verify that the resulting rule also integrates x and x^3 exactly by symmetry.
4. Explain why exactness through degree $2p - 1$ is the maximum possible with $2p$ free parameters.

6.2.3 Richardson Extrapolation and Romberg Integration

Thm 6.8 **(Richardson Extrapolation)** Given an $O(h^2)$ estimate $A(h)$, build an $O(h^4)$ estimate by:

$$A_{new} = \frac{4A(h/2) - A(h)}{3}.$$

Proof *Proof.* Let $A(h)$ be an $O(h^2)$ approximation of the true value I. Using a Taylor expansion in h:

$$A(h) = I + C_1 h^2 + C_2 h^4 + O(h^6).$$

Halving the step size gives:

$$\begin{aligned} A(h/2) &= I + C_1(h/2)^2 + C_2(h/2)^4 + O(h^6) \\ &= I + \frac{1}{4}C_1 h^2 + \frac{1}{16}C_2 h^4 + O(h^6). \end{aligned}$$

To eliminate the $O(h^2)$ error term, multiply $A(h/2)$ by 4 and subtract $A(h)$:

$$\begin{aligned} 4A(h/2) - A(h) &= (4I + C_1 h^2 + \frac{1}{4}C_2 h^4) - (I + C_1 h^2 + C_2 h^4) + O(h^6) \\ &= 3I - \frac{3}{4}C_2 h^4 + O(h^6). \end{aligned}$$

Dividing by 3 yields the $O(h^4)$ estimate:

$$I = \frac{4A(h/2) - A(h)}{3} + O(h^4).$$

□

Def 6.8 **(Romberg Integration)** Iterative application of Richardson extrapolation to the trapezoidal rule.

Remark 6.8 **(Romberg idea)** Romberg integration repeatedly cancels the leading even powers in the trapezoidal error expansion. It is effective when f is smooth enough for that expansion to be accurate.

6.2.4 Adaptive Quadrature

Remark 6.9 **(Adaptive quadrature)** Adaptive quadrature focuses function evaluations where f is difficult and uses large panels where f is smooth. The usual mechanism compares one large panel with two smaller panels; if the difference is above tolerance ε, the method recurses.

Remark 6.10 **(Nonsmooth integrands)** Algebraic error rates assume enough derivatives. Corners, jumps, and endpoint singularities reduce the observed order. Adaptive quadrature helps by isolating nonsmooth regions, but it cannot restore smoothness.

Remark 6.11 **(Monte Carlo contrast)** Deterministic quadrature is efficient in low dimension for smooth functions. Monte Carlo convergence is only $O(N^{-1/2})$, but its rate is largely dimension-independent, which makes it useful for high-dimensional integrals.

6.2.5 Periodic Functions and Spectral Accuracy

Thm 6.9 **(Periodic Trapezoidal Spectral Accuracy)** For smooth periodic functions on $[0, 2\pi]$, the trapezoidal rule can converge faster than any algebraic power of h. For analytic periodic functions, the convergence is exponential.

Ex 6.9

1. Apply the trapezoidal rule to e^{ikx} on $[0, 2\pi]$ and determine when the discrete sum is exactly zero.
2. Use a Fourier series argument to explain why smooth periodic functions are integrated unusually well.
3. Compare errors for $\sin(x)$, $\exp(\cos x)$, and $|x - \pi|$ as n increases.

6.2.6 Exercises

Ex 6.10

1. Doubling nodes in Trapezoidal rule reduces error by $4\times$. Use Thm 6.6 to predict Simpson's reduction factor.
2. Derive the $p = 2$ Gauss points for $[-1, 1]$ using Ex 6.8.
3. Use Richardson extrapolation from Thm 6.8 to improve $T(h)$ and $T(h/2)$ to get Simpson's rule.
4. Integration of $\sin(x)$ on $[0, 2\pi]$ with $n = 4$ nodes. Is the error 0? Why?
5. Compare composite Simpson on $f(x) = |x - 1/2|$ with a smooth quartic. Estimate the observed order.

6.3 Root Finding

Iterative methods for solving $f(x) = 0$ for continuous $f : \mathbb{R} \to \mathbb{R}$.

Def 6.9 **(Stopping Tests for Root Finding)** Common stopping tests are

$$|f(x_k)| \le \texttt{tol}_f, \qquad |x_{k+1} - x_k| \le \texttt{tol}_x(1 + |x_k|).$$

A small residual alone can be misleading when $f'(x^*)$ is small. A small step alone can be misleading when the iteration stagnates.

6.3.1 Bracketing Methods

Guaranteed convergence to a root within a known interval.

Thm 6.10 **(Bisection Method)** Given $f \in C[a, b]$ with $f(a)f(b) < 0$, a root $x^* \in (a, b)$ exists. The method repeatedly halves the interval:

1. $m = (a + b)/2$
2. If $f(a)f(m) < 0$, new interval is $[a, m]$, else $[m, b]$.

Error Bound: After n steps, $|x_n - x^*| \le (b - a)/2^n$.

Ex 6.11

1. Use the intermediate value theorem to prove that the initial bracket contains a root.
2. Show that the bisection update preserves the sign-change bracket.
3. Prove that the bracket length after n steps is $(b - a)/2^n$.
4. Derive the error bound in Thm 6.10.

Remark 6.12 **(Bisection trade-off)** Bisection is robust because it preserves a bracket, but its convergence is only linear. Each step gains exactly one bit of accuracy.

Remark 6.13 **(Bracketing default)** If a sign-changing interval is known and function evaluations are cheap, use a bracketed method. It gives a certificate that a root remains inside the current interval.

6.3.2 Open Methods

Faster local convergence, but sensitive to the initial guess x_0 and may diverge.

Def 6.10 **(Fixed-Point Iteration)** A root problem can be rewritten as $x = g(x)$ and iterated by

$$x_{k+1} = g(x_k).$$

If $|g'(x^*)| < 1$, the fixed point is locally attracting.

Ex 6.12

1. Use the mean value theorem to show $|x_{k+1} - x^*| \le L|x_k - x^*|$ if $|g'(x)| \le L < 1$ near x^*.
2. Rewrite Newton's method as a fixed-point iteration.
3. Compute $g'(x^*)$ for Newton's fixed-point map at a simple root.

Def 6.11 **(Newton's Method)** Linearizes f at the current iterate:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Thm 6.11 **(Quadratic Convergence)** If $f''(x)$ is continuous near a simple root x^* ($f'(x^*) \neq 0$), Newton's method converges quadratically:

$$|x_{k+1} - x^*| \approx \frac{|f''(x^*)|}{2|f'(x^*)|}|x_k - x^*|^2.$$

Ex 6.13

1. Expand $f(x^*)$ about x_k using a second-order Taylor formula.
2. Divide by $f'(x_k)$ and isolate the Newton correction $f(x_k)/f'(x_k)$.
3. Substitute $x_{k+1} = x_k - f(x_k)/f'(x_k)$.
4. Show that $|x_{k+1} - x^*| \le C|x_k - x^*|^2$ once x_k is close enough to a simple root.
5. Explain why the assumption $f'(x^*) \neq 0$ is essential.

Remark 6.14 **(Newton failure modes)** 1. x_0 too far from x^*. 2. $f'(x_k) \approx 0$ (zero derivative). 3. Multiple roots ($f'(x^*) = 0$): convergence degrades to linear with rate $1/2$.

Thm 6.12 **(Newton on a Multiple Root)** If $f(x) = (x - x^*)^m q(x)$ with $q(x^*) \neq 0$ and $m > 1$, then Newton's method converges locally linearly with asymptotic factor

$$1 - \frac{1}{m}.$$

Ex 6.14

1. Compute $f'(x)$ for $f(x) = (x - x^*)^m q(x)$.
2. Substitute f and f' into the Newton update.

3. Keep the leading term as $x_k \to x^*$.

4. Derive the factor in Thm 6.12.

Def 6.12 **(Secant Method)** Approximates f' via finite differences of the last two iterates:

$$x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Requires two initial points (x_0, x_1).

Thm 6.13 **(Superlinear Convergence)** The secant method converges with order $\phi = (1+\sqrt{5})/2 \approx 1.618$ (the golden ratio). **Efficiency:** Faster than bisection, slower than Newton, but requires only 1 function evaluation per step (Newton requires 2: f and f').

Ex 6.15

1. Derive the secant update by replacing $f'(x_k)$ in Newton's method with a finite difference through $(x_{k-1}, f(x_{k-1}))$ and $(x_k, f(x_k))$.

2. Compare the information used by Newton and secant.

3. Numerically estimate the convergence order for $f(x) = x^2 - 2$.

6.3.3 Brent's Method

A standard robust algorithm for one-dimensional root-finding.

Thm 6.14 **(The Hybrid Approach)** Brent's method combines:

1. **Bisection:** For guaranteed global convergence.

2. **Secant:** For fast local convergence.

3. **IQI:** Inverse Quadratic Interpolation using 3 points.

It uses fast open steps by default, falling back to bisection only if the step is outside the bracket or not improving quickly enough.

Remark 6.15 **Implementation:** Used in `scipy.optimize.brentq`.

Def 6.13 **(Method Selection)**

Available information	Typical method
Sign-changing bracket	Bisection or Brent
Good initial guess and derivative	Newton
Good initial guesses, no derivative	Secant
Need robust scalar default	Brent

6.3.4 Exercises

Ex 6.16

1. Bisection on $f(x) = x^3 - 2$ on $[1, 2]$ for 4 steps. Find the guaranteed error using Thm 6.10.
2. Newton on $f(x) = x^2 - 2$ from $x_0 = 1$. Verify the doubling of digits using Thm 6.11.
3. Prove that for $f(x) = x^2$, Newton converges to 0 with linear rate $1/2$ using Thm 6.12.
4. Find the root of $e^x = 3x$ near $x = 1$ via Secant method. Compare evaluations vs. Newton.
5. Construct an example where $|f(x_k)|$ is small but $|x_k - x^*|$ is not small. Relate it to Def 6.9.

7 Ordinary Differential Equations

7.1 Numerical Methods for ODEs

An ordinary differential equation prescribes the derivative of an unknown function. A time stepping method replaces the exact flow by a sequence of computable updates. The numerical questions are accuracy, stability, stiffness, and cost per step.

7.1.1 Initial Value Problems

Def 7.1 **(Initial Value Problem)** An initial value problem (IVP) has the form

$$\boldsymbol{y}'(t) = \boldsymbol{f}(t, \boldsymbol{y}(t)), \qquad \boldsymbol{y}(t_0) = \boldsymbol{y}_0,$$

where $\boldsymbol{y}(t) \in \mathbb{R}^m$. The scalar case is $m = 1$.

Remark 7.1 **(IVP vs. BVP)** In an IVP, all data are given at one time. In a boundary value problem, conditions are imposed at different points, such as $u(0) = u(1) = 0$. IVPs are naturally marched forward; BVPs usually lead to global linear or nonlinear systems.

Def 7.2 **(Flow Map)** For the IVP in Def 7.1, the exact flow map φ_h satisfies

$$\boldsymbol{y}(t + h) = \varphi_h(t, \boldsymbol{y}(t)).$$

A one-step method replaces φ_h by a computable map Φ_h:

$$\boldsymbol{y}_{n+1} = \Phi_h(t_n, \boldsymbol{y}_n), \qquad t_n = t_0 + nh.$$

Def 7.3 **(Autonomous System)** An ODE is autonomous if $\boldsymbol{f}$ does not depend explicitly on time:

$$\boldsymbol{y}' = \boldsymbol{f}(\boldsymbol{y}).$$

Non-autonomous systems can be made autonomous by adding $t' = 1$ as one more equation.

Thm 7.1 **(Existence and Uniqueness Condition)** If $\boldsymbol{f}(t, \boldsymbol{y})$ is continuous in t and Lipschitz continuous in $\boldsymbol{y}$ on a neighborhood of $(t_0, \boldsymbol{y}_0)$, then the IVP in Def 7.1 has a unique local solution.

Remark 7.2 **(Role of Lipschitz continuity)** Numerical convergence is meaningful only when the exact IVP is well posed. If nearby initial data can lead to different solution branches, small rounding or truncation errors may not remain small.

Ex 7.1

1. Show that $y' = 2\sqrt{|y|}$, $y(0) = 0$, has more than one solution.
2. Identify which hypothesis of Thm 7.1 fails.
3. Explain why a time stepping method cannot be expected to select a unique physical branch without extra information.

7.1.2 Error and Convergence

Def 7.4 **(Local Truncation Error)** For a one-step method $\boldsymbol{y}_{n+1} = \Phi_h(t_n, \boldsymbol{y}_n)$, the local truncation error is the one-step defect made from exact data:

$$\boldsymbol{\tau}_{n+1} = \frac{\boldsymbol{y}(t_{n+1}) - \Phi_h(t_n, \boldsymbol{y}(t_n))}{h}.$$

The method is consistent if $\|\boldsymbol{\tau}_{n+1}\| \to 0$ as $h \to 0$.

Def 7.5 **(Global Error and Order)** The global error at t_n is

$$\boldsymbol{e}_n = \boldsymbol{y}(t_n) - \boldsymbol{y}_n.$$

A method has order p on a fixed interval $[t_0, T]$ if

$$\max_{0 \le n \le N} \|\boldsymbol{e}_n\| = O(h^p), \qquad Nh = T - t_0.$$

Thm 7.2 **(Local to Global Error)** For a stable one-step method, a one-step defect $O(h^{p+1})$ gives global error $O(h^p)$ on a fixed time interval.

Ex 7.2

1. Suppose each step contributes a local error of size $O(h^{p+1})$.
2. Count the number of steps needed to reach a fixed final time.
3. Explain why this gives $O(h^p)$ global error when previous errors are not amplified strongly.
4. State where stability enters this argument.

Def 7.6 **(Consistency, Stability, Convergence)** For time stepping methods, consistency means the discrete update matches the differential equation as $h \to 0$. Stability means perturbations introduced at one step do not grow uncontrollably over a fixed interval. Convergence means $\boldsymbol{y}_n \to \boldsymbol{y}(t_n)$ as $h \to 0$.

Thm 7.3 **(ODE Lax Principle)** For a consistent one-step method applied to a Lipschitz IVP, stability on a fixed interval implies convergence. For linear multistep methods, consistency plus zero-stability implies convergence.

Ex 7.3

1. Write the error recurrence for a one-step method by subtracting the numerical update from the exact one-step update.
2. Bound the propagated error using a Lipschitz constant for $\boldsymbol{f}$.
3. Add the local defect term from Def 7.4.
4. Apply a discrete Gronwall argument to justify the first statement in Thm 7.3.

7.1.3 Euler Methods

Def 7.7 **(Forward Euler)** The explicit Euler method uses the slope at the left endpoint:

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + h\boldsymbol{f}(t_n, \boldsymbol{y}_n).$$

It is explicit because the right side is known before step $n+1$ is computed.

Thm 7.4 **(Forward Euler Error)** Forward Euler has local truncation error $O(h)$ in the sense of Def 7.4, one-step defect $O(h^2)$, and global error $O(h)$ for smooth solutions on fixed intervals.

Ex 7.4

1. Taylor expand the exact solution:

$$\boldsymbol{y}(t_{n+1}) = \boldsymbol{y}(t_n) + h\boldsymbol{y}'(t_n) + \frac{h^2}{2}\boldsymbol{y}''(\xi_n).$$

2. Substitute $\boldsymbol{y}'(t_n) = \boldsymbol{f}(t_n, \boldsymbol{y}(t_n))$.
3. Compare with Def 7.7.
4. Use Thm 7.2 to obtain the global order.

Example **(Forward Euler on decay)** For $y' = -y$, $y(0) = 1$, Forward Euler gives

$$y_n = (1 - h)^n.$$

At $t = 1$ with $h = 1/4$, this gives $(3/4)^4 = 81/256 \approx 0.3164$, while the exact value is $e^{-1} \approx 0.3679$.

Def 7.8 **(Backward Euler)** The implicit Euler method uses the slope at the right endpoint:

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + h\boldsymbol{f}(t_{n+1}, \boldsymbol{y}_{n+1}).$$

At each step one solves a nonlinear equation for $\boldsymbol{y}_{n+1}$.

Def 7.9 **(Implicit Step Equation)** For Backward Euler, define

$$\boldsymbol{g}(\mathbf{Y}) = \mathbf{Y} - \boldsymbol{y}_n - h\boldsymbol{f}(t_{n+1}, \mathbf{Y}).$$

The next step is the root $\boldsymbol{g}(\mathbf{Y}) = \mathbf{0}$. Newton's method requires linear systems with

$$\mathbf{I} - h\mathbf{J}_f(t_{n+1}, \mathbf{Y}),$$

where $\mathbf{J}_f$ is the Jacobian of $\boldsymbol{f}$ with respect to $\boldsymbol{y}$.

Thm 7.5 **(Backward Euler Error)** Backward Euler is first order: its one-step defect is $O(h^2)$ and its global error is $O(h)$ for smooth solutions.

Ex 7.5

1. Taylor expand $\boldsymbol{y}(t_n)$ about t_{n+1}.
2. Rearrange the expansion to express $\boldsymbol{y}(t_{n+1})$ in terms of $\boldsymbol{y}(t_n)$ and $\boldsymbol{y}'(t_{n+1})$.
3. Substitute $\boldsymbol{y}'(t_{n+1}) = \boldsymbol{f}(t_{n+1}, \boldsymbol{y}(t_{n+1}))$.
4. Compare with Def 7.8.

Def 7.10 **(Trapezoidal Method)** The implicit trapezoidal method averages endpoint slopes:

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \frac{h}{2}\left[\boldsymbol{f}(t_n, \boldsymbol{y}_n) + \boldsymbol{f}(t_{n+1}, \boldsymbol{y}_{n+1})\right].$$

It is also called Crank-Nicolson when used for time-dependent PDE discretizations.

Thm 7.6 **(Trapezoidal Order)** The trapezoidal method has one-step defect $O(h^3)$ and global error $O(h^2)$ for smooth solutions.

Ex 7.6

1. Write the exact integral identity
$$\boldsymbol{y}(t_{n+1}) = \boldsymbol{y}(t_n) + \int_{t_n}^{t_{n+1}} \boldsymbol{f}(t, \boldsymbol{y}(t))\, dt.$$
2. Approximate the integral by the scalar trapezoidal rule from Def 6.6.
3. Use Thm 6.5 to explain the one-step defect.
4. Convert the one-step defect to global order using Thm 7.2.

7.1.4 Absolute Stability and Stiffness

Def 7.11 **(Test Equation and Stability Function)** The scalar test equation is
$$y' = \lambda y, \qquad \mathrm{Re}(\lambda) < 0.$$
For a one-step method applied to this equation, the update has the form
$$y_{n+1} = R(z) y_n, \qquad z = h\lambda.$$
The function R is the stability function.

Def 7.12 **(Absolute Stability Region)** The absolute stability region of a one-step method is
$$\mathcal{S} = \{z \in \mathbb{C} : \ |R(z)| \le 1\}.$$
The method is stable on the test equation when $h\lambda \in \mathcal{S}$.

Thm 7.7 **(Euler Stability Functions)** For the test equation in Def 7.11,
$$R_{\mathrm{FE}}(z) = 1 + z, \qquad R_{\mathrm{BE}}(z) = \frac{1}{1 - z}, \qquad R_{\mathrm{TR}}(z) = \frac{1 + z/2}{1 - z/2}.$$
Forward Euler is stable in the disk $|1 + z| \le 1$. Backward Euler and trapezoidal rule are A-stable.

Proof *Proof.* For Forward Euler, $y_{n+1} = y_n + h\lambda y_n = (1 + z) y_n$. For Backward Euler,
$$y_{n+1} = y_n + h\lambda y_{n+1} \quad \Rightarrow \quad y_{n+1} = \frac{1}{1 - z} y_n.$$
For trapezoidal rule,
$$y_{n+1} = y_n + \frac{z}{2}(y_n + y_{n+1}) \quad \Rightarrow \quad y_{n+1} = \frac{1 + z/2}{1 - z/2} y_n.$$
If $\mathrm{Re}(z) \le 0$, then $|1| \le |1 - z|$ and $|1 + z/2| \le |1 - z/2|$, so Backward Euler and trapezoidal rule are A-stable. □

Ex 7.7

1. Draw the Forward Euler stability disk from $|1+z| \le 1$.
2. For real $\lambda < 0$, show that Forward Euler requires $h \le 2/|\lambda|$.
3. Verify $|1 + z/2| \le |1 - z/2|$ when $\mathrm{Re}(z) \le 0$ by squaring both sides.
4. Explain why A-stability does not imply L-stability.

Def 7.13 **(A-Stability and L-Stability)** A method is A-stable if its stability region contains the full left half-plane:

$$\{z \in \mathbb{C} : \mathrm{Re}(z) \le 0\} \subseteq \mathcal{S}.$$

An A-stable method is L-stable if additionally $R(z) \to 0$ as $z \to -\infty$ along the real axis.

Remark 7.3 **(Damping)** Backward Euler is L-stable, so very fast decaying modes are strongly damped. The trapezoidal method is A-stable but not L-stable, since $R_{\mathrm{TR}}(z) \to -1$ as $z \to -\infty$.

Def 7.14 **(Stiffness)** An IVP is stiff when stability forces an explicit method to take much smaller steps than accuracy requires. For a linear system

$$\boldsymbol{y}' = \mathbf{A}\boldsymbol{y},$$

the eigenvalues of $\mathbf{A}$ determine the explicit stability restriction through $h\lambda_i \in \mathcal{S}$.

Example **(Fast decay with slow forcing)** The equation

$$y' = -1000(y - \cos t) - \sin t$$

has exact solution $y(t) = \cos t$ for consistent initial data. The solution varies on an $O(1)$ time scale, but Forward Euler is restricted by the fast stable mode near $\lambda = -1000$ and requires $h \lesssim 0.002$.

Ex 7.8

1. Verify directly that $y(t) = \cos t$ solves the stiff example above.
2. Linearize the equation around the exact solution and identify the error equation.
3. Apply Thm 7.7 to obtain the Forward Euler stability restriction.
4. Explain why this restriction is unrelated to the visible time scale of $\cos t$.

7.1.5 Runge-Kutta Methods

Def 7.15 **(Explicit Runge-Kutta Method)** An s-stage explicit Runge-Kutta method has stages

$$\boldsymbol{k}_i = \boldsymbol{f}\left(t_n + c_i h,\, \boldsymbol{y}_n + h\sum_{j<i} a_{ij}\boldsymbol{k}_j\right), \qquad i = 1, \dots, s,$$

and update

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + h\sum_{i=1}^{s} b_i \boldsymbol{k}_i.$$

The coefficients are collected in a Butcher tableau.

Def 7.16 **(Explicit Midpoint Method)** The two-stage explicit midpoint method is

$$\begin{aligned} \boldsymbol{k}_1 &= \boldsymbol{f}(t_n, \boldsymbol{y}_n), \\ \boldsymbol{k}_2 &= \boldsymbol{f}(t_n + h/2, \boldsymbol{y}_n + h\boldsymbol{k}_1/2), \\ \boldsymbol{y}_{n+1} &= \boldsymbol{y}_n + h\boldsymbol{k}_2. \end{aligned}$$

It has global order 2.

Ex 7.9

1. Taylor expand the exact solution to second order.
2. Taylor expand $\boldsymbol{f}(t_n + h/2, \boldsymbol{y}_n + h\boldsymbol{k}_1/2)$ about $(t_n, \boldsymbol{y}_n)$.
3. Match terms to show second-order consistency.
4. Derive the stability function of explicit midpoint on $y' = \lambda y$.

Def 7.17 **(RK4)** The classical fourth-order Runge-Kutta method is

$$\begin{aligned} \boldsymbol{k}_1 &= \boldsymbol{f}(t_n, \boldsymbol{y}_n), \\ \boldsymbol{k}_2 &= \boldsymbol{f}(t_n + h/2, \boldsymbol{y}_n + h\boldsymbol{k}_1/2), \\ \boldsymbol{k}_3 &= \boldsymbol{f}(t_n + h/2, \boldsymbol{y}_n + h\boldsymbol{k}_2/2), \\ \boldsymbol{k}_4 &= \boldsymbol{f}(t_n + h, \boldsymbol{y}_n + h\boldsymbol{k}_3), \\ \boldsymbol{y}_{n+1} &= \boldsymbol{y}_n + \frac{h}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4). \end{aligned}$$

It has one-step defect $O(h^5)$ and global error $O(h^4)$ for smooth nonstiff problems.

Thm 7.8 **(RK4 Stability Polynomial)** Applied to $y' = \lambda y$, RK4 has stability function

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}.$$

This is the degree-four Taylor polynomial of e^z.

Ex 7.10

1. Apply Def 7.17 to $y' = \lambda y$.
2. Express each stage in terms of $z = h\lambda$ and y_n.
3. Substitute the stages into the RK4 update.
4. Derive the stability polynomial in Thm 7.8.

Remark 7.4 **(High order is not stiffness control)** RK4 has higher accuracy than Forward Euler on nonstiff problems, but its stability region is still bounded. For stiff decay, an explicit high-order method can remain limited by stability rather than accuracy.

7.1.6 Adaptive Time Stepping

Def 7.18 **(Embedded Runge-Kutta Pair)** An embedded pair computes two approximations of different orders from shared stages:

$$\boldsymbol{y}_{n+1}^{[p]}, \qquad \boldsymbol{y}_{n+1}^{[p+1]}.$$

The difference

$$\boldsymbol{e}_{n+1} = \boldsymbol{y}_{n+1}^{[p+1]} - \boldsymbol{y}_{n+1}^{[p]}$$

estimates the local error of the lower-order method.

Def 7.19 **(Weighted Error Test)** Adaptive solvers usually accept a step when

$$\left\| \frac{\boldsymbol{e}_{n+1}}{\text{atol} + \text{rtol}\,|\boldsymbol{y}_{n+1}|} \right\| \le 1,$$

where the absolute value and division are componentwise.

Thm 7.9 **(Adaptive Step Formula)** If the local error estimate satisfies $\|\boldsymbol{e}\| \approx Ch^{p+1}$, then a step size targeting tolerance ε is

$$h_{\text{new}} = s\, h_{\text{old}} \left(\frac{\varepsilon}{\|\boldsymbol{e}\|} \right)^{1/(p+1)},$$

where $0 < s < 1$ is a safety factor.

Ex 7.11

1. Assume $\|\boldsymbol{e}_{\text{old}}\| \approx Ch_{\text{old}}^{p+1}$.

2. Solve $Ch_{\text{new}}^{p+1} \approx \varepsilon$.

3. Eliminate C using the old error estimate.

4. Explain why production codes also limit the maximum growth and shrinkage of h.

Remark 7.5 **(RK45)** Dormand-Prince RK45 is a common embedded explicit pair. It is a good default for smooth nonstiff IVPs, not for stiff systems.

7.1.7 Linear Multistep Methods

Def 7.20 **(Linear Multistep Method)** A k-step linear multistep method has the form

$$\sum_{j=0}^{k} \alpha_j \boldsymbol{y}_{n+j} = h \sum_{j=0}^{k} \beta_j \boldsymbol{f}(t_{n+j}, \boldsymbol{y}_{n+j}).$$

It reuses previous solution values and previous right-hand side evaluations.

Def 7.21 **(Adams-Bashforth 2)** The two-step Adams-Bashforth method is explicit:

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \frac{h}{2}\left(3\boldsymbol{f}(t_n, \boldsymbol{y}_n) - \boldsymbol{f}(t_{n-1}, \boldsymbol{y}_{n-1})\right).$$

It has global order 2.

Def 7.22 **(Adams-Moulton 2)** The two-step Adams-Moulton method is implicit:

$$\boldsymbol{y}_{n+1} = \boldsymbol{y}_n + \frac{h}{2}\left(\boldsymbol{f}(t_{n+1}, \boldsymbol{y}_{n+1}) + \boldsymbol{f}(t_n, \boldsymbol{y}_n)\right).$$

This is the trapezoidal method from Def 7.10.

Def 7.23 **(BDF2)** The second-order backward differentiation formula is

$$\frac{3\boldsymbol{y}_{n+1} - 4\boldsymbol{y}_n + \boldsymbol{y}_{n-1}}{2h} = \boldsymbol{f}(t_{n+1}, \boldsymbol{y}_{n+1}).$$

BDF methods are standard implicit methods for stiff IVPs.

Def 7.24 **(Zero-Stability)** A linear multistep method is zero-stable if perturbations in starting values remain uniformly bounded as $h \to 0$ on a fixed interval. Algebraically, the roots of

$$\rho(\xi) = \sum_{j=0}^{k} \alpha_j \xi^j$$

must satisfy $|\xi| \le 1$, and roots on the unit circle must be simple.

Thm 7.10 **(Dahlquist Equivalence Theorem)** For a consistent linear multistep method applied to a well-posed IVP, convergence is equivalent to zero-stability.

Ex 7.12

1. For AB2, write the polynomial $\rho(\xi)$.
2. Check the root condition in Def 7.24.
3. Repeat for BDF2.
4. Explain why consistency alone is not enough for a multistep method.

Remark 7.6 **(BDF for stiffness)** BDF methods trade implicit linear or nonlinear solves for much larger stable time steps on stiff decay problems. Variable-order BDF methods are used in many stiff ODE codes.

7.1.8 Systems and Eigenvalues

Def 7.25 **(First-Order System Form)** A higher-order scalar equation can be rewritten as a first-order system. For example,

$$q'' = F(q, q', t)$$

becomes

$$q' = v, \qquad v' = F(q, v, t).$$

Thm 7.11 **(Linear Constant-Coefficient System)** For

$$\boldsymbol{y}' = \mathbf{A}\boldsymbol{y}, \qquad \boldsymbol{y}(0) = \boldsymbol{y}_0,$$

the exact solution is

$$\boldsymbol{y}(t) = e^{t\mathbf{A}}\boldsymbol{y}_0.$$

If $\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^{-1}$, then

$$\boldsymbol{y}(t) = \mathbf{V}e^{t\mathbf{D}}\mathbf{V}^{-1}\boldsymbol{y}_0.$$

Each eigenvalue controls one modal growth, decay, or oscillation rate.

Ex 7.13

1. Differentiate $e^{t\mathbf{A}}\boldsymbol{y}_0$ to verify the first formula in Thm 7.11.
2. Substitute $\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^{-1}$ into the matrix exponential.
3. For $\lambda = a + ib$, interpret $e^{\lambda t}$ in terms of growth and oscillation.
4. Explain why eigenvalues of the Jacobian predict local stiffness for nonlinear systems.

Remark 7.7 **(Jacobian spectrum)** Near a state $\boldsymbol{y}_*$, nonlinear dynamics are locally approximated by $\boldsymbol{e}' = \mathbf{J}_f(\boldsymbol{y}_*)\boldsymbol{e}$. Large negative eigenvalues produce fast stable transients. Eigenvalues near the imaginary axis produce oscillation or slow decay.

7.1.9 Hamiltonian and Symplectic Methods

Def 7.26 **(Hamiltonian System)** For position $\boldsymbol{q}$ and velocity or momentum $\boldsymbol{v}$, a separable Hamiltonian has

$$H(\boldsymbol{q},\boldsymbol{v}) = \frac{1}{2}\|\boldsymbol{v}\|^2 + V(\boldsymbol{q}), \qquad \boldsymbol{q}' = \boldsymbol{v}, \qquad \boldsymbol{v}' = \mathbf{F}(\boldsymbol{q}) = -\nabla V(\boldsymbol{q}).$$

The exact flow conserves H.

Def 7.27 **(Velocity Verlet)** For the Hamiltonian system in Def 7.26, Velocity Verlet is

$$\begin{aligned}
\boldsymbol{v}_{n+1/2} &= \boldsymbol{v}_n + \frac{h}{2}\mathbf{F}(\boldsymbol{q}_n),\\
\boldsymbol{q}_{n+1} &= \boldsymbol{q}_n + h\boldsymbol{v}_{n+1/2},\\
\boldsymbol{v}_{n+1} &= \boldsymbol{v}_{n+1/2} + \frac{h}{2}\mathbf{F}(\boldsymbol{q}_{n+1}).
\end{aligned}$$

It is second order, time reversible, and symplectic.

Thm 7.12 **(Harmonic Oscillator Energy Test)** For $q' = v$, $v' = -q$, Forward Euler maps the energy

$$E(q,v) = \frac{1}{2}(q^2 + v^2)$$

to $(1+h^2)E(q,v)$ after one step. Thus Forward Euler has artificial energy growth on this oscillator.

Proof *Proof.* Forward Euler gives

$$q_{n+1} = q_n + hv_n, \qquad v_{n+1} = v_n - hq_n.$$

Therefore

$$q_{n+1}^2 + v_{n+1}^2 = (q_n + hv_n)^2 + (v_n - hq_n)^2 = (1+h^2)(q_n^2 + v_n^2).$$

Multiplying by $1/2$ proves the claim. □

Ex 7.14

1. Apply Velocity Verlet to $q' = v$, $v' = -q$.
2. Show that reversing the step size from h to $-h$ undoes one Verlet step.
3. Compare long-time energy behavior of Forward Euler, RK4, and Velocity Verlet numerically.

Remark 7.8 **(Symplectic accuracy)** For long Hamiltonian simulations, preserving phase-space geometry can matter more than minimizing one-step error. Symplectic methods usually keep the energy error bounded and oscillatory over long times.

7.1.10 Method Choice

Def 7.28 **(Common Solver Regimes)**

Problem	Typical method
Smooth nonstiff IVP	Adaptive explicit RK, such as RK45
High-accuracy nonstiff IVP	Higher-order adaptive explicit RK
Stiff IVP	Implicit BDF or implicit Runge-Kutta
Hamiltonian long-time simulation	Symplectic method, such as Verlet
Large stiff system	Implicit method with sparse Jacobian solves

Remark 7.9 **(SciPy interface)** In `scipy.integrate.solve_ivp`, methods such as `RK45` and `DOP853` are explicit nonstiff solvers. Methods such as `BDF` and `Radau` are intended for stiff problems and benefit from Jacobian information.

Ex 7.15

1. For $y' = -y$, compare Forward Euler, RK4, and RK45 on accuracy versus function evaluations.
2. For the stiff example in Ex 7.8, compare RK45 and BDF using the same tolerance.
3. For a sparse semidiscrete heat equation $\boldsymbol{y}' = \mathbf{A}\boldsymbol{y}$, identify why Jacobian sparsity matters.
4. For the harmonic oscillator, compare final-time error and energy error for RK4 and Velocity Verlet.

8 Applications

8.1 Principal Component Analysis (PCA)

Optimal linear dimensionality reduction via variance maximization.

8.1.1 Definitions and Preprocessing

Def 8.1 **(Data Centering)** Data matrix $\mathbf{X} \in \mathbb{R}^{n\times p}$ (n samples, p features) must be **centered**:

$$X_{ij} \leftarrow X_{ij} - \bar{x}_j, \quad \bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} X_{ij}.$$

Throughout this section, $\mathbf{X}$ is assumed centered.

Def 8.2 **(Sample Covariance Matrix)** The matrix $\mathbf{S} = \frac{1}{n-1}\mathbf{X}^T\mathbf{X} \in \mathbb{R}^{p\times p}$.

- **Diagonal:** S_{jj} is the variance of feature j.
- **Off-diagonal:** S_{ij} is the covariance between features i and j.
- **Property:** $\mathbf{S}$ is symmetric positive semidefinite (SPSD).

Ex 8.1

1. Show that centering makes each column of $\mathbf{X}$ have mean zero.
2. Prove that the diagonal entries of $\mathbf{S}$ are sample variances.
3. Prove that $\mathbf{S}$ is symmetric positive semidefinite by computing $\boldsymbol{z}^T\mathbf{S}\boldsymbol{z}$.

8.1.2 PCA via SVD

Thm 8.1 **(Principal Directions and Scores)** Let $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ be the SVD of the centered data matrix.

- **Principal Directions (Loadings):** Columns of $\mathbf{V}$ (eigenvectors of $\mathbf{S}$).
- **Principal Components (Scores):** $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$.
- **Variances:** $\lambda_i = \mathrm{Var}(\boldsymbol{z}_i) = \frac{\sigma_i^2}{n-1}$.

Proof *Proof.* Since $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$,

$$\mathbf{S} = \frac{1}{n-1}\mathbf{X}^T\mathbf{X} = \mathbf{V}\left(\frac{\mathbf{\Sigma}^2}{n-1}\right)\mathbf{V}^T.$$

Thus the columns of $\mathbf{V}$ diagonalize the covariance matrix. The scores satisfy $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$, so their sample covariance is diagonal with entries $\sigma_i^2/(n-1)$. □

Remark 8.1 **(PCA stability)** Never form $\mathbf{X}^T\mathbf{X}$ explicitly for PCA. Squaring the data matrix doubles the condition number ($\kappa(\mathbf{S}) = \kappa(\mathbf{X})^2$). Compute PCA directly via the SVD of $\mathbf{X}$.

Ex 8.2

1. Starting from $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, compute $\mathbf{S} = (n-1)^{-1}\mathbf{X}^T\mathbf{X}$.
2. Show that the columns of $\mathbf{V}$ are eigenvectors of $\mathbf{S}$.
3. Show that the corresponding eigenvalues are $\lambda_i = \sigma_i^2/(n-1)$.
4. Compute the score matrix $\mathbf{Z} = \mathbf{X}\mathbf{V}$ and prove that $\mathrm{Cov}(\mathbf{Z})$ is diagonal.
5. Use Ex 4.6 to explain the stability warning above.

8.1.3 Variance and Dimensionality Reduction

Def 8.3 **(Proportion of Variance Explained (PVE))** For the first k components:

$$\text{PVE}_k = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{p} \sigma_i^2}.$$

Thm 8.2 **(Optimal Approximation)** The rank-k matrix

$$\mathbf{X}_k = \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$$

is the optimal k-dimensional linear approximation of the data in the Frobenius norm, by the Eckart-Young theorem.

Ex 8.3

1. Use Thm 4.8 to prove Thm 8.2.
2. Show that $\|\mathbf{X} - \mathbf{X}_k\|_F^2 = \sum_{i>k} \sigma_i^2$.
3. Show that $\text{PVE}_k = 1 - \|\mathbf{X} - \mathbf{X}_k\|_F^2 / \|\mathbf{X}\|_F^2$.
4. Explain why a PVE threshold is a modeling choice, not a theorem.

8.1.4 Implementation Details

Remark 8.2 **(Standardization)** If features have different units, PCA will be dominated by large-scale features. Standardize by dividing each centered column by its standard deviation: $X_{ij} \leftarrow (X_{ij} - \bar{x}_j)/s_j$.

Remark 8.3 **(Interpretation)** Principal components are variance directions, not causal factors. A large loading identifies a direction of variation in the data matrix; it does not by itself explain why that variation occurs.

Def 8.4 **(Kernel PCA (High-Dimensional Case))** When $p \gg n$, work with the $n \times n$ Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^T$. Principal directions are recovered via $\boldsymbol{v}_i = \mathbf{X}^T \boldsymbol{u}_i / \sigma_i$.

8.1.5 Exercises

Ex 8.4

1. Center $\mathbf{X} = \begin{pmatrix} 2 & 1 \\ -1 & 3 \\ -1 & -4 \end{pmatrix}$ and compute $\mathbf{S}$. Find the PVE for $k = 1$.

2. Prove that PC scores $\boldsymbol{z}_i, \boldsymbol{z}_j$ are uncorrelated for $i \neq j$ using Ex 8.2.

3. Load the Iris dataset. Plot the data in the basis of the first two PCs. Do species cluster?

4. Verify that the sum of variances $\sum \lambda_i$ equals the total variance $\mathrm{Tr}(\mathbf{S})$.

8.2 Spectral Graph Theory and PageRank

Graph algorithms become numerical linear algebra after choosing matrix conventions. Undirected graphs lead to symmetric Laplacians. Directed ranking problems lead to stochastic matrices and eigenvectors.

8.2.1 Graph Matrices

Remark 8.4 **(Convention)** For undirected graphs, $\mathbf{A}$ is symmetric. For directed random walks in this section, probability vectors are columns and P_{ij} is the probability of moving from node j to node i.

Def 8.5 **(Adjacency and Degree Matrices)** For a graph $G = (V, E)$ with n vertices:

- **Adjacency A:** $A_{ij} = w_{ij}$ if $(i, j) \in E$, else 0.
- **Degree D:** Diagonal matrix with $D_{ii} = d_i = \sum_j w_{ij}$.

Def 8.6 **(Graph Laplacian)** Defined as $\mathbf{L} = \mathbf{D} - \mathbf{A}$.

- **Discrete Derivative:** $(\mathbf{L}\boldsymbol{f})_i = \sum_{j \sim i} w_{ij}(f_i - f_j)$.
- **Quadratic Form:** $\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = \sum_{(i,j) \in E} w_{ij}(x_i - x_j)^2$, with each undirected edge counted once.

Prop 8.3 **(Laplacian Positive Semidefinite)** For an undirected weighted graph with nonnegative weights, $\mathbf{L}$ is symmetric positive semidefinite and $\mathbf{L1} = \mathbf{0}$.

Proof *Proof.* Symmetry follows from $w_{ij} = w_{ji}$. Using the quadratic form in Def 8.6,

$$\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = \sum_{(i,j) \in E} w_{ij}(x_i - x_j)^2 \geq 0.$$

Thus $\mathbf{L}$ is positive semidefinite. If $\boldsymbol{x} = \mathbf{1}$, every difference $x_i - x_j$ is zero, and direct substitution gives $\mathbf{L1} = \mathbf{0}$. □

Example **(Three-node path)** For $1 - 2 - 3$,

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \qquad \mathbf{L} = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}.$$

The zero eigenvector is $\mathbf{1}$ because constant signals have no graph variation.

Thm 8.4 **(Spectral Properties)** For a connected graph with $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$:

1. $\lambda_1 = 0$ with eigenvector $\mathbf{1}$.
2. **Multiplicity of 0:** Equals the number of connected components.
3. **Fiedler Value (λ_2):** Measures algebraic connectivity. $\lambda_2 > 0$ iff graph is connected.

Ex 8.5

1. Use Def 8.6 to show $\mathbf{L1} = \mathbf{0}$.
2. Derive $\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2$ for edges counted once.
3. Conclude that $\mathbf{L}$ is positive semidefinite for an undirected graph.
4. Show that if a graph has two connected components, then there are two independent vectors in $\mathcal{N}(\mathbf{L})$.

Def 8.7 **(Normalized Laplacians)**

- **Symmetric:** $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2}$. (Eigenvalues in $[0, 2]$).
- **Random Walk:** $\mathbf{L}_{\text{rw}} = \mathbf{I} - \mathbf{A}\mathbf{D}^{-1}$ under the column-stochastic convention.

Remark 8.5 **(Degree normalization)** The unnormalized Laplacian can let high-degree vertices dominate cuts. Normalized Laplacians measure variation relative to vertex degree.

8.2.2 Spectral Clustering

Thm 8.5 **(Spectral Relaxation)** Minimizing a graph cut (RatioCut) is NP-hard. Spectral methods relax this to an eigenvector problem:

1. Compute k smallest eigenvectors of $\mathbf{L}$ (starting from $\boldsymbol{v}_2$).
2. Embed vertices in $\mathbb{R}^k$ using these eigenvectors.
3. Cluster embedding via k-means.

8.2.3 Random Walks and PageRank

Def 8.8 **(Random Walk Matrix)** Transition matrix $\mathbf{P} = \mathbf{A}\mathbf{D}^{-1}$ under the column-stochastic convention.

- $P_{ij} = w_{ij}/d_j$ is the probability of moving from j to i.
- **Stationary Distribution:** $\boldsymbol{\pi}$ such that $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$. For undirected graphs, $\pi_i = d_i / \sum d_j$.

Ex 8.6

1. Show that the columns of $\mathbf{P} = \mathbf{A}\mathbf{D}^{-1}$ sum to one.
2. For the path graph $1 - 2 - 3$, compute $\mathbf{P}$.
3. Verify that $\pi_i = d_i / \sum_j d_j$ satisfies $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$.

Def 8.9 **(PageRank and Google Matrix)** Models a random surfer on a directed web graph with teleportation:

$$\mathbf{G} = d\mathbf{S} + \frac{1-d}{N}\mathbf{J},$$

- d is the damping factor (standard: 0.85).
- $\mathbf{S}$ is column-stochastic and handles dangling nodes.
- $\mathbf{J}$ is the all-ones matrix (teleportation).

Computation: PageRank vector $\mathbf{r}$ is the dominant eigenvector of $\mathbf{G}$, computed via power iteration.

Remark 8.6 **(Dangling nodes)** A page with no outgoing links gives a zero column before normalization. The standard fix replaces that column by the uniform probability vector before forming $\mathbf{S}$.

8.2.4 Exercises

Ex 8.7

1. Construct $\mathbf{L}$ for a 3-vertex path graph. Find its eigenvalues by hand.
2. Show that $\mathbf{L}\mathbf{1} = \mathbf{0}$ for any graph.
3. Prove $\mathbf{L}$ is positive semidefinite using the quadratic form.
4. Implement PageRank power iteration for a 10-node random graph.

8.3 Linear Operators and the Poisson Equation

Differential equations become matrix problems after discretization. The finite matrix should preserve the main structure of the operator: sparsity, symmetry, definiteness, spectrum, and conditioning.

8.3.1 Differential Operators and Discretization

Def 8.10 **(Linear Operator)** Map $\mathcal{A} : X \to Y$ between function spaces satisfying:

$$\mathcal{A}(\alpha u + v) = \alpha \mathcal{A}u + \mathcal{A}v.$$

Matrices are finite-dimensional operators; $\frac{d}{dx}$ and ∇^2 are infinite-dimensional.

Def 8.11 **(Dirichlet Poisson Problem)** The one-dimensional Poisson problem with homogeneous Dirichlet boundary data is

$$-u''(x) = f(x), \qquad 0 < x < 1, \qquad u(0) = u(1) = 0.$$

The boundary conditions are part of the problem data.

Def 8.12 **(Grid Function)** For n interior grid points, let $h = 1/(n+1)$ and $x_i = ih$, $i = 1, \ldots, n$. A grid function is the vector

$$\boldsymbol{u} = (u_1, \ldots, u_n)^T, \qquad u_i \approx u(x_i).$$

The boundary values are fixed separately by the boundary conditions.

Def 8.13 **(Finite Difference Stencil)** Approximating $u''(x)$ on a grid with spacing h:

$$u''(x_k) \approx \frac{u(x_{k-1}) - 2u(x_k) + u(x_{k+1})}{h^2}.$$

Accuracy: Second-order $O(h^2)$ via Taylor expansion.

Ex 8.8

1. Taylor expand $u(x+h)$ and $u(x-h)$ about x.
2. Add the two expansions.
3. Solve for $u''(x)$.
4. Identify the leading error term and prove the $O(h^2)$ claim in Def 8.13.

Def 8.14 **(1-D Discrete Laplacian)** Discretizing $-d^2/dx^2$ on $[0,1]$ with $u(0) = u(1) = 0$ yields the tridiagonal system $\mathbf{Tu} = \mathbf{f}$:

$$\mathbf{T} = \frac{1}{h^2}\text{tridiag}(-1, 2, -1) \in \mathbb{R}^{n\times n}.$$

Remark 8.7 **(Sparsity)** Each row of $\mathbf{T}$ uses only the grid point and its two neighbors. This locality is why finite difference matrices are sparse.

Thm 8.6 **(Spectrum Convergence)** Eigenvalues of $\mathbf{T}$ are $\lambda_k = \frac{4}{h^2}\sin^2(\frac{k\pi}{2(n+1)})$. As $n \to \infty$, $\lambda_k \to k^2\pi^2$ (eigenvalues of continuous $-d^2/dx^2$).

Ex 8.9

1. Let $(\boldsymbol{v}_k)_j = \sin(jk\pi h)$ with $h = 1/(n+1)$.
2. Compute the j-th entry of $\mathbf{T}\boldsymbol{v}_k$.
3. Use $\sin(A-B) + \sin(A+B) = 2\sin A\cos B$.
4. Use $1 - \cos\theta = 2\sin^2(\theta/2)$.
5. Derive the eigenvalue formula in Thm 8.6.
6. Let $h \to 0$ with fixed k and recover $k^2\pi^2$.

Thm 8.7 **(Discrete Poisson Structure)** The matrix $\mathbf{T}$ in Def 8.14 is sparse, symmetric positive definite, and has condition number

$$\kappa_2(\mathbf{T}) = O(h^{-2}).$$

Proof *Proof.* Sparsity and symmetry are immediate from the tridiagonal stencil. By Thm 8.6, all eigenvalues are positive because $1 \le k \le n$ and $0 < k\pi/(2(n+1)) < \pi/2$. Thus $\mathbf{T}$ is SPD. Also

$$\lambda_{\min} \sim \pi^2, \qquad \lambda_{\max} \sim \frac{4}{h^2},$$

so $\kappa_2(\mathbf{T}) = \lambda_{\max}/\lambda_{\min} = O(h^{-2})$. □

Ex 8.10

1. Use the stencil to count the number of nonzeros in $\mathbf{T}$.
2. Use the quadratic form $\boldsymbol{u}^T\mathbf{T}\boldsymbol{u}$ to show positive definiteness for nonzero interior grid functions.
3. Use Thm 8.6 to estimate $\lambda_{\min}$ and $\lambda_{\max}$.
4. Derive the condition number estimate in Thm 8.7.

8.3.2 Operator Norms and Conditioning

Def 8.15 **(Boundedness)** An operator is **bounded** if $\|\mathcal{A}u\|_Y \leq C\|u\|_X$ for some $C < \infty$.

- Matrices are always bounded.
- Differential operators are **unbounded**.

Remark 8.8 **Numerical Consequence:** Because the continuous Laplacian is unbounded, its discretization $\mathbf{T}$ becomes increasingly ill-conditioned as $h \to 0$:

$$\kappa(\mathbf{T}) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx \frac{4/h^2}{\pi^2} = O(h^{-2}).$$

Refining the grid $2\times$ quadruples the condition number.

Remark 8.9 **(Solver consequence)** Dense Gaussian elimination ignores sparsity. For Poisson matrices, sparse Cholesky, CG with preconditioning, multigrid, or FFT-based solvers are the natural algorithms, depending on geometry and boundary conditions.

8.3.3 Exercises

Ex 8.11

1. Build $\mathbf{T}$ for $n = 50$ and plot its eigenvalues vs. $k^2\pi^2$ using Thm 8.6.
2. Show that $\kappa(\mathbf{T}) \to \infty$ as $n \to \infty$ using Thm 8.7.
3. Implement a 2-D Poisson solver on a unit square. Verify $O(N^{3/2})$ cost for direct sparse solve.
4. Prove the differentiation operator is unbounded on $L^2[0, 1]$ using $u_k(x) = \sin(k\pi x)$.

8.4 Fourier Analysis as Change of Basis

Fourier analysis provides a framework for representing functions and data in terms of frequency components. In engineering, this transformation is critical for signal processing, image compression, and solving differential equations where the Laplacian operator is diagonalized by the Fourier basis.

8.4.1 Discrete Fourier Transform (DFT)

Def 8.16 **(Periodic Grid Signal)** A discrete signal $\boldsymbol{x} \in \mathbb{C}^n$ is interpreted as one period of a periodic sequence:

$$x_{j+n} = x_j.$$

The DFT represents this signal in the basis of discrete complex exponentials.

Def 8.17 **(DFT Matrix)** Unitary matrix $\mathbf{F} \in \mathbb{C}^{n\times n}$ with entries:

$$F_{jk} = \frac{1}{\sqrt{n}}\omega_n^{jk}, \quad \omega_n = e^{-2\pi i/n}.$$

- **Unitary Property:** $\mathbf{F}^*\mathbf{F} = \mathbf{I}$. Orthonormal columns.
- **Inverse DFT:** $\mathbf{F}^{-1} = \mathbf{F}^*$.

Thm 8.8 **(Parseval's Theorem)** The DFT is an isometry: $\|\mathbf{F}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2$. Total energy is preserved between time and frequency domains.

Proof *Proof.* Since $\mathbf{F}$ is unitary, $\mathbf{F}^*\mathbf{F} = \mathbf{I}$. Hence

$$\|\mathbf{F}\boldsymbol{x}\|_2^2 = (\mathbf{F}\boldsymbol{x})^*(\mathbf{F}\boldsymbol{x}) = \boldsymbol{x}^*\mathbf{F}^*\mathbf{F}\boldsymbol{x} = \boldsymbol{x}^*\boldsymbol{x} = \|\boldsymbol{x}\|_2^2.$$

Taking square roots gives the claim. □

Ex 8.12

1. Use Def 8.17 to compute the (j, ℓ) entry of $\mathbf{F}^*\mathbf{F}$.
2. Evaluate the finite geometric sum that appears.
3. Conclude $\mathbf{F}^*\mathbf{F} = \mathbf{I}$.
4. Explain why this is the finite Fourier analog of orthonormal coordinates.

Def 8.18 **(Frequency Bins)** For samples on $[0, 2\pi)$, the integer frequency attached to index k is usually ordered as

$$0, 1, 2, \ldots, \lfloor n/2 \rfloor, -\lfloor (n-1)/2 \rfloor, \ldots, -1.$$

This is the ordering returned by `np.fft.fftfreq` after scaling by n.

Remark 8.10 **(Normalization convention)** Some libraries put the factor $1/n$ in the inverse transform instead of using the unitary factor $1/\sqrt{n}$ in both directions. The formulas change by constants, but the frequency content is the same.

Def 8.19 **(Nyquist Frequency and Aliasing)** On n equally spaced samples of a 2π-periodic signal, frequencies that differ by multiples of n are indistinguishable:

$$e^{i(k+n)x_j} = e^{ikx_j}.$$

The largest resolvable unsigned frequency is the Nyquist frequency, approximately $n/2$ modes per period.

Ex 8.13

1. Let $x_j = 2\pi j/n$. Prove $e^{i(k+n)x_j} = e^{ikx_j}$ for every grid point.
2. For $n = 8$, list the frequency bin ordering from Def 8.18.
3. Sample $\sin(7x)$ on $n = 8$ points and identify its aliased frequency.

8.4.2 Convolution and Circulant Matrices

Def 8.20 **(Circulant Matrix)** Matrix $\mathbf{C}$ where each row is a cyclic shift of the first column $\boldsymbol{c}$.

- **Diagonalization:** Every circulant matrix is diagonalized by the DFT: $\mathbf{C} = \mathbf{F}^* \text{diag}(\mathbf{F}\boldsymbol{c})\mathbf{F}$.
- **Eigenvalues:** The eigenvalues of $\mathbf{C}$ are the DFT coefficients of its first column.

Thm 8.9 **(Convolution Theorem)** Circular convolution $\boldsymbol{z} = \boldsymbol{c} * \boldsymbol{x}$ is equivalent to pointwise multiplication in frequency:

$$\mathbf{F}(\boldsymbol{c} * \boldsymbol{x}) = (\mathbf{F}\boldsymbol{c}) \odot (\mathbf{F}\boldsymbol{x}).$$

Complexity: Reduced from $O(n^2)$ (direct) to $O(n \log n)$ (FFT).

Example **(Circular convolution for $n = 3$)** If $\boldsymbol{c} = (c_0, c_1, c_2)^T$, then

$$\boldsymbol{c} * \boldsymbol{x} = \begin{pmatrix} c_0 & c_2 & c_1 \\ c_1 & c_0 & c_2 \\ c_2 & c_1 & c_0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix}.$$

The wraparound terms are a consequence of the periodic convention in Def 8.16.

8.4.3 The Fast Fourier Transform (FFT)

Thm 8.10 **(Cooley-Tukey Algorithm)** Computes the DFT in $O(n \log n)$ flops for $n = 2^p$.

- **Mechanism:** Recursively splits n-point DFT into two $n/2$-point DFTs (even/odd indices).
- **NumPy:** Use `np.fft.fft` and `np.fft.ifft`.

Ex 8.14

1. Split the DFT sum into even and odd input indices.
2. Show that each part is an $n/2$-point DFT multiplied by phase factors.
3. Write the recurrence $T(n) = 2T(n/2) + O(n)$.
4. Solve the recurrence to obtain the cost in Thm 8.10.

Remark 8.11 **Spectral Differentiation:** For smooth periodic signals, differentiate by multiplying frequency components $\hat{x}_k$ by ik. Provides exponential accuracy compared to polynomial accuracy of finite differences.

Thm 8.11 **(Fourier Differentiation Rule)** For a smooth 2π-periodic function with Fourier series

$$f(x) = \sum_k \hat{f}_k e^{ikx},$$

the derivative has coefficients

$$\widehat{f'}_k = ik\hat{f}_k.$$

8.4.4 Exercises

Ex 8.15

1. Write out the 4×4 DFT matrix explicitly. Verify it is unitary using Ex 8.12.
2. Solve the periodic Poisson equation $-u'' = f$ via FFT. (*Hint: Laplacian is circulant*).
3. Compare the speed of `np.fft.fft(x)` vs. `F @ x` for $n = 4096$.
4. Implement spectral differentiation for $f(x) = \sin(x)$ using Thm 8.11 and check error vs. n.

8.5 Markov Chains and Stochastic Matrices

Markov chains model random processes whose next state depends only on the current state. With the column convention used here, probability vectors are columns and the update is $\boldsymbol{x}^{(k+1)} = \mathbf{P}\boldsymbol{x}^{(k)}$.

8.5.1 Definitions

Def 8.21 **(Probability Vector)** A vector $\boldsymbol{x} \in \mathbb{R}^n$ is a probability vector if

$$x_i \geq 0, \qquad \sum_{i=1}^{n} x_i = 1.$$

The entry x_i is the probability of state i.

Def 8.22 **(Column-Stochastic Matrix)** Matrix $\mathbf{P} \in \mathbb{R}^{n\times n}$ where:

1. $P_{ij} \geq 0$.

2. $\sum_i P_{ij} = 1$ (Columns sum to 1).

Remark 8.12 **(Column convention)** Column j of $\mathbf{P}$ contains the probabilities of moving from state j to all possible next states. Some texts use row-stochastic matrices and row probability vectors. The two conventions are transposes of each other.

Thm 8.12 **(Spectrum of a Stochastic Matrix)** If $\mathbf{P}$ is column-stochastic, then $\lambda = 1$ is an eigenvalue and every eigenvalue satisfies $|\lambda| \leq 1$.

Proof *Proof.* Since $\mathbf{1}^T\mathbf{P} = \mathbf{1}^T$, 1 is a left eigenvalue, hence an eigenvalue. To bound the spectrum, apply the row-stochastic argument to $\mathbf{P}^T$: if $\mathbf{P}^T\boldsymbol{v} = \lambda\boldsymbol{v}$ and $|v_j| = \|\boldsymbol{v}\|_\infty$, then

$$|\lambda|\,|v_j| = \left|\sum_i P_{ij}v_i\right| \leq \sum_i P_{ij}|v_i| \leq |v_j|.$$

Thus $|\lambda| \leq 1$. □

Ex 8.16

1. Show that $\mathbf{1}^T\mathbf{P} = \mathbf{1}^T$ for a column-stochastic matrix.
2. Conclude that 1 is a left eigenvalue of $\mathbf{P}$, hence an eigenvalue.
3. Prove $|\lambda| \leq 1$ first for a row-stochastic matrix by taking the largest-magnitude entry of an eigenvector.
4. Apply the previous step to $\mathbf{P}^T$ to prove Thm 8.12.

Def 8.23 **(Markov Chain)** Probability distributions $\boldsymbol{x}^{(k)}$ evolving by $\boldsymbol{x}^{(k+1)} = \mathbf{P}\boldsymbol{x}^{(k)}$.

- After k steps: $\boldsymbol{x}^{(k)} = \mathbf{P}^k\boldsymbol{x}^{(0)}$.
- Long-run behavior is governed by the eigendecomposition of $\mathbf{P}$.

Def 8.24 **(Stationary Distribution)** A probability vector $\boldsymbol{\pi}$ such that $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$.

- Right eigenvector of $\mathbf{P}$ for $\lambda = 1$.
- Represents the equilibrium state of the chain.

Example **(Two-state chain)** Let

$$\mathbf{P} = \begin{pmatrix} 1-a & b \\ a & 1-b \end{pmatrix}, \qquad 0 < a, b < 1.$$

Solving $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$ with $\pi_1 + \pi_2 = 1$ gives

$$\boldsymbol{\pi} = \frac{1}{a+b}\begin{pmatrix} b \\ a \end{pmatrix}.$$

Def 8.25 **(Reducible and Periodic Chains)** A chain is reducible if some state cannot be reached from another state. A state has period $d > 1$ if returns to that state occur only at times divisible by d. A primitive chain is irreducible and aperiodic.

Example **(Two failure modes)** The identity matrix is reducible: every state is absorbing. The matrix

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

is irreducible but periodic with period 2, so the distribution oscillates instead of converging.

8.5.2 Convergence and Mixing

Thm 8.13 **(Perron-Frobenius for Chains)** If $\mathbf{P}$ is **irreducible** (all states reachable) and **primitive** (no cycles), then:

1. $\lambda_1 = 1$ is simple and unique.
2. $|\lambda_i| < 1$ for all other eigenvalues.
3. $\lim_{k\to\infty} \mathbf{P}^k \boldsymbol{x}^{(0)} = \boldsymbol{\pi}$ for any initial state.

Def 8.26 **(Spectral Gap)** The gap $\delta = 1 - |\lambda_2|$ controls convergence speed.

Thm 8.14 **(Spectral Mixing Heuristic)** For a diagonalizable primitive chain, the distance to stationarity is dominated asymptotically by the second eigenvalue:

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{\pi}\| \approx C|\lambda_2|^k.$$

Thus a larger spectral gap $\delta = 1 - |\lambda_2|$ means faster mixing, and the iteration count to reach tolerance ε scales like

$$k_\varepsilon \approx \frac{\log(1/\varepsilon)}{\delta}$$

when δ is small.

Ex 8.17

1. Write an initial distribution as $\boldsymbol{x}^{(0)} = \boldsymbol{\pi} + \sum_{i\geq 2} c_i \boldsymbol{v}_i$ using eigenvectors of $\mathbf{P}$.
2. Apply $\mathbf{P}^k$ and use $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$.
3. Identify the slowest-decaying term.
4. Use $\log(1-\delta) \approx -\delta$ to derive the mixing-time estimate in Thm 8.14.

8.5.3 Properties and Design

Def 8.27 **(Detailed Balance (Reversibility))** A chain is reversible if $\pi_j P_{ij} = \pi_i P_{ji}$ for all i, j.

Thm 8.15 **(Reversible Chains Have Real Spectrum)** If $\mathbf{P}$ satisfies detailed balance with stationary distribution $\boldsymbol{\pi}$ and $\mathbf{D}_{\boldsymbol{\pi}} = \operatorname{diag}(\boldsymbol{\pi})$, then

$$\mathbf{D}_{\boldsymbol{\pi}}^{-1/2} \mathbf{P} \mathbf{D}_{\boldsymbol{\pi}}^{1/2}$$

is symmetric. Therefore reversible chains have real eigenvalues.

Ex 8.18

1. Compute the (i, j) entry of $\mathbf{D}_{\boldsymbol{\pi}}^{-1/2} \mathbf{P} \mathbf{D}_{\boldsymbol{\pi}}^{1/2}$.
2. Use detailed balance from Def 8.27 to show that this entry equals the (j, i) entry.
3. Explain why similarity preserves eigenvalues.
4. Conclude Thm 8.15.

Remark 8.13 **(Design Tool: Metropolis-Hastings)** Construct a reversible chain for a desired $\boldsymbol{\pi}$ by accepting moves with probability $a = \min(1, \frac{\pi_{\text{new}} Q(\text{old}|\text{new})}{\pi_{\text{old}} Q(\text{new}|\text{old})})$.

Thm 8.16 **(Metropolis-Hastings Detailed Balance)** Let Q_{ij} be a proposal probability from state j to state i. Define

$$A_{ij} = \min\left(1, \frac{\pi_i Q_{ji}}{\pi_j Q_{ij}}\right), \qquad i \neq j.$$

The transition probabilities $P_{ij} = Q_{ij} A_{ij}$ for $i \neq j$, with the remaining probability kept at state j, satisfy detailed balance with $\boldsymbol{\pi}$.

Proof *Proof.* For $i \neq j$,

$$\pi_j P_{ij} = \pi_j Q_{ij} \min\left(1, \frac{\pi_i Q_{ji}}{\pi_j Q_{ij}}\right) = \min(\pi_j Q_{ij}, \pi_i Q_{ji}).$$

The same calculation with i and j interchanged gives

$$\pi_i P_{ji} = \min(\pi_i Q_{ji}, \pi_j Q_{ij}).$$

Thus $\pi_j P_{ij} = \pi_i P_{ji}$. □

8.5.4 Exercises

Ex 8.19

1. Construct $\mathbf{P}$ for a 2-state chain with transition probabilities $\{0.1, 0.2\}$. Find $\boldsymbol{\pi}$.
2. Prove that $|\lambda| \leq 1$ for any stochastic matrix using Ex 8.16.
3. Simulate 100 steps of PageRank on a 5-node graph. Check convergence rate vs. λ_2 using Thm 8.14.
4. Find $\boldsymbol{\pi}$ for a random walk on K_4 (complete graph).
5. Give one reducible chain and one periodic irreducible chain. Explain which part of Thm 8.13 fails.
6. Verify detailed balance for a three-state Metropolis-Hastings chain using Thm 8.16.

8.6 Koopman Theory, DMD, and SINDy

Koopman analysis studies nonlinear dynamics through linear evolution of observables. DMD fits a finite-dimensional linear model from snapshots. SINDy fits sparse governing equations from a chosen function library.

8.6.1 Koopman Operator Theory

Def 8.28 **(Nonlinear Dynamics and Observables)** $\boldsymbol{x}^{(k+1)} = \mathbf{F}(\boldsymbol{x}^{(k)})$ on $\mathcal{M} \subseteq \mathbb{R}^n$.

- **Observable:** Scalar function $g : \mathcal{M} \to \mathbb{R}$.
- **Koopman Operator $\mathcal{K}$:** Linear operator acting on observables: $(\mathcal{K}g)(\boldsymbol{x}) = g(\mathbf{F}(\boldsymbol{x}))$.

Lifting: $\mathcal{K}$ is linear even if $\mathbf{F}$ is nonlinear, at the cost of being infinite-dimensional.

Prop 8.17 **(Koopman Linearity)** The Koopman operator in Def 8.28 is linear on observables:

$$\mathcal{K}(\alpha g + \beta h) = \alpha \mathcal{K} g + \beta \mathcal{K} h.$$

Proof *Proof.* For every state $\boldsymbol{x}$,

$$\begin{aligned}[\mathcal{K}(\alpha g + \beta h)](\boldsymbol{x}) &= (\alpha g + \beta h)(\mathbf{F}(\boldsymbol{x})) \\ &= \alpha g(\mathbf{F}(\boldsymbol{x})) + \beta h(\mathbf{F}(\boldsymbol{x})) \\ &= [\alpha \mathcal{K} g + \beta \mathcal{K} h](\boldsymbol{x}).\end{aligned}$$

Since this holds for every $\boldsymbol{x}$, the two observables are equal. □

Example **(Nonlinear map)** Let $x_{k+1} = x_k^2$. For observables $g_1(x) = x$ and $g_2(x) = x^2$,

$$(\mathcal{K}g_1)(x) = x^2 = g_2(x), \qquad (\mathcal{K}g_2)(x) = x^4.$$

The state map is nonlinear, but the operator $g \mapsto g \circ F$ is linear on the space of observables.

Def 8.29 **(Koopman Eigenfunctions)** Observable φ satisfying $\mathcal{K}\varphi = \mu\varphi$.

- **Temporal Evolution:** $\varphi(\boldsymbol{x}^{(k)}) = \mu^k \varphi(\boldsymbol{x}^{(0)})$.
- **Significance:** Provides a global linear coordinate system for nonlinear dynamics.

8.6.2 Dynamic Mode Decomposition (DMD)

Def 8.30 **(DMD Algorithm)** Finds the best linear fit $\mathbf{Y} \approx \mathbf{A}\mathbf{X}$ for snapshot matrices:

$$\mathbf{X} = [\boldsymbol{x}^{(0)} \dots \boldsymbol{x}^{(m-1)}], \quad \mathbf{Y} = [\boldsymbol{x}^{(1)} \dots \boldsymbol{x}^{(m)}].$$

1. Compute truncated SVD: $\mathbf{X} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^T$.
2. Construct $r \times r$ reduced operator: $\tilde{\mathbf{A}} = \mathbf{U}_r^T \mathbf{Y} \mathbf{V}_r \mathbf{\Sigma}_r^{-1}$.
3. DMD eigenvalues $\{\lambda_i\}$ and modes $\{\boldsymbol{\phi}_i\}$ are recovered from $\tilde{\mathbf{A}}$.

Thm 8.18 **(DMD Least Squares Model)** The best-fit linear map in Frobenius norm is

$$\mathbf{A}_* = \mathbf{Y}\mathbf{X}^\dagger.$$

The reduced operator in Def 8.30 is the projection of this map onto the rank-r left singular subspace of $\mathbf{X}$.

Ex 8.20

1. Use the least squares normal equations to derive $\mathbf{A}_* = \mathbf{Y}\mathbf{X}^\dagger$.
2. Substitute the truncated SVD $\mathbf{X} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^T$.
3. Show that $\mathbf{U}_r^T \mathbf{A}_* \mathbf{U}_r = \mathbf{U}_r^T \mathbf{Y} \mathbf{V}_r \mathbf{\Sigma}_r^{-1}$.
4. Interpret this as the small operator $\tilde{\mathbf{A}}$ in Def 8.30.

Remark 8.14 **Interpretation:** DMD decomposes data into spatiotemporal modes $\boldsymbol{\phi}_i$ with associated frequencies and growth rates. It is the data-driven approximation of the Koopman operator using the identity dictionary.

8.6.3 SINDy: Sparse Identification

Def 8.31 **(Sparse Regression for Dynamics)** Identifies governing equations $\dot{\boldsymbol{x}} = \boldsymbol{\Xi}^T \boldsymbol{\Theta}(\boldsymbol{x})$ where $\boldsymbol{\Theta}$ is a library of candidate functions (monomials, sin, etc.).

- **STLS Algorithm:** Sequential Thresholded Least Squares. Iteratively solves least squares and thresholds small coefficients to promote sparsity.
- **Outcome:** Parsimonious, interpretable models (e.g., recovering Lorenz equations).

Remark 8.15 **(DMD vs. SINDy)** DMD fits a linear time-advance map for snapshot data. SINDy fits a sparse differential equation in a chosen library. DMD returns modes and growth rates; SINDy returns equation coefficients.

Remark 8.16 **(Noisy derivatives)** SINDy is sensitive to derivative estimates. Smoothing, weak-form regression, or integral formulations are often needed when data are noisy.

8.6.4 Exercises

Ex 8.21

1. Prove Prop 8.17 without looking at the proof.
2. Implement DMD for a 2D damped oscillator. Verify the identified eigenvalues.
3. Build a polynomial library $\boldsymbol{\Theta}$ for $\boldsymbol{x} \in \mathbb{R}^2$ up to degree 2.
4. Use SINDy (via STLS) to identify the Lotka-Volterra predator-prey model from noisy data.
5. Compare DMD and SINDy on the same two-dimensional linear system. Which quantities should agree?